

Mathematical Foundations, Architecture Diagrams, and State Analysis of All Modern LLM Training Methods

Chief Strategist J

June 12, 2026

Colour Legend

	Frozen / base weights (not trained)
	Trainable components
	Data / activations
	Loss / gradients
	Merged / deployed outputs
	External models / judges / scorers
	Environment / external systems

Document structure: This reference covers **85 LLM training methods** organised into 14 parts. Each method includes: (1) the core mathematical objective, (2) a TikZ architecture diagram, and (3) a before/after state table.

Contents

I	Foundation Training Methods	3
1	Pretraining	3
1.1	Mathematical Foundation	3
1.2	Architecture Flow Diagram	3
1.3	State Transition Table	3
1.4	Python Implementation	3
2	Continued Pretraining (CPT)	5
2.1	Mathematical Foundation	5
2.2	Architecture Flow Diagram	5
2.3	State Transition Table	5
2.4	Python Implementation	6
3	Domain Adaptive Pretraining (DAPT)	7
3.1	Mathematical Foundation	7
3.2	Architecture Flow Diagram	7
3.3	State Transition Table	8
3.4	Python Implementation	8
4	Task Adaptive Pretraining (TAPT)	9
4.1	Mathematical Foundation	9
4.2	Architecture Flow Diagram	9
4.3	State Transition Table	9
4.4	Python Implementation	10
II	Supervised Fine-Tuning Methods	11

5	Supervised Fine-Tuning (SFT)	11
5.1	Mathematical Foundation	11
5.2	Architecture Flow Diagram	11
5.3	State Transition Table	11
5.4	Python Implementation	11
6	Instruction Tuning	13
6.1	Mathematical Foundation	13
6.2	Architecture Flow Diagram	13
6.3	State Transition Table	13
6.4	Python Implementation	13
7	Multi-Task Training	15
7.1	Mathematical Foundation	15
7.2	Architecture Flow Diagram	15
7.3	State Transition Table	15
7.4	Python Implementation	15
8	Chain-of-Thought SFT	17
8.1	Mathematical Foundation	17
8.2	Architecture Flow Diagram	17
8.3	State Transition Table	17
8.4	Python Implementation	17
9	Step-by-Step SFT	19
9.1	Mathematical Foundation	19
9.2	Architecture Flow Diagram	19
9.3	State Transition Table	19
9.4	Python Implementation	19
10	Process Supervision	21
10.1	Mathematical Foundation	21
10.2	Architecture Flow Diagram	21
10.3	State Transition Table	21
10.4	Python Implementation	21
III	Parameter-Efficient Fine-Tuning (PEFT)	23
11	Low-Rank Adaptation (LoRA)	23
11.1	Mathematical Foundation	23
11.2	Architecture Flow Diagram	23
11.3	State Transition Table	23
11.4	Python Implementation	24
12	Quantized LoRA (QLoRA)	25
12.1	Mathematical Foundation	25
12.2	Architecture Flow Diagram	25
12.3	State Transition Table	25
12.4	Python Implementation	25
13	Adaptive LoRA (AdaLoRA)	27
13.1	Mathematical Foundation	27
13.2	Architecture Flow Diagram	27
13.3	State Transition Table	27
13.4	Python Implementation	27
14	Weight-Decomposed LoRA (DoRA)	29
14.1	Mathematical Foundation	29
14.2	Architecture Flow Diagram	29
14.3	State Transition Table	29
14.4	Python Implementation	29

15 IA³ (Infused Adapter by Inhibiting and Amplifying Inner Activations)	31
15.1 Mathematical Foundation	31
15.2 Architecture Flow Diagram	31
15.3 State Transition Table	31
15.4 Python Implementation	31
16 Adapter Tuning	33
16.1 Mathematical Foundation	33
16.2 Architecture Flow Diagram	33
16.3 State Transition Table	33
16.4 Python Implementation	33
17 Prefix Tuning	34
17.1 Mathematical Foundation	34
17.2 Architecture Flow Diagram	34
17.3 State Transition Table	34
17.4 Python Implementation	34
18 Prompt Tuning	36
18.1 Mathematical Foundation	36
18.2 Architecture Flow Diagram	36
18.3 State Transition Table	36
18.4 Python Implementation	36
19 P-Tuning v2	37
19.1 Mathematical Foundation	37
19.2 Architecture Flow Diagram	37
19.3 State Transition Table	37
19.4 Python Implementation	37
IV Preference Alignment Methods	39
20 Reinforcement Learning from Human Feedback (RLHF)	39
20.1 Mathematical Foundation	39
20.2 Architecture Flow Diagram	39
20.3 State Transition Table	39
21 Direct Preference Optimization (DPO)	40
21.1 Mathematical Foundation	40
21.2 Architecture Flow Diagram	40
21.3 State Transition Table	40
22 Identity Preference Optimization (IPO)	41
22.1 Mathematical Foundation	41
22.2 Architecture Flow Diagram	41
22.3 State Transition Table	41
23 Odds Ratio Preference Optimization (ORPO)	42
23.1 Mathematical Foundation	42
23.2 Architecture Flow Diagram	42
23.3 State Transition Table	42
24 Kahneman-Tversky Optimization (KTO)	43
24.1 Mathematical Foundation	43
24.2 Architecture Flow Diagram	43
24.3 State Transition Table	43
25 Simple Preference Optimization (SimPO)	44
25.1 Mathematical Foundation	44
25.2 Architecture Flow Diagram	44
25.3 State Transition Table	44

26 Constrained Preference Optimization (CPO)	45
26.1 Mathematical Foundation	45
26.2 Architecture Flow Diagram	45
26.3 State Transition Table	45
27 Anchored Preference Optimization (APO)	46
27.1 Mathematical Foundation	46
27.2 Architecture Flow Diagram	46
27.3 State Transition Table	46
28 Rank Responses from Human Feedback (RRHF)	47
28.1 Mathematical Foundation	47
28.2 Architecture Flow Diagram	47
28.3 State Transition Table	47
29 Reinforcement Learning from AI Feedback (RLAIF)	48
29.1 Mathematical Foundation	48
29.2 Architecture Flow Diagram	48
29.3 State Transition Table	48
30 Constitutional AI	49
30.1 Mathematical Foundation	49
30.2 Architecture Flow Diagram	49
30.3 State Transition Table	49
V Reinforcement Learning Training Methods	50
31 Proximal Policy Optimization (PPO)	50
31.1 Mathematical Foundation	50
31.2 Architecture Flow Diagram	50
31.3 State Transition Table	50
32 Group Relative Policy Optimization (GRPO)	51
32.1 Mathematical Foundation	51
32.2 Architecture Flow Diagram	51
32.3 State Transition Table	51
33 Rejection Fine-Tuning (RFT)	52
33.1 Mathematical Foundation	52
33.2 Architecture Flow Diagram	52
33.3 State Transition Table	52
34 REINFORCE	53
34.1 Mathematical Foundation	53
34.2 Architecture Flow Diagram	53
34.3 State Transition Table	53
35 Advantage Actor-Critic (A2C / A3C)	54
35.1 Mathematical Foundation	54
35.2 Architecture Flow Diagram	54
35.3 State Transition Table	54
36 Self-Play Reinforcement Learning	55
36.1 Mathematical Foundation	55
36.2 Architecture Flow Diagram	55
36.3 State Transition Table	55
37 Monte Carlo Tree Search RL (MCTS-RL)	56
37.1 Mathematical Foundation	56
37.2 Architecture Flow Diagram	56
37.3 State Transition Table	56

38 Search-Augmented RL (Search-RL)	57
38.1 Mathematical Foundation	57
38.2 Architecture Flow Diagram	57
38.3 State Transition Table	57
39 Tree-of-Thought Training	58
39.1 Mathematical Foundation	58
39.2 Architecture Flow Diagram	58
39.3 State Transition Table	58
40 Self-Consistency Training	59
40.1 Mathematical Foundation	59
40.2 Architecture Flow Diagram	59
40.3 State Transition Table	59
VI Reward Modeling	60
41 Reward Model (RM)	60
41.1 Mathematical Foundation	60
41.2 Architecture Flow Diagram	60
41.3 State Transition Table	60
42 Process Reward Model (PRM)	61
42.1 Mathematical Foundation	61
42.2 Architecture Flow Diagram	61
42.3 State Transition Table	61
43 Outcome Reward Model (ORM)	62
43.1 Mathematical Foundation	62
43.2 Architecture Flow Diagram	62
43.3 State Transition Table	62
44 Verifier Model	63
44.1 Mathematical Foundation	63
44.2 Architecture Flow Diagram	63
44.3 State Transition Table	63
45 Critic Model	64
45.1 Mathematical Foundation	64
45.2 Architecture Flow Diagram	64
45.3 State Transition Table	64
VII Synthetic Data Generation	65
46 Self-Instruct	65
46.1 Mathematical Foundation	65
46.2 Architecture Flow Diagram	65
46.3 State Transition Table	65
47 Evol-Instruct	66
47.1 Mathematical Foundation	66
47.2 Architecture Flow Diagram	66
47.3 State Transition Table	66
48 Evol-Code	67
48.1 Mathematical Foundation	67
48.2 Architecture Flow Diagram	67
48.3 State Transition Table	67
49 Synthetic Data Generation	68
49.1 Mathematical Foundation	68
49.2 Architecture Flow Diagram	68
49.3 State Transition Table	68

50 Rejection Sampling	69
50.1 Mathematical Foundation	69
50.2 Architecture Flow Diagram	69
50.3 State Transition Table	69
51 Best-of-N Sampling	70
51.1 Mathematical Foundation	70
51.2 Architecture Flow Diagram	70
51.3 State Transition Table	70
VIII Data Curation	71
52 Data Filtering	71
52.1 Mathematical Foundation	71
52.2 Architecture Flow Diagram	71
52.3 State Transition Table	71
53 Deduplication	72
53.1 Mathematical Foundation	72
53.2 Architecture Flow Diagram	72
53.3 State Transition Table	72
54 Active Learning	73
54.1 Mathematical Foundation	73
54.2 Architecture Flow Diagram	73
54.3 State Transition Table	73
55 Curriculum Learning	74
55.1 Mathematical Foundation	74
55.2 Architecture Flow Diagram	74
55.3 State Transition Table	74
IX Knowledge Distillation	75
56 Knowledge Distillation	75
56.1 Mathematical Foundation	75
56.2 Architecture Flow Diagram	75
56.3 State Transition Table	75
57 Response Distillation	76
57.1 Mathematical Foundation	76
57.2 Architecture Flow Diagram	76
57.3 State Transition Table	76
58 Chain-of-Thought Distillation	77
58.1 Mathematical Foundation	77
58.2 Architecture Flow Diagram	77
58.3 State Transition Table	77
59 Policy Distillation	78
59.1 Mathematical Foundation	78
59.2 Architecture Flow Diagram	78
59.3 State Transition Table	78
60 Reward Distillation	79
60.1 Mathematical Foundation	79
60.2 Architecture Flow Diagram	79
60.3 State Transition Table	79
61 Retrieval Distillation	80
61.1 Mathematical Foundation	80
61.2 Architecture Flow Diagram	80
61.3 State Transition Table	80

X	Model Compression	81
62	Quantization-Aware Training (QAT)	81
62.1	Mathematical Foundation	81
62.2	Architecture Flow Diagram	81
62.3	State Transition Table	81
63	Pruning	82
63.1	Mathematical Foundation	82
63.2	Architecture Flow Diagram	82
63.3	State Transition Table	82
64	Post-Training Quantization (PTQ)	83
64.1	Mathematical Foundation	83
64.2	Architecture Flow Diagram	83
64.3	State Transition Table	83
XI	Agentic Training	84
65	Tool-Use Training	84
65.1	Mathematical Foundation	84
65.2	Architecture Flow Diagram	84
65.3	State Transition Table	84
66	Function Calling Training	85
66.1	Mathematical Foundation	85
66.2	Architecture Flow Diagram	85
66.3	State Transition Table	85
67	Multi-Agent Training	86
67.1	Mathematical Foundation	86
67.2	Architecture Flow Diagram	86
67.3	State Transition Table	86
XII	Multimodal Training	87
68	Vision-Language Training	87
68.1	Mathematical Foundation	87
68.2	Architecture Flow Diagram	87
68.3	State Transition Table	87
69	Grounding Training	88
69.1	Mathematical Foundation	88
69.2	Architecture Flow Diagram	88
69.3	State Transition Table	88
XIII	Knowledge Injection	89
70	Retrieval-Augmented Training	89
70.1	Mathematical Foundation	89
70.2	Architecture Flow Diagram	89
70.3	State Transition Table	89
71	Memory Tuning	90
71.1	Mathematical Foundation	90
71.2	Architecture Flow Diagram	90
71.3	State Transition Table	90
72	Domain Knowledge Injection	91
72.1	Mathematical Foundation	91
72.2	Architecture Flow Diagram	91
72.3	State Transition Table	91

XIV	Deployment and Training Efficiency	92
73	Mixed Precision Training	92
73.1	Mathematical Foundation	92
73.2	Architecture Flow Diagram	92
73.3	State Transition Table	92
74	Gradient Checkpointing	93
74.1	Mathematical Foundation	93
74.2	Architecture Flow Diagram	93
74.3	State Transition Table	93
75	Flash Attention	94
75.1	Mathematical Foundation	94
75.2	Architecture Flow Diagram	94
75.3	State Transition Table	94
76	Fully Sharded Data Parallel (FSDP)	95
76.1	Mathematical Foundation	95
76.2	Architecture Flow Diagram	95
76.3	State Transition Table	95
77	DeepSpeed ZeRO	96
77.1	Mathematical Foundation	96
77.2	Architecture Flow Diagram	96
77.3	State Transition Table	96

Contents

Part I

Foundation Training Methods

Pretraining

Mathematical Foundation

Causal language modeling models the probability of a sequence $x = (x_1, \dots, x_T)$ as the product of autoregressive conditional probabilities:

$$P(x) = \prod_{t=1}^T P(x_t \mid x_{<t}) \quad (1)$$

The hidden representation $h_t^{(l)}$ at layer l and token position t is computed using multi-head causal self-attention. The causal attention mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (2)$$

This mask is added directly to the scaled query-key dot products:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V \quad (3)$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$. This prevents information leakage from future tokens. At the output layer, the logits $z_t \in \mathbb{R}^{|\mathcal{V}|}$ are computed via $z_t = h_t^{(L)} W_{\text{vocab}}$. To maintain numerical stability, softmax is computed by subtracting the maximum logit:

$$P(x_t = v \mid x_{<t}) = \frac{\exp(z_{t,v} - \max_k z_{t,k})}{\sum_{j \in \mathcal{V}} \exp(z_{t,j} - \max_k z_{t,k})} \quad (4)$$

The causal model parameters θ are trained by minimizing the cross-entropy loss over a dataset $\mathcal{D}$:

$$\mathcal{L}_{\text{PT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^T \log P_\theta(x_t \mid x_{<t}) \quad (5)$$

Updates are made using the AdamW optimizer. Let $g_t = \nabla_\theta \mathcal{L}_t$ be the gradient at step t . The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (6)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (7)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (8)$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \quad (9)$$

where η is the learning rate, λ is the weight decay rate, β_1, β_2 are the exponential decay rates, and ϵ prevents division by zero. A cosine learning rate schedule with linear warmup is employed:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}} \pi \right) \right) \quad (10)$$

Architecture Flow Diagram

State Transition Table

Python Implementation

```
1 import torch
2 import torch.nn as nn
3 import math
4
5 class CausalSelfAttention(nn.Module):
6     def __init__(self, d_model, num_heads):
7         super().__init__()
8         assert d_model % num_heads == 0
```

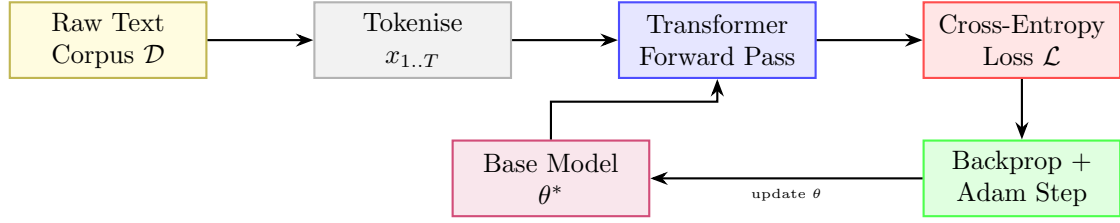


Figure 1: Pretraining pipeline: corpus sampling, forward pass, loss computation, and weight update loop.

Property	Before	After
Parameter source	Random init $\theta_0 \sim \mathcal{N}(0, \sigma^2)$	Converged weights θ^*
Training corpus	Not consumed	Trillions of tokens processed
Perplexity	Very high ($> 10^3$)	Low (< 20 on held-out data)
World knowledge	None	Broad language & factual knowledge
requires_grad	True (all params)	True (all params)

Table 1: State properties before and after pretraining.

```

9      self.d_model = d_model
10     self.num_heads = num_heads
11     self.head_dim = d_model // num_heads
12
13     self.q_proj = nn.Linear(d_model, d_model)
14     self.k_proj = nn.Linear(d_model, d_model)
15     self.v_proj = nn.Linear(d_model, d_model)
16     self.out_proj = nn.Linear(d_model, d_model)
17
18     def forward(self, x):
19         # x shape: (batch_size, seq_len, d_model)
20         b, t, c = x.size()
21
22         q = self.q_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
23         k = self.k_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
24         v = self.v_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
25
26         # Compute scaled query-key dot product attention
27         scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.head_dim)
28
29         # Apply causal attention mask
30         mask = torch.triu(torch.ones(t, t, device=x.device), diagonal=1) * -1e9
31         scores = scores + mask.view(1, 1, t, t)
32
33         attn_weights = torch.softmax(scores, dim=-1)
34         out = torch.matmul(attn_weights, v) # (b, num_heads, t, head_dim)
35         out = out.transpose(1, 2).contiguous().view(b, t, c)
36         return self.out_proj(out)
37
38     class PretrainingCausalLM(nn.Module):
39         def __init__(self, vocab_size, d_model, num_heads):
40             super().__init__()
41             self.attn = CausalSelfAttention(d_model, num_heads)
42             self.vocab_proj = nn.Linear(d_model, vocab_size)
43             self.loss_fn = nn.CrossEntropyLoss(reduction='mean')
44
45         def forward(self, x, targets=None):
46             # x: hidden states or token embeddings, shape: (batch_size, seq_len, d_model)
47             # targets: target token ids, shape: (batch_size, seq_len)
48             h = self.attn(x)
49             logits = self.vocab_proj(h)
50
51             loss = None
52             if targets is not None:
53                 shift_logits = logits[..., :-1, :].contiguous()
54                 shift_targets = targets[..., 1:].contiguous()
55                 loss = self.loss_fn(shift_logits.view(-1, shift_logits.size(-1)), shift_targets.view(-1))
56
57             return logits, loss

```

Continued Pretraining (CPT)

Mathematical Foundation

Causal language modeling models the probability of a sequence $x = (x_1, \dots, x_T)$ as the product of autoregressive conditional probabilities:

$$P(x) = \prod_{t=1}^T P(x_t | x_{<t}) \quad (11)$$

The hidden representation $h_t^{(l)}$ at layer l and token position t is computed using multi-head causal self-attention. The causal attention mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (12)$$

This mask is added directly to the scaled query-key dot products:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V \quad (13)$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$. This prevents information leakage from future tokens. At the output layer, the logits $z_t \in \mathbb{R}^{|V|}$ are computed via $z_t = h_t^{(L)} W_{\text{vocab}}$. To maintain numerical stability, softmax is computed by subtracting the maximum logit:

$$P(x_t = v | x_{<t}) = \frac{\exp(z_{t,v} - \max_k z_{t,k})}{\sum_{j \in V} \exp(z_{t,j} - \max_k z_{t,k})} \quad (14)$$

The causal model parameters θ are trained by minimizing the cross-entropy loss over a dataset $\mathcal{D}$:

$$\mathcal{L}_{\text{PT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^T \log P_\theta(x_t | x_{<t}) \quad (15)$$

Updates are made using the AdamW optimizer. Let $g_t = \nabla_\theta \mathcal{L}_t$ be the gradient at step t . The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (16)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (17)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (18)$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \quad (19)$$

where η is the learning rate, λ is the weight decay rate, β_1, β_2 are the exponential decay rates, and ϵ prevents division by zero. A cosine learning rate schedule with linear warmup is employed:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}} \pi \right) \right) \quad (20)$$

Architecture Flow Diagram

State Transition Table

Property	Before	After
Starting checkpoint	Pretrained θ_{base}	$\theta' = \theta_{\text{base}} + \Delta\theta$
New domain knowledge	Absent	Incorporated
Original knowledge	Retained	Partially retained via replay
Learning rate schedule	Finished	Warm-restart applied
Perplexity on new domain	High	Significantly reduced

Table 2: State properties before and after continued pretraining.

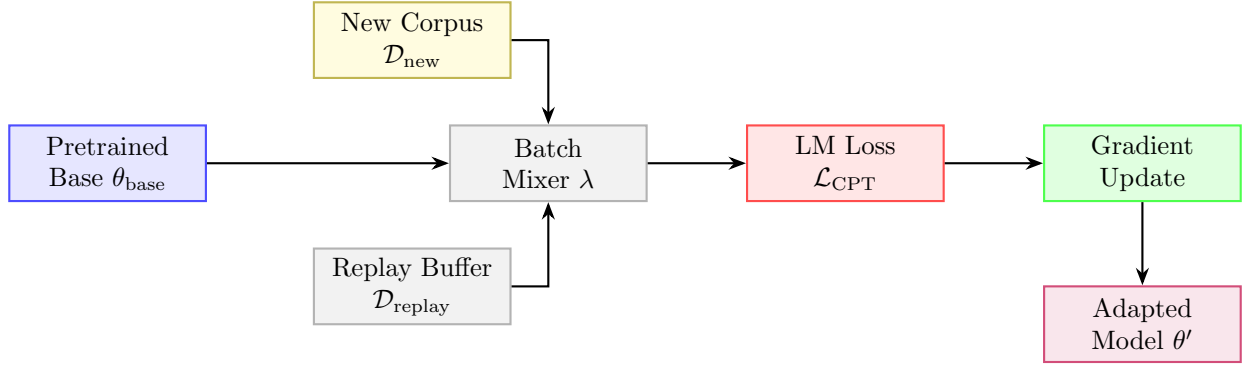


Figure 2: CPT mixes new corpus and replay buffer batches to update a pretrained checkpoint.

Python Implementation

```

1 import torch
2 import torch.nn as nn
3 import math
4
5 class CausalSelfAttention(nn.Module):
6     def __init__(self, d_model, num_heads):
7         super().__init__()
8         assert d_model % num_heads == 0
9         self.d_model = d_model
10        self.num_heads = num_heads
11        self.head_dim = d_model // num_heads
12
13        self.q_proj = nn.Linear(d_model, d_model)
14        self.k_proj = nn.Linear(d_model, d_model)
15        self.v_proj = nn.Linear(d_model, d_model)
16        self.out_proj = nn.Linear(d_model, d_model)
17
18    def forward(self, x):
19        # x shape: (batch_size, seq_len, d_model)
20        b, t, c = x.size()
21
22        q = self.q_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
23        k = self.k_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
24        v = self.v_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
25
26        # Compute scaled query-key dot product attention
27        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.head_dim)
28
29        # Apply causal attention mask
30        mask = torch.triu(torch.ones(t, t, device=x.device), diagonal=1) * -1e9
31        scores = scores + mask.view(1, 1, t, t)
32
33        attn_weights = torch.softmax(scores, dim=-1)
34        out = torch.matmul(attn_weights, v) # (b, num_heads, t, head_dim)
35        out = out.transpose(1, 2).contiguous().view(b, t, c)
36        return self.out_proj(out)
37
38 class PretrainingCausalLM(nn.Module):
39     def __init__(self, vocab_size, d_model, num_heads):
40         super().__init__()
41         self.attn = CausalSelfAttention(d_model, num_heads)
42         self.vocab_proj = nn.Linear(d_model, vocab_size)
43         self.loss_fn = nn.CrossEntropyLoss(reduction='mean')
44
45     def forward(self, x, targets=None):
46        # x: hidden states or token embeddings, shape: (batch_size, seq_len, d_model)
47        # targets: target token ids, shape: (batch_size, seq_len)
48        h = self.attn(x)
49        logits = self.vocab_proj(h)
50
51        loss = None
52        if targets is not None:
53            shift_logits = logits[..., :-1, :].contiguous()
54            shift_targets = targets[..., 1:].contiguous()
55            loss = self.loss_fn(shift_logits.view(-1, shift_logits.size(-1)), shift_targets.view(-1))
56
57        return logits, loss
  
```

Domain Adaptive Pretraining (DAPT)

Mathematical Foundation

Causal language modeling models the probability of a sequence $x = (x_1, \dots, x_T)$ as the product of autoregressive conditional probabilities:

$$P(x) = \prod_{t=1}^T P(x_t | x_{<t}) \quad (21)$$

The hidden representation $h_t^{(l)}$ at layer l and token position t is computed using multi-head causal self-attention. The causal attention mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (22)$$

This mask is added directly to the scaled query-key dot products:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V \quad (23)$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$. This prevents information leakage from future tokens. At the output layer, the logits $z_t \in \mathbb{R}^{|\mathcal{V}|}$ are computed via $z_t = h_t^{(L)} W_{\text{vocab}}$. To maintain numerical stability, softmax is computed by subtracting the maximum logit:

$$P(x_t = v | x_{<t}) = \frac{\exp(z_{t,v} - \max_k z_{t,k})}{\sum_{j \in \mathcal{V}} \exp(z_{t,j} - \max_k z_{t,k})} \quad (24)$$

The causal model parameters θ are trained by minimizing the cross-entropy loss over a dataset $\mathcal{D}$:

$$\mathcal{L}_{\text{PT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^T \log P_\theta(x_t | x_{<t}) \quad (25)$$

Updates are made using the AdamW optimizer. Let $g_t = \nabla_\theta \mathcal{L}_t$ be the gradient at step t . The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (26)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (27)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (28)$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \quad (29)$$

where η is the learning rate, λ is the weight decay rate, β_1, β_2 are the exponential decay rates, and ϵ prevents division by zero. A cosine learning rate schedule with linear warmup is employed:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}} \pi \right) \right) \quad (30)$$

Architecture Flow Diagram

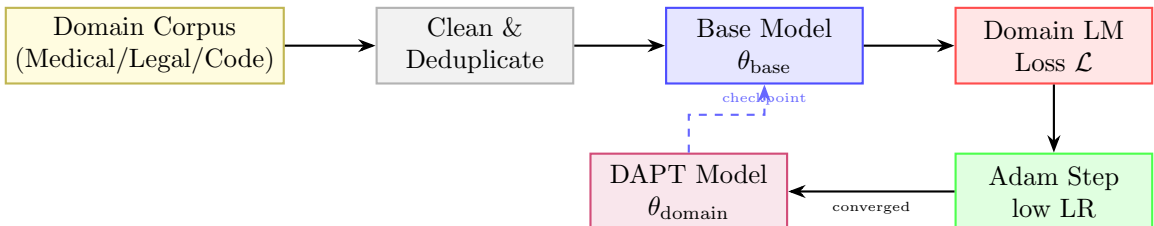


Figure 3: DAPT pipeline: domain corpus is cleaned, fed into the base model, and trained with standard LM loss.

Property	Before	After
Domain vocabulary familiarity	Low	High
Domain perplexity	High ($\rho \gg 1$)	Low (near 1)
General language ability	Strong	Marginally reduced
Downstream task (same domain)	Weak	Significantly stronger
Training data type	General web text	Domain-specific unlabelled

Table 3: State properties before and after domain adaptive pretraining.

State Transition Table

Python Implementation

```

1 import torch
2 import torch.nn as nn
3 import math
4
5 class CausalSelfAttention(nn.Module):
6     def __init__(self, d_model, num_heads):
7         super().__init__()
8         assert d_model % num_heads == 0
9         self.d_model = d_model
10        self.num_heads = num_heads
11        self.head_dim = d_model // num_heads
12
13        self.q_proj = nn.Linear(d_model, d_model)
14        self.k_proj = nn.Linear(d_model, d_model)
15        self.v_proj = nn.Linear(d_model, d_model)
16        self.out_proj = nn.Linear(d_model, d_model)
17
18    def forward(self, x):
19        # x shape: (batch_size, seq_len, d_model)
20        b, t, c = x.size()
21
22        q = self.q_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
23        k = self.k_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
24        v = self.v_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
25
26        # Compute scaled query-key dot product attention
27        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.head_dim)
28
29        # Apply causal attention mask
30        mask = torch.triu(torch.ones(t, t, device=x.device), diagonal=1) * -1e9
31        scores = scores + mask.view(1, 1, t, t)
32
33        attn_weights = torch.softmax(scores, dim=-1)
34        out = torch.matmul(attn_weights, v) # (b, num_heads, t, head_dim)
35        out = out.transpose(1, 2).contiguous().view(b, t, c)
36        return self.out_proj(out)
37
38 class PretrainingCausalLM(nn.Module):
39     def __init__(self, vocab_size, d_model, num_heads):
40         super().__init__()
41         self.attn = CausalSelfAttention(d_model, num_heads)
42         self.vocab_proj = nn.Linear(d_model, vocab_size)
43         self.loss_fn = nn.CrossEntropyLoss(reduction='mean')
44
45     def forward(self, x, targets=None):
46        # x: hidden states or token embeddings, shape: (batch_size, seq_len, d_model)
47        # targets: target token ids, shape: (batch_size, seq_len)
48        h = self.attn(x)
49        logits = self.vocab_proj(h)
50
51        loss = None
52        if targets is not None:
53            shift_logits = logits[..., :-1, :].contiguous()
54            shift_targets = targets[..., 1:].contiguous()
55            loss = self.loss_fn(shift_logits.view(-1, shift_logits.size(-1)), shift_targets.view(-1))
56
57        return logits, loss

```

Task Adaptive Pretraining (TAPT)

Mathematical Foundation

Causal language modeling models the probability of a sequence $x = (x_1, \dots, x_T)$ as the product of autoregressive conditional probabilities:

$$P(x) = \prod_{t=1}^T P(x_t | x_{<t}) \quad (31)$$

The hidden representation $h_t^{(l)}$ at layer l and token position t is computed using multi-head causal self-attention. The causal attention mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (32)$$

This mask is added directly to the scaled query-key dot products:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V \quad (33)$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$. This prevents information leakage from future tokens. At the output layer, the logits $z_t \in \mathbb{R}^{|V|}$ are computed via $z_t = h_t^{(L)} W_{\text{vocab}}$. To maintain numerical stability, softmax is computed by subtracting the maximum logit:

$$P(x_t = v | x_{<t}) = \frac{\exp(z_{t,v} - \max_k z_{t,k})}{\sum_{j \in V} \exp(z_{t,j} - \max_k z_{t,k})} \quad (34)$$

The causal model parameters θ are trained by minimizing the cross-entropy loss over a dataset $\mathcal{D}$:

$$\mathcal{L}_{\text{PT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^T \log P_\theta(x_t | x_{<t}) \quad (35)$$

Updates are made using the AdamW optimizer. Let $g_t = \nabla_\theta \mathcal{L}_t$ be the gradient at step t . The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (36)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (37)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (38)$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \quad (39)$$

where η is the learning rate, λ is the weight decay rate, β_1, β_2 are the exponential decay rates, and ϵ prevents division by zero. A cosine learning rate schedule with linear warmup is employed:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}} \pi \right) \right) \quad (40)$$

Architecture Flow Diagram

State Transition Table

Property	Before	After
Task vocabulary alignment	Partial	Strong
Corpus size	Millions of docs	Thousands of docs
Training epochs	1–3	10–100 (small corpus)
Downstream label efficiency	Requires more labels	Requires fewer labels
Transfer to other tasks	Good	Reduced (narrow focus)

Table 4: State properties before and after task adaptive pretraining.

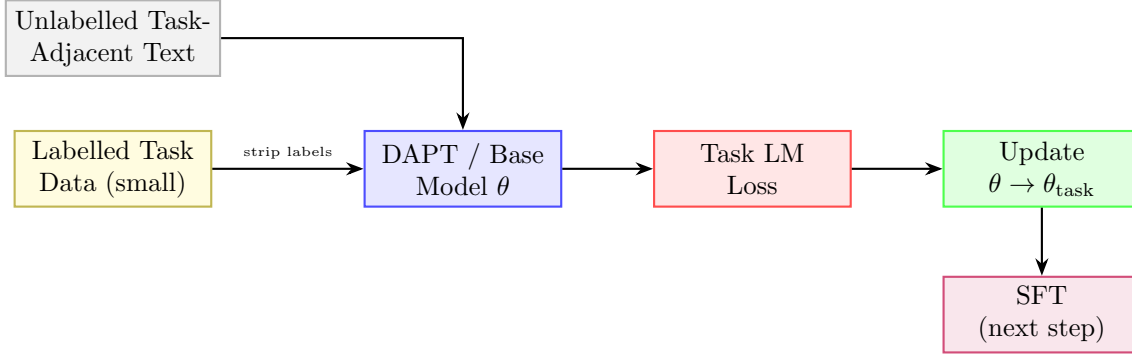


Figure 4: TAPT uses task-adjacent unlabelled text to bridge base model and supervised fine-tuning.

Python Implementation

```

1 import torch
2 import torch.nn as nn
3 import math
4
5 class CausalSelfAttention(nn.Module):
6     def __init__(self, d_model, num_heads):
7         super().__init__()
8         assert d_model % num_heads == 0
9         self.d_model = d_model
10        self.num_heads = num_heads
11        self.head_dim = d_model // num_heads
12
13        self.q_proj = nn.Linear(d_model, d_model)
14        self.k_proj = nn.Linear(d_model, d_model)
15        self.v_proj = nn.Linear(d_model, d_model)
16        self.out_proj = nn.Linear(d_model, d_model)
17
18    def forward(self, x):
19        # x shape: (batch_size, seq_len, d_model)
20        b, t, c = x.size()
21
22        q = self.q_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
23        k = self.k_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
24        v = self.v_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
25
26        # Compute scaled query-key dot product attention
27        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.head_dim)
28
29        # Apply causal attention mask
30        mask = torch.triu(torch.ones(t, t, device=x.device), diagonal=1) * -1e9
31        scores = scores + mask.view(1, 1, t, t)
32
33        attn_weights = torch.softmax(scores, dim=-1)
34        out = torch.matmul(attn_weights, v) # (b, num_heads, t, head_dim)
35        out = out.transpose(1, 2).contiguous().view(b, t, c)
36        return self.out_proj(out)
37
38 class PretrainingCausalLM(nn.Module):
39     def __init__(self, vocab_size, d_model, num_heads):
40         super().__init__()
41         self.attn = CausalSelfAttention(d_model, num_heads)
42         self.vocab_proj = nn.Linear(d_model, vocab_size)
43         self.loss_fn = nn.CrossEntropyLoss(reduction='mean')
44
45     def forward(self, x, targets=None):
46        # x: hidden states or token embeddings, shape: (batch_size, seq_len, d_model)
47        # targets: target token ids, shape: (batch_size, seq_len)
48        h = self.attn(x)
49        logits = self.vocab_proj(h)
50
51        loss = None
52        if targets is not None:
53            shift_logits = logits[..., :-1, :].contiguous()
54            shift_targets = targets[..., 1:].contiguous()
55            loss = self.loss_fn(shift_logits.view(-1, shift_logits.size(-1)), shift_targets.view(-1))
56
57        return logits, loss
  
```

Part II

Supervised Fine-Tuning Methods

Supervised Fine-Tuning (SFT)

Mathematical Foundation

SFT trains the model on structured prompt-response pairs $(x, y) \sim \mathcal{D}_{\text{SFT}}$ using teacher forcing. Let $s = [x; y]$ be the concatenated token sequence of length $M + U$, where $x = (s_1, \dots, s_M)$ is the prompt and $y = (s_{M+1}, \dots, s_{M+U})$ is the response. The key math lies in applying a target label mask $L \in \{-100, \mathcal{V}\}^{M+U}$ defined as:

$$L_t = \begin{cases} -100 & \text{if } t \leq M \\ s_t & \text{if } t > M \end{cases} \quad (41)$$

The cross-entropy loss function ignores targets set to -100 , directing gradient updates exclusively to the response generation tokens:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{U} \sum_{t=M+1}^{M+U} \log P_{\theta}(s_t \mid s_{<t}) \quad (42)$$

This loss ignores the prompt distribution $P(x)$, preventing the model from degrading its output generation capability to match arbitrary prompts. The gradients only backpropagate through response tokens:

$$\nabla_{\theta} \mathcal{L}_{\text{SFT}} = -\frac{1}{U} \sum_{t=M+1}^{M+U} \nabla_{\theta} \log P_{\theta}(s_t \mid s_{<t}) \quad (43)$$

Architecture Flow Diagram

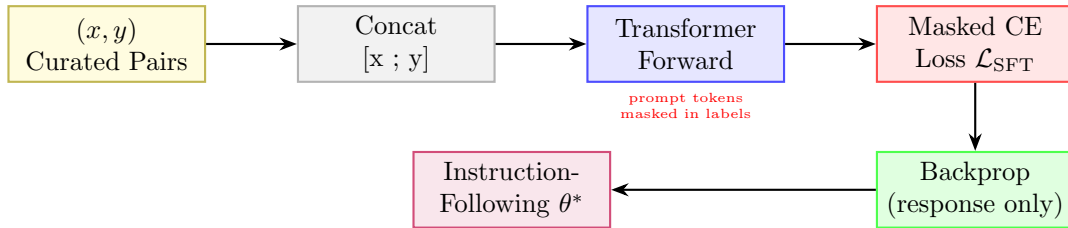


Figure 5: SFT pipeline with prompt-masked loss: gradients flow only through response tokens.

State Transition Table

Property	Before	After
Behaviour	Completion only	Instruction following
Loss region	N/A	Response tokens only
Label masking	None	Prompt masked to -100
Data format	Raw text	(instruction, response) pairs
Output quality	Random completion	Task-aligned responses

Table 5: State properties before and after supervised fine-tuning.

Python Implementation

```
1 import torch
2 import torch.nn as nn
3
4 class SFTDatasetCollator:
5     def __init__(self, tokenizer, ignore_index=-100):
6         self.tokenizer = tokenizer
7         self.ignore_index = ignore_index
8
```

```

9     def __call__(self, batch):
10         # batch: list of dicts with 'prompt' and 'response' strings
11         input_ids_list = []
12         labels_list = []
13
14         for item in batch:
15             prompt_tokens = self.tokenizer.encode(item['prompt'])
16             res_tokens = self.tokenizer.encode(item['response'])
17
18             # Concatenate prompt + response
19             input_ids = prompt_tokens + res_tokens
20             # Mask the prompt tokens in the label
21             labels = [self.ignore_index] * len(prompt_tokens) + res_tokens
22
23             input_ids_list.append(torch.tensor(input_ids))
24             labels_list.append(torch.tensor(labels))
25
26             # Pad sequences to max length in batch
27             padded_inputs = nn.utils.rnn.pad_sequence(input_ids_list, batch_first=True, padding_value=
self.tokenizer.pad_token_id)
28             padded_labels = nn.utils.rnn.pad_sequence(labels_list, batch_first=True, padding_value=self.
ignore_index)
29
30             return padded_inputs, padded_labels
31
32 class SFTModuleLoss(nn.Module):
33     def __init__(self, ignore_index=-100):
34         super().__init__()
35         self.ce = nn.CrossEntropyLoss(ignore_index=ignore_index)
36
37     def forward(self, logits, targets):
38         # Shifting logits and targets for next-token prediction
39         shift_logits = logits[..., :-1, :].contiguous()
40         shift_targets = targets[..., 1:].contiguous()
41         return self.ce(shift_logits.view(-1, shift_logits.size(-1)), shift_targets.view(-1))

```

Instruction Tuning

Mathematical Foundation

Instruction Tuning exposes the model to multi-turn conversational patterns. A prompt template $T(I, x)$ converts an instruction I and context x into structured sequences. Turn boundaries are defined using chat-formatting systems (like ChatML). A single sequence layout is:

$$S = \langle \text{user} \rangle I \parallel x \langle \text{assistant} \rangle y \langle \text{eos} \rangle \quad (44)$$

Let $\mathcal{D}_{\text{inst}}$ consist of instructions spanning hundreds of tasks. The model parameters θ are optimized on the joint multi-task dataset:

$$\mathcal{L}_{\text{IT}}(\theta) = -\mathbb{E}_{(I, x, y) \sim \mathcal{D}_{\text{inst}}} \sum_{t=1}^{|y|} \log P_{\theta}(y_t \mid T(I, x), y_{<t}) \quad (45)$$

We mask all prompt-side tokens, including conversational format tags (like $\langle \text{user} \rangle$ and instruction text), ensuring parameters are trained only on synthesizing assistant responses:

$$\mathcal{L}_{\text{IT}}(\theta) = -\frac{1}{|y|} \sum_{i=1}^{|T(I, x)| + |y|} M_i \log P_{\theta}(S_i \mid S_{<i}), \quad M_i = \mathbb{I}(i > |T(I, x)|) \quad (46)$$

Architecture Flow Diagram

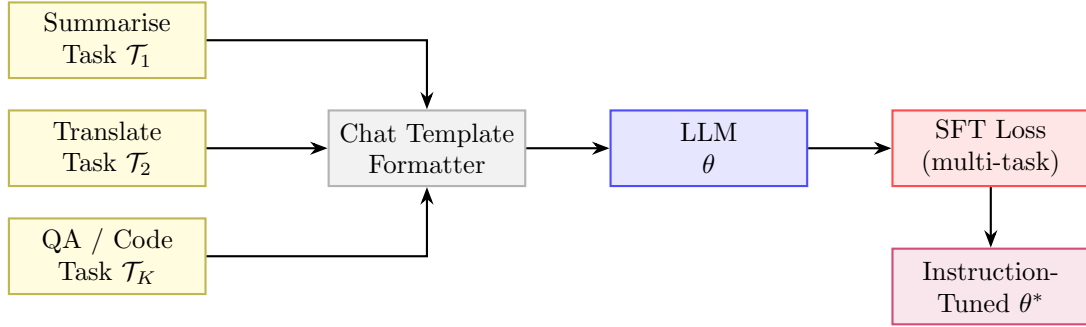


Figure 6: Instruction tuning over diverse task types funnelled through a unified chat template.

State Transition Table

Property	Before	After
Zero-shot generalisation	Poor	Strong across task types
Task diversity in training	N/A	K heterogeneous tasks
Input format	Free-form text	Structured chat template
Prompt sensitivity	High	Reduced
Held-out task performance	Weak	Strong (FLAN, InstructGPT)

Table 6: State properties before and after instruction tuning.

Python Implementation

```

1 import torch
2 import torch.nn as nn
3
4 class InstructionFormatter(nn.Module):
5     def __init__(self, tokenizer, ignore_index=-100):
6         super().__init__()
7         self.tokenizer = tokenizer
8         self.ignore_index = ignore_index
9         self.ce = nn.CrossEntropyLoss(ignore_index=ignore_index)
10
11     def format_chatml(self, instruction, context, response):

```

```

12     prompt_text = f"<|im_start|>user\n{instruction}\n{context}<|im_end|>\n<|im_start|>assistant\
n"
13     response_text = f"{response}<|im_end|>\n"
14
15     prompt_ids = self.tokenizer.encode(prompt_text)
16     response_ids = self.tokenizer.encode(response_text)
17
18     input_ids = prompt_ids + response_ids
19     targets = [self.ignore_index] * len(prompt_ids) + response_ids
20
21     return torch.tensor(input_ids), torch.tensor(targets)
22
23 def forward(self, logits, targets):
24     shift_logits = logits[..., :-1, :].contiguous()
25     shift_targets = targets[..., 1:].contiguous()
26     return self.ce(shift_logits.view(-1, shift_logits.size(-1)), shift_targets.view(-1))

```

Multi-Task Training

Mathematical Foundation

Multi-Task Training aggregates losses from K distinct tasks. A simple summation can result in tasks with large gradient scales dominating the parameter updates. To resolve this, we use the GradNorm algorithm. Let W be the shared parameter weights (e.g., the last transformer block weights). We define the gradient norm for task k at training step t as:

$$G_k(t) = \|\nabla_W(w_k(t)\mathcal{L}_k(t))\|_2 \quad (47)$$

Let $\bar{G}(t) = \frac{1}{K} \sum_k G_k(t)$ be the average gradient norm. The target gradient norm for task k balances task difficulties:

$$\hat{G}_k(t) = \bar{G}(t) \times \left(\frac{\tilde{\mathcal{L}}_k(t)}{\frac{1}{K} \sum_j \tilde{\mathcal{L}}_j(t)} \right)^\alpha \quad (48)$$

where $\tilde{\mathcal{L}}_k(t) = \mathcal{L}_k(t)/\mathcal{L}_k(0)$ is the task loss ratio indicating task training progress, and α is a hyperparameter managing structural gradient balancing. The task weights $w_k(t)$ are updated by minimizing the L1 distance between $G_k(t)$ and $\hat{G}_k(t)$:

$$\mathcal{L}_{\text{grad}}(w_1, \dots, w_K; t) = \sum_{k=1}^K |G_k(t) - \hat{G}_k(t)| \quad (49)$$

The final joint parameter objective is:

$$\mathcal{L}_{\text{MT}}(\theta) = \sum_{k=1}^K w_k \mathcal{L}_k(\theta) \quad (50)$$

Architecture Flow Diagram

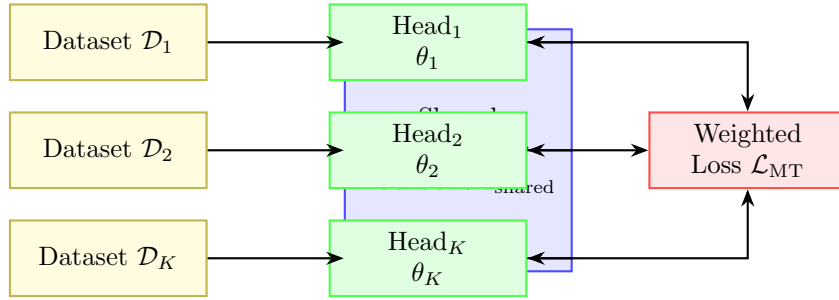


Figure 7: Multi-task training: each dataset feeds task-specific heads on a shared backbone.

State Transition Table

Property	Before	After
Backbone specialisation	Single task	K tasks jointly
Task heads	1	K heads
Generalisation	Narrow	Broad
Data efficiency	Per-task only	Cross-task transfer
Gradient conflicts	N/A	Managed via λ_k

Table 7: State properties before and after multi-task training.

Python Implementation

```

1 import torch
2 import torch.nn as nn
3
4 class GradNormMultiTaskTrainer:
5     def __init__(self, model, num_tasks, alpha=0.12, lr_w=0.025):
6         self.model = model
7         self.num_tasks = num_tasks

```

```

8     self.alpha = alpha
9     self.lr_w = lr_w
10
11     # Learnable task weights, initialized to 1.0
12     self.w = nn.Parameter(torch.ones(num_tasks, requires_grad=True))
13     self.initial_losses = None
14     self.optimizer_w = torch.optim.Adam([self.w], lr=lr_w)
15
16 def train_step(self, task_losses, shared_weights_layer, optimizer_model):
17     # task_losses: tensor of shape (num_tasks,) containing current task losses
18     if self.initial_losses is None:
19         self.initial_losses = task_losses.detach().clone()
20
21     weighted_losses = self.w * task_losses
22     total_loss = weighted_losses.sum()
23
24     # Backward pass on the main model parameters
25     optimizer_model.zero_grad()
26     total_loss.backward(retain_graph=True)
27
28     # Compute gradient norms G_k for each task with respect to shared weights
29     norms = []
30     for i in range(self.num_tasks):
31         grads = torch.autograd.grad(weighted_losses[i], list(shared_weights_layer.parameters()),
32 retain_graph=True, create_graph=True)
33         # Accumulate squared norms of all parameters in shared weights layer
34         norm = torch.sqrt(sum(g.pow(2).sum() for g in grads if g is not None))
35         norms.append(norm)
36     norms = torch.stack(norms)
37
38     # Calculate loss ratios and target gradient norms
39     loss_ratios = task_losses.detach() / self.initial_losses
40     mean_norm = norms.mean().detach()
41     mean_ratio = loss_ratios.mean()
42
43     targets = mean_norm * (loss_ratios / mean_ratio) ** self.alpha
44
45     # Update task weights w
46     self.optimizer_w.zero_grad()
47     grad_loss = torch.sum(torch.abs(norms - targets))
48     grad_loss.backward()
49
50     # Update weights and normalize them
51     self.optimizer_w.step()
52     with torch.no_grad():
53         self.w.copy_(self.w / self.w.sum() * self.num_tasks)
54
55     optimizer_model.step()
56     return total_loss.item(), self.w.detach().tolist()

```

Chain-of-Thought SFT

Mathematical Foundation

CoT SFT trains models to output intermediate reasoning steps before generating the final answer. Let x be the prompt, $c = (c_1, \dots, c_M)$ be the reasoning chain tokens, and $y = (y_1, \dots, y_U)$ be the final answer tokens. The joint probability factorization is:

$$P(c, y | x) = \prod_{i=1}^M P_{\theta}(c_i | x, c_{<i}) \prod_{j=1}^U P_{\theta}(y_j | x, c, y_{<j}) \quad (51)$$

The model is trained by minimizing the joint loss:

$$\mathcal{L}_{\text{CoT}}(\theta) = -\beta \sum_{i=1}^M \log P_{\theta}(c_i | x, c_{<i}) - (1 - \beta) \sum_{j=1}^U \log P_{\theta}(y_j | x, c, y_{<j}) \quad (52)$$

During autoregressive generation, we sample tokens using Temperature-scaled softmax and Nucleus (top- p) filtering. The logits z_t are modified as:

$$\tilde{z}_{t,i} = \frac{z_{t,i}}{T} \quad (53)$$

where $T > 0$ is the temperature. The top- p vocabulary subset $\mathcal{V}^{(p)}$ at step t is the smallest set satisfying:

$$\sum_{v \in \mathcal{V}^{(p)}} P(v | x, s_{<t}) \geq p \quad (54)$$

Tokens outside $\mathcal{V}^{(p)}$ are assigned probability 0 before final selection.

Architecture Flow Diagram

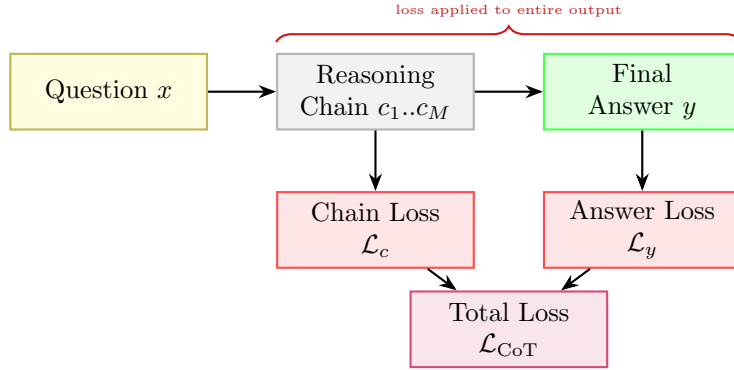


Figure 8: CoT SFT: loss is computed over both the reasoning chain and the final answer.

State Transition Table

Property	Before	After
Output format	Direct answer	Reasoning chain + answer
Training signal per example	$ y $ tokens	$ c + y $ tokens
Multi-step reasoning	Weak	Strong
Data annotation cost	Low	High (chain annotation)
Accuracy on complex tasks	Low	Substantially improved

Table 8: State properties before and after Chain-of-Thought SFT.

Python Implementation


```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class CoTGenerator(nn.Module):
6     def __init__(self, model):
7         super().__init__()
8         self.model = model
9
10    def sample_top_p(self, logits, top_p=0.9, temperature=0.7):
11        # logits shape: (1, vocab_size)
12        scaled_logits = logits / temperature
13        probs = F.softmax(scaled_logits, dim=-1)
14
15        sorted_probs, sorted_indices = torch.sort(probs, descending=True, dim=-1)
16        cumulative_probs = torch.cumsum(sorted_probs, dim=-1)
17
18        # Remove tokens with cumulative probability above top_p (but keep the first one)
19        sorted_indices_to_remove = cumulative_probs > top_p
20        sorted_indices_to_remove[..., 1:] = sorted_indices_to_remove[..., :-1].clone()
21        sorted_indices_to_remove[..., 0] = False
22
23        # Set filtered logits to negative infinity
24        indices_to_remove = sorted_indices_to_remove.scatter(1, sorted_indices,
sorted_indices_to_remove)
25        scaled_logits[indices_to_remove] = -1e9
26
27        # Re-compute probabilities and sample
28        filtered_probs = F.softmax(scaled_logits, dim=-1)
29        next_token = torch.multinomial(filtered_probs, num_samples=1)
30        return next_token
31
32    def generate(self, input_ids, max_len=100, top_p=0.9, temp=0.7):
33        # Autoregressive generation loop
34        generated = input_ids.clone()
35        for _ in range(max_len):
36            outputs, _ = self.model(generated)
37            next_logit = outputs[:, -1, :]
38            next_token = self.sample_top_p(next_logit, top_p=top_p, temperature=temp)
39            generated = torch.cat([generated, next_token], dim=1)
40        return generated

```

Step-by-Step SFT

Mathematical Foundation

Step-by-Step SFT formats the reasoning chain into discrete steps separated by a boundary token $T_{\text{step}} = [\text{STEP}]$. Let the response contain S steps: $s = (s_1, T_{\text{step}}, s_2, T_{\text{step}}, \dots, s_S, T_{\text{step}})$. The step-level loss is:

$$\mathcal{L}_{\text{SBS}}(\theta) = - \sum_{k=1}^S \sum_{t=1}^{|s_k|} \log P_{\theta}(s_{k,t} \mid x, s_{<k}, s_{k,<t}) - \sum_{k=1}^S \log P_{\theta}(T_{\text{step}} \mid x, s_{\leq k}) \quad (55)$$

We define the average step perplexity PPL_k as:

$$\text{PPL}_k = \exp \left(- \frac{1}{|s_k|} \sum_{t=1}^{|s_k|} \log P_{\theta}(s_{k,t} \mid x, s_{<k}, s_{k,<t}) \right) \quad (56)$$

Steps with high perplexity ($\text{PPL}_k > \tau$, where τ is a threshold) indicate low model confidence and can be flagged for step-level revision during generation. The model is trained to minimize the combined cross-entropy loss over all steps.

Architecture Flow Diagram

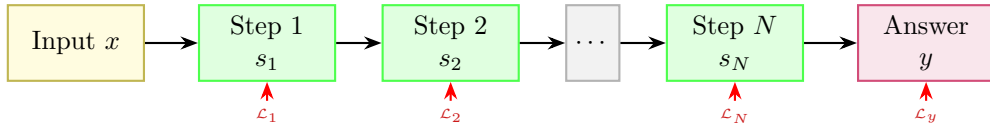


Figure 9: Step-by-Step SFT: each numbered step and the final answer contribute to the total loss.

State Transition Table

Property	Before	After
Solution structure	Unstructured	Numbered procedural steps
Step delimiter tokens	Not in vocabulary	Learned as first-class tokens
Interpretability	Low	Step-level traceability
Annotation format	Raw answer	$(x, [s_1, \dots, s_N], y)$
Task decomposition ability	Implicit	Explicit

Table 9: State properties before and after Step-by-Step SFT.

Python Implementation

```

1 import torch
2 import torch.nn as nn
3
4 class StepByStepLossAndConfidence(nn.Module):
5     def __init__(self, step_token_id=50257, ignore_index=-100):
6         super().__init__()
7         self.step_token_id = step_token_id
8         self.ignore_index = ignore_index
9         self.ce = nn.CrossEntropyLoss(reduction='none')
10
11     def forward(self, logits, targets):
12         # logits: (batch_size, seq_len, vocab_size)
13         # targets: (batch_size, seq_len)
14         shift_logits = logits[..., :-1, :].contiguous()
15         shift_targets = targets[..., 1:].contiguous()
16
17         batch_size, seq_len = shift_targets.size()
18
19         flat_logits = shift_logits.view(-1, shift_logits.size(-1))
20         flat_targets = shift_targets.view(-1)
21
22         # Calculate loss per token position

```

```

23     losses = self.ce(flat_logits, flat_targets).view(batch_size, seq_len)
24
25     step_perplexities = []
26     for b in range(batch_size):
27         step_indices = (shift_targets[b] == self.step_token_id).nonzero(as_tuple=True)[0]
28         start = 0
29         for idx in step_indices:
30             step_loss = losses[b, start:idx.item()]
31             # Exclude masked tokens
32             valid_losses = step_loss[shift_targets[b, start:idx.item()] != self.ignore_index]
33             if len(valid_losses) > 0:
34                 step_ppl = torch.exp(valid_losses.mean())
35                 step_perplexities.append(step_ppl.item())
36             start = idx.item() + 1
37
38     mean_loss = losses[shift_targets != self.ignore_index].mean()
39     return mean_loss, step_perplexities

```

Process Supervision

Mathematical Foundation

Process Supervision trains a Process Reward Model (PRM) using step-level binary correctness labels. Let x be the problem description, and $s_1, \dots, s_S$ be the generated steps. Human or model-based annotations assign correctness labels $r_k \in \{0, 1\}$ for each step s_k . The PRM model outputs correctness probabilities by applying a logistic sigmoid function to the step boundary token representation:

$$\hat{r}_k = \sigma(\text{PRM}_\phi(h_{t_k})) = \frac{1}{1 + \exp(-\text{PRM}_\phi(h_{t_k}))} \quad (57)$$

where h_{t_k} is the transformer hidden output at index t_k representing the step boundary token. The PRM parameters ϕ are optimized using binary cross entropy:

$$\mathcal{L}_{\text{PRM}}(\phi) = - \sum_{k=1}^S [r_k \log \hat{r}_k + (1 - r_k) \log(1 - \hat{r}_k)] \quad (58)$$

During inference, candidate sequences are generated. The total sequence correctness score is calculated as the product of step correctness probabilities:

$$\text{Score}(s_{\leq S}) = \prod_{k=1}^S \hat{r}_k \quad (59)$$

This score is used to rank candidate rollouts in Best-of- N sampling or guide selection during Monte Carlo Tree Search (MCTS).

Architecture Flow Diagram

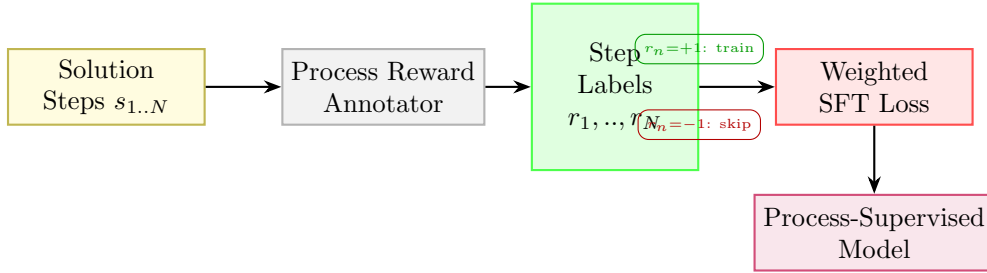


Figure 10: Process supervision: a step annotator labels each reasoning step; only correct steps contribute to the SFT loss.

State Transition Table

Property	Before	After
Supervision granularity	Final answer only	Per-step labels
Error signal	Outcome-level	Step-level
Error localisation	None	Identifies which step failed
Annotation cost	Low	High (step-level labelling)
Reasoning quality	Outcome-driven	Step-correctness driven

Table 10: State properties before and after process supervision.

Python Implementation

```

1 import torch
2 import torch.nn as nn
3
4 class ProcessRewardModel(nn.Module):
5     def __init__(self, hidden_dim, step_token_id=50257):
6         super().__init__()
7         self.step_token_id = step_token_id

```

```

8         # Binary classification head
9         self.value_head = nn.Linear(hidden_dim, 1)
10        self.bce = nn.BCEWithLogitsLoss()
11
12    def forward(self, hidden_states, input_ids, step_labels=None):
13        # hidden_states shape: (batch_size, seq_len, hidden_dim)
14        # input_ids shape: (batch_size, seq_len)
15        # step_labels: tensor containing float correctness labels {0.0, 1.0} at step boundaries,
16        # shape: (batch_size, seq_len)
17        logits = self.value_head(hidden_states).squeeze(-1) # (batch_size, seq_len)
18
19        step_mask = (input_ids == self.step_token_id)
20
21        loss = None
22        if step_labels is not None:
23            # Mask out non-step boundary token positions in loss calculation
24            valid_logits = logits[step_mask]
25            valid_labels = step_labels[step_mask]
26            loss = self.bce(valid_logits, valid_labels)
27
28        # Compute predicted step probabilities
29        probs = torch.sigmoid(logits) * step_mask.float()
30        return probs, loss

```

Part III

Parameter-Efficient Fine-Tuning (PEFT)

Low-Rank Adaptation (LoRA)

Mathematical Foundation

LoRA represents the weight update matrix $\Delta W \in \mathbb{R}^{k \times d}$ for a linear projection layer as the product of two low-rank matrices $B \in \mathbb{R}^{k \times r}$ and $A \in \mathbb{R}^{r \times d}$, where the rank satisfies $r \ll \min(d, k)$:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} B A x \quad (60)$$

where α is a constant scaling parameter. The base weight matrix $W_0 \in \mathbb{R}^{k \times d}$ is frozen; only B and A are updated. Under a loss $\mathcal{L}$, the gradient propagation rules using the chain rule are derived as follows: Let $z = Ax \in \mathbb{R}^r$ be the intermediate bottleneck activation. Then $h = W_0 x + \frac{\alpha}{r} B z$. The gradient with respect to output activation is $\nabla_h \mathcal{L} \in \mathbb{R}^{k \times 1}$ (represented as a column vector). The gradient with respect to the intermediate bottleneck representation z :

$$\nabla_z \mathcal{L} = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L} \in \mathbb{R}^{r \times 1} \quad (61)$$

The gradient with respect to the matrix B :

$$\nabla_B \mathcal{L} = \frac{\alpha}{r} \nabla_h \mathcal{L} z^\top = \frac{\alpha}{r} \nabla_h \mathcal{L} x^\top A^\top \in \mathbb{R}^{k \times r} \quad (62)$$

The gradient with respect to the matrix A :

$$\nabla_A \mathcal{L} = \nabla_z \mathcal{L} x^\top = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L} x^\top \in \mathbb{R}^{r \times d} \quad (63)$$

To eliminate latency at deployment, the weights are merged directly into the base model:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} B A \quad (64)$$

Architecture Flow Diagram

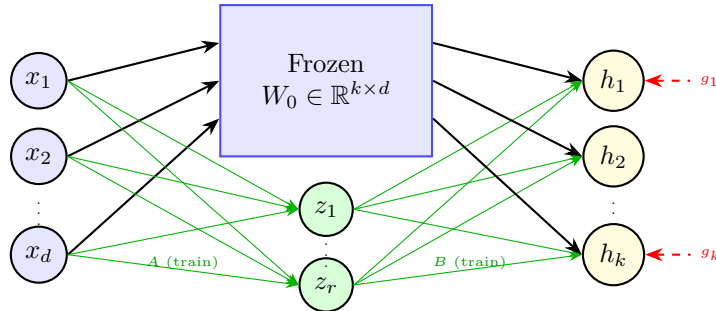


Figure 11: LoRA: input flows through frozen W_0 and trainable low-rank path $A \rightarrow B$; outputs are summed.

State Transition Table

Property	Before	After
Base weights W_0	requires_grad = True	Frozen (False)
Trainable params	$k \times d$	$r(k + d) \ll kd$
Matrix B	Does not exist	$\mathbb{R}^{k \times r}$, init = 0
Matrix A	Does not exist	$\mathbb{R}^{r \times d}$, init $\sim \mathcal{N}$
Inference overhead	None	Extra BA multiply (or merged)

Table 11: State properties before and after LoRA injection.

Python Implementation

```
1 import torch
2 import torch.nn as nn
3
4 class LoRALinear(nn.Module):
5     def __init__(self, in_features, out_features, r=8, alpha=16):
6         super().__init__()
7         self.linear = nn.Linear(in_features, out_features)
8         self.linear.weight.requires_grad = False # Freeze base weight
9
10        self.r = r
11        self.alpha = alpha
12        self.scaling = alpha / r
13
14        # Low-rank adapter matrices
15        self.lora_A = nn.Parameter(torch.zeros(r, in_features))
16        self.lora_B = nn.Parameter(torch.zeros(out_features, r))
17
18        # Initialization
19        nn.init.kaiming_uniform_(self.lora_A, a=5**0.5)
20        nn.init.zeros_(self.lora_B) # B is initialized to 0, making Delta W = 0 initially
21
22    def merge_weights(self):
23        # Update base weights directly
24        with torch.no_grad():
25            self.linear.weight.add_(self.scaling * (self.lora_B @ self.lora_A))
26
27    def unmerge_weights(self):
28        # Subtract back the low rank product
29        with torch.no_grad():
30            self.linear.weight.sub_(self.scaling * (self.lora_B @ self.lora_A))
31
32    def forward(self, x):
33        # Compute forward pass: W0*x + (alpha/r)*B*A*x
34        base_out = self.linear(x)
35        lora_out = (x @ self.lora_A.t() @ self.lora_B.t()) * self.scaling
36        return base_out + lora_out
```

Quantized LoRA (QLoRA)

Mathematical Foundation

QLoRA quantizes the base weight W_0 to 4-bit NormalFloat (NF4) and uses Double Quantization (DQ) to minimize memory requirements. NF4 divides the standard normal distribution $\mathcal{N}(0, 1)$ into $2^k = 16$ equal-area intervals. The quantization levels q_i are calculated using the standard normal quantile function Q_X :

$$q_i = \frac{1}{2} \left(Q_X \left(\frac{i}{2^k} \right) + Q_X \left(\frac{i+1}{2^k} \right) \right) \quad (65)$$

The base weights are normalized to $[-1, 1]$ using block-wise scaling. Let the block size be $B = 64$. For a block W_b , the scale is $c_b = \max(|W_b|)$. The quantized values are:

$$q_t = \arg \min_{i \in \{0, \dots, 15\}} \left| \frac{W_{b,t}}{c_b} - q_i \right| \quad (66)$$

Double Quantization (DQ) quantizes the scaling factors c_b themselves to 8-bit floats (c_b^{FP8}) with a second scale block size of 256, reducing scale memory overhead from $32/64 = 0.5$ bits/weight to $8/64 + 32/(64 \times 256) \approx 0.127$ bits/weight. During backpropagation, the NF4 weights are dequantized to FP16/BF16 to compute outputs, and gradients are computed only for the low-rank parameters:

$$h = \text{dequant}(c_b, W^{\text{NF4}})x + \frac{\alpha}{r} B A x \quad (67)$$

Architecture Flow Diagram

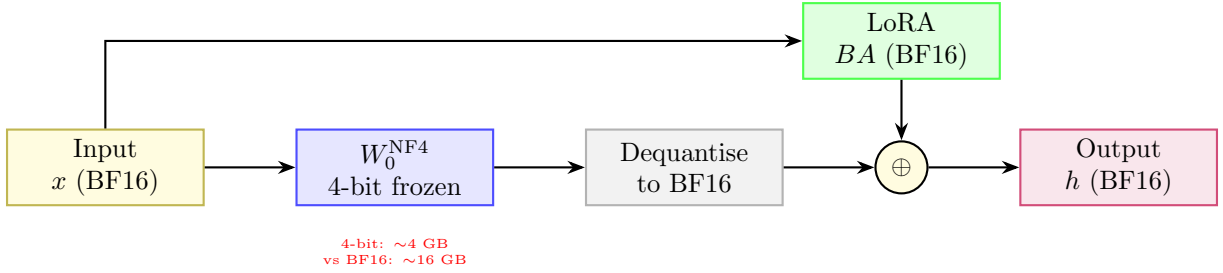


Figure 12: QLoRA: 4-bit frozen base dequantised at runtime; LoRA adapters trained in BF16.

State Transition Table

Property	Before	After
W_0 precision	BF16 / FP16	NF4 (4-bit)
GPU memory for 70B model	~140 GB	~35 GB
LoRA adapter precision	N/A	BF16
Training fidelity	Full	Near-full (straight-through)
Inference latency	Baseline	+dequant overhead

Table 12: State properties before and after QLoRA.

Python Implementation

```

1 import torch
2 import torch.nn as nn
3
4 class QLoRALinear(nn.Module):
5     def __init__(self, in_features, out_features, r=8, alpha=16, block_size=64):
6         super().__init__()
7         self.in_features = in_features
8         self.out_features = out_features
9         self.block_size = block_size
10        self.scaling = alpha / r
11

```



```

12     # Simulated NF4 weights stored as half precision, frozen
13     self.weight_nf4 = nn.Parameter(torch.randn(out_features, in_features).half(), requires_grad=
False)
14     # Block-wise scales for dequantization
15     self.scales = nn.Parameter(torch.zeros(out_features, (in_features + block_size - 1) //
block_size).half(), requires_grad=False)
16     self.compute_simulated_scales()
17
18     self.lora_A = nn.Parameter(torch.zeros(r, in_features))
19     self.lora_B = nn.Parameter(torch.zeros(out_features, r))
20     nn.init.kaiming_uniform_(self.lora_A, a=5*0.5)
21     nn.init.zeros_(self.lora_B)
22
23     def compute_simulated_scales(self):
24         # Group weights into blocks and compute max values
25         w = self.weight_nf4.data
26         for i in range(self.scales.size(1)):
27             start = i * self.block_size
28             end = min(start + self.block_size, self.in_features)
29             block = w[:, start:end]
30             self.scales[:, i] = torch.max(torch.abs(block), dim=-1).values
31
32     def dequantize(self):
33         # Dequantize simulated NF4 block-by-block
34         dequantized = torch.zeros_like(self.weight_nf4)
35         for i in range(self.scales.size(1)):
36             start = i * self.block_size
37             end = min(start + self.block_size, self.in_features)
38             scale = self.scales[:, i].unsqueeze(-1)
39             dequantized[:, start:end] = self.weight_nf4[:, start:end] * scale
40         return dequantized
41
42     def forward(self, x):
43         # Dequantize base weights on the fly
44         weight = self.dequantize()
45         base_out = torch.nn.functional.linear(x, weight)
46         lora_out = (x @ self.lora_A.t() @ self.lora_B.t()) * self.scaling
47         return base_out + lora_out

```

Adaptive LoRA (AdaLoRA)

Mathematical Foundation

AdaLoRA parameterizes incremental weight updates using a singular value decomposition (SVD) representation to dynamically adjust the adapter rank during training:

$$\Delta W = P\Lambda Q \quad (68)$$

where $P \in \mathbb{R}^{d \times r}$ and $Q \in \mathbb{R}^{r \times k}$ are left and right singular vectors, and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_r) \in \mathbb{R}^{r \times r}$ contains singular values. To prevent SVD divergence and maintain parameter stability, we enforce orthogonality constraints:

$$\mathcal{L}_{\text{orth}}(P, Q) = \gamma (\|P^\top P - I\|_F^2 + \|QQ^\top - I\|_F^2) \quad (69)$$

We estimate the training importance of each singular triplet $(P_{*,i}, \lambda_i, Q_{i,*})$ using gradient-weight product magnitude:

$$S_i = s(\lambda_i) + \frac{1}{d} \sum_{j=1}^d s(P_{j,i}) + \frac{1}{k} \sum_{j=1}^k s(Q_{i,j}) \quad (70)$$

where $s(\theta) = |\theta \cdot \nabla_\theta \mathcal{L}|$. At designated intervals, triplets with the lowest importance scores S_i are pruned by zeroing out their singular values λ_i , dynamically reducing the adapter rank budget.

Architecture Flow Diagram

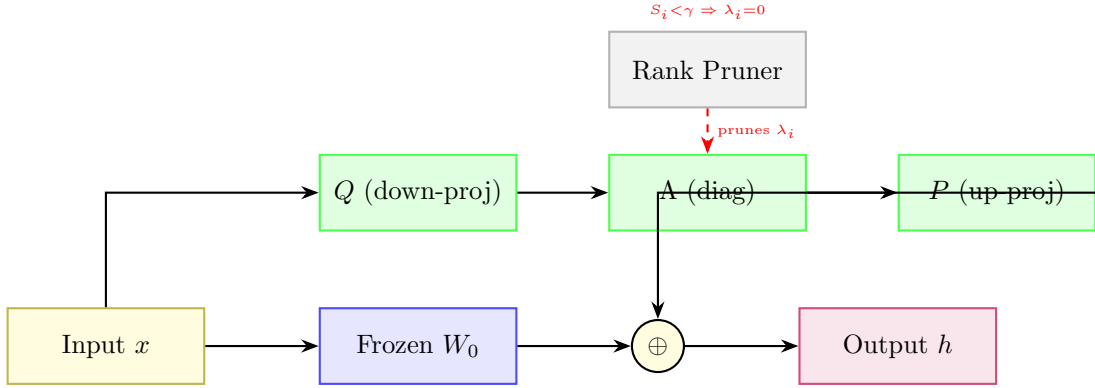


Figure 13: AdaLoRA: SVD-decomposed update with importance-based rank pruning per layer.

State Transition Table

Property	Before	After
Rank per layer	Fixed r	Adaptive (varies by importance)
Update structure	BA product	PAQ SVD triplet
Parameter budget	Fixed	Redistributed across layers
Low-importance ranks	Included	Pruned ($\lambda_i = 0$)
Training stability	Standard	Constrained by SVD orthogonality

Table 13: State properties before and after AdaLoRA.

Python Implementation

```

1 import torch
2 import torch.nn as nn
3
4 class AdaLoRALinear(nn.Module):
5     def __init__(self, in_features, out_features, r=8, gamma=0.1):
6         super().__init__()
7         self.linear = nn.Linear(in_features, out_features)
8         self.linear.weight.requires_grad = False
9         self.r = r

```

```

10     self.gamma = gamma
11
12     # SVD Parameterization
13     self.lora_P = nn.Parameter(torch.randn(out_features, r))
14     self.lora_E = nn.Parameter(torch.randn(r)) # Singular values
15     self.lora_Q = nn.Parameter(torch.randn(r, in_features))
16
17     nn.init.orthogonal_(self.lora_P)
18     nn.init.zeros_(self.lora_E)
19     nn.init.orthogonal_(self.lora_Q)
20
21     def compute_orthogonality_loss(self):
22         #  $P^T * P - I$ 
23         p_orth = torch.matmul(self.lora_P.t(), self.lora_P) - torch.eye(self.r, device=self.lora_P.
device)
24         #  $Q * Q^T - I$ 
25         q_orth = torch.matmul(self.lora_Q, self.lora_Q.t()) - torch.eye(self.r, device=self.lora_Q.
device)
26         return self.gamma * (torch.norm(p_orth, p='fro')**2 + torch.norm(q_orth, p='fro')**2)
27
28     def prune_rank(self, keep_mask):
29         # keep_mask: boolean tensor of shape (r,) indicating active singular values
30         with torch.no_grad():
31             self.lora_E.copy_(self.lora_E * keep_mask.float())
32
33     def forward(self, x):
34         base_out = self.linear(x)
35         #  $\Delta W = P * \text{diag}(E) * Q$ 
36         delta_W = self.lora_P @ torch.diag(self.lora_E) @ self.lora_Q
37         lora_out = x @ delta_W.t()
38         return base_out + lora_out

```

Weight-Decomposed LoRA (DoRA)

Mathematical Foundation

DoRA decomposes the weight matrix $W \in \mathbb{R}^{k \times d}$ into its magnitude vector $m \in \mathbb{R}^k$ and direction matrix $V \in \mathbb{R}^{k \times d}$:

$$W = m \cdot \frac{V}{\|V\|_r} = m \cdot \frac{W_0 + \Delta W}{\|W_0 + \Delta W\|_r} \quad (71)$$

where $\|\cdot\|_r$ is the L2 norm computed along each row (corresponding to output channels). In parameter-efficient training, we freeze the direction matrix to the base weight $V = W_0$ and introduce low-rank directional updates using adapters $\Delta V = \frac{\alpha}{r}BA$ (where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{k \times r}$):

$$W = m \cdot \frac{W_0 + \frac{\alpha}{r}BA}{\|W_0 + \frac{\alpha}{r}BA\|_r} \quad (72)$$

Only the magnitude vector m and low-rank directional matrices B and A are trainable. The gradient flow for the directional components is:

$$\nabla_V \mathcal{L} = \frac{m}{\|V\|_c} \left(\nabla_W \mathcal{L} - \frac{V}{\|V\|_c^2} \odot (V^\top \nabla_W \mathcal{L}) \right) \quad (73)$$

This mathematically separates the updates to magnitude and direction, mimicking full parameter fine-tuning behavior while retaining low parameter overhead.

Architecture Flow Diagram

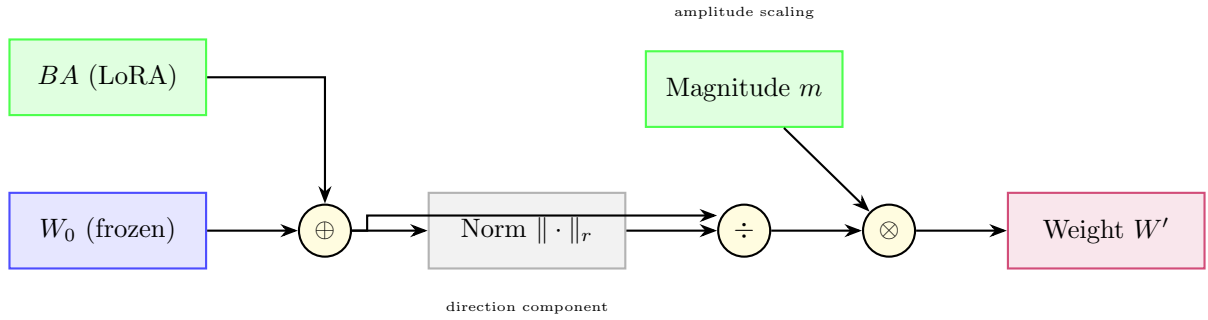


Figure 14: DoRA: weight decomposed into direction (LoRA-adapted) and magnitude (trainable scalar).

State Transition Table

Property	Before (LoRA)	After (DoRA)
Magnitude component	Implicit in W_0	Explicit trainable m
Direction component	$W_0 + BA$ unnorm	Column-normalised
Full FT similarity	Lower	Higher
Trainable params	$r(d+k)$	$r(d+k) + k$
Catastrophic forgetting	Low	Lower

Table 14: State properties comparing LoRA and DoRA.

Python Implementation

```

1 import torch
2 import torch.nn as nn
3
4 class DoRALinear(nn.Module):
5     def __init__(self, in_features, out_features, r=8, alpha=16):
6         super().__init__()
7         self.linear = nn.Linear(in_features, out_features)
8         self.linear.weight.requires_grad = False
9
10        self.r = r

```

```

11     self.alpha = alpha
12     self.scaling = alpha / r
13
14     # Directional adapter matrices
15     self.lora_A = nn.Parameter(torch.zeros(r, in_features))
16     self.lora_B = nn.Parameter(torch.zeros(out_features, r))
17     nn.init.kaiming_uniform_(self.lora_A, a=5*0.5)
18     nn.init.zeros_(self.lora_B)
19
20     # Magnitude parameters m, initialized to column norm of base weights
21     W0 = self.linear.weight.data
22     self.m = nn.Parameter(torch.norm(W0, p=2, dim=1))
23
24 def forward(self, x):
25     W0 = self.linear.weight
26     delta_W = (self.lora_B @ self.lora_A) * self.scaling
27
28     # Calculate updated direction matrix V
29     V = W0 + delta_W
30     V_norm = torch.norm(V, p=2, dim=1, keepdim=True)
31
32     # Construct weight matrix with separate magnitude and direction components
33     W = self.m.unsqueeze(1) * (V / V_norm)
34     return torch.nn.functional.linear(x, W, self.linear.bias)

```

IA³ (Infused Adapter by Inhibiting and Amplifying Inner Activations)

Mathematical Foundation

IA³ introduces element-wise scaling vectors to directly modify inner activations in attention keys, values, and intermediate feed-forward layers. Let the scaling vectors be $l_K \in \mathbb{R}^d$, $l_V \in \mathbb{R}^d$, and $l_{FFN} \in \mathbb{R}^{d_{FFN}}$. The attention layer activations are computed as:

$$\tilde{K} = K \odot l_K = K \cdot \text{diag}(l_K) \quad (74)$$

$$\tilde{V} = V \odot l_V = V \cdot \text{diag}(l_V) \quad (75)$$

$$\text{Attention}(Q, \tilde{K}, \tilde{V}) = \text{softmax}\left(\frac{Q\tilde{K}^\top}{\sqrt{d_k}}\right)\tilde{V} \quad (76)$$

In the feed-forward network (FFN), let W_1 and W_2 be the weight matrices. The intermediate activations are scaled before the output projection:

$$\text{FFN}(x) = \sigma(xW_1 \odot l_{FFN})W_2 = \sigma(xW_1 \cdot \text{diag}(l_{FFN}))W_2 \quad (77)$$

The base model weights W_Q, W_K, W_V, W_1, W_2 are frozen. This diagonal matrix transformation scales the columns of the projection matrices with minimal parameter updates.

Architecture Flow Diagram

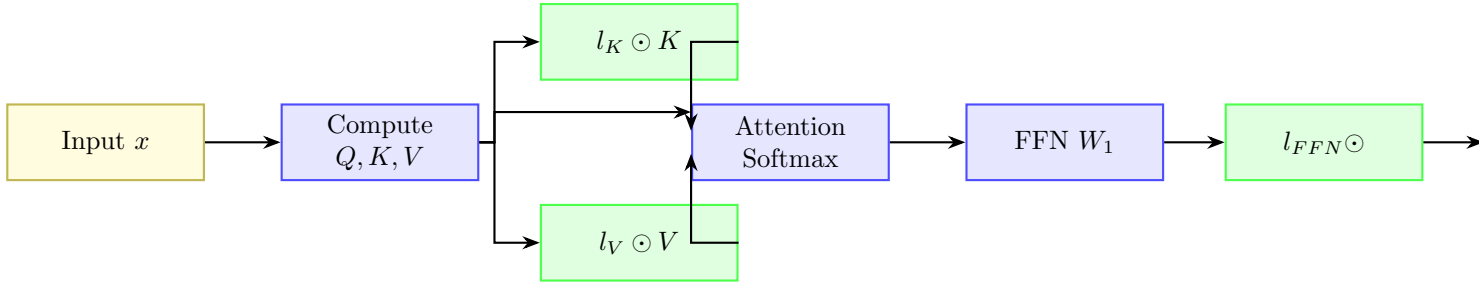


Figure 15: IA³: three tiny scaling vectors rescale keys, values, and FFN intermediate activations.

State Transition Table

Property	Before	After
Trainable params	0 (frozen)	$3dL$ scalars
Init value of l vectors	N/A	1 (identity behaviour)
K, V, FFN activations	Unchanged	Element-wise scaled
Param overhead vs LoRA	N/A	$\sim 10\times$ fewer params
Task adaptation strength	None	Moderate

Table 15: State properties before and after IA³ injection.

Python Implementation

```

1 import torch
2 import torch.nn as nn
3
4 class IA3AttentionWrapper(nn.Module):
5     def __init__(self, key_dim, value_dim):
6         super().__init__()
7         # Scaling vectors initialized to ones
8         self.l_K = nn.Parameter(torch.ones(key_dim))
9         self.l_V = nn.Parameter(torch.ones(value_dim))
10
11     def scale_keys(self, K):
12         # K shape: (batch_size, num_heads, seq_len, head_dim) or (batch_size, seq_len, key_dim)
13         return K * self.l_K

```

```

14
15     def scale_values(self, V):
16         return V * self.l_V
17
18 class IA3FFNWrapper(nn.Module):
19     def __init__(self, ffn_dim):
20         super().__init__()
21         self.l_FFN = nn.Parameter(torch.ones(ffn_dim))
22
23     def scale_hidden(self, hidden_states):
24         # hidden_states: intermediate feedforward layer activations
25         return hidden_states * self.l_FFN

```

Adapter Tuning

Mathematical Foundation

Adapter tuning inserts small bottleneck layers into the transformer block, typically after attention and feed-forward modules. Let $h \in \mathbb{R}^d$ be the input hidden state. In the Pfeiffer architecture, a single adapter is inserted per block. In the Houlsby architecture, two adapters are inserted per block. The mathematical transformation for an adapter block is:

$$h' = h + \sigma(hW_{down})W_{up} \quad (78)$$

where $W_{down} \in \mathbb{R}^{d \times r}$ projects to a bottleneck dimension $r \ll d$, σ is an activation function (typically GELU), and $W_{up} \in \mathbb{R}^{r \times d}$ projects back to the original dimension. To ensure the adapter behaves as an identity mapping at the start of training, W_{up} is initialized to zero:

$$h'|_{t=0} = h + \sigma(hW_{down}) \cdot 0 = h \quad (79)$$

Architecture Flow Diagram

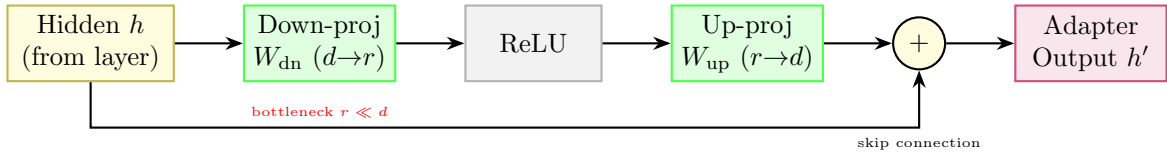


Figure 16: Adapter module: down-project to rank r , apply nonlinearity, up-project back, plus skip connection.

State Transition Table

Property	Before	After
Sub-layer output path	Direct pass-through	+adapter bottleneck
Transformer weights	Trainable	Frozen
New params per layer	0	$2dr + 2d$
Adapter init	N/A	Near-identity ($W_{dn} \sim \mathcal{N}$)
Inference latency	Baseline	+two extra matmuls per layer

Table 16: State properties before and after adapter insertion.

Python Implementation

```

1 import torch
2 import torch.nn as nn
3
4 class BottleneckAdapter(nn.Module):
5     def __init__(self, d_model, bottleneck_dim=16):
6         super().__init__()
7         self.down_proj = nn.Linear(d_model, bottleneck_dim)
8         self.act = nn.GELU()
9         self.up_proj = nn.Linear(bottleneck_dim, d_model)
10        self.layer_norm = nn.LayerNorm(d_model)
11
12        # Initialize up projection to zero to act as identity mapping at start
13        nn.init.normal_(self.down_proj.weight, std=1e-3)
14        nn.init.zeros_(self.up_proj.weight)
15
16    def forward(self, x):
17        residual = x
18        out = self.down_proj(x)
19        out = self.act(out)
20        out = self.up_proj(out)
21        # Apply layer normalization with residual connection
22        return self.layer_norm(residual + out)

```


Prefix Tuning

Mathematical Foundation

Prefix Tuning prepends continuous task-specific virtual prefix vectors $P_K, P_V \in \mathbb{R}^{l \times d}$ directly to the Key and Value matrices at every transformer layer. Let the original Key and Value matrices for sequence length T be $K, V \in \mathbb{R}^{T \times d}$. The modified Key and Value matrices become:

$$\tilde{K} = [P_K; K] \in \mathbb{R}^{(l+T) \times d} \quad (80)$$

$$\tilde{V} = [P_V; V] \in \mathbb{R}^{(l+T) \times d} \quad (81)$$

To stabilize optimization, during training the prefix is reparameterized using a Multilayer Perceptron (MLP) over a small source embedding E_k :

$$P_K = \text{MLP}_K(E_k), \quad P_V = \text{MLP}_V(E_k) \quad (82)$$

where $E_k \in \mathbb{R}^{l \times d_{\text{mid}}}$. The MLP is discarded at inference, and only the generated static prefix matrices P_K, P_V are saved and used. The attention output is computed as:

$$\text{Attention}(Q, \tilde{K}, \tilde{V}) = \text{softmax}\left(\frac{Q[P_K; K]^\top}{\sqrt{d_k}}\right) [P_V; V] \quad (83)$$

Architecture Flow Diagram

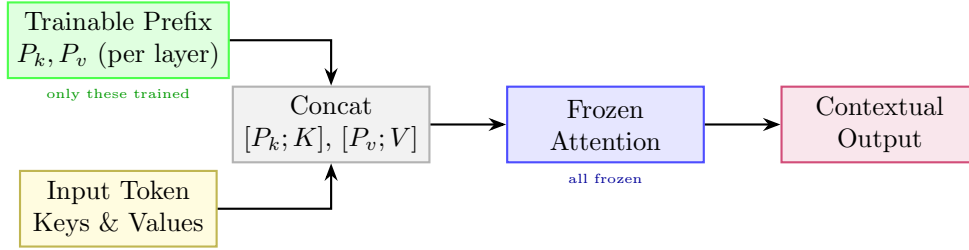


Figure 17: Prefix Tuning: trainable prefix vectors extend the key/value context at every attention layer.

State Transition Table

Property	Before	After
Attention context length	T tokens	$T + l$ tokens (prefix prepended)
Transformer params	Trainable	Fully frozen
Trainable params	0	$2 \times l \times d_h \times L$
Prefix init	N/A	Via MLP reparameterisation
Task switch at inference	Reload model	Swap prefix only

Table 17: State properties before and after prefix tuning.

Python Implementation

```

1 import torch
2 import torch.nn as nn
3
4 class PrefixTuningAttention(nn.Module):
5     def __init__(self, d_model, num_heads, prefix_len=10):
6         super().__init__()
7         self.d_model = d_model
8         self.num_heads = num_heads
9         self.head_dim = d_model // num_heads
10        self.prefix_len = prefix_len
11
12        # Key/Value projection layers
13        self.q_proj = nn.Linear(d_model, d_model)
14        self.k_proj = nn.Linear(d_model, d_model)
15        self.v_proj = nn.Linear(d_model, d_model)
  
```

```

16         self.out_proj = nn.Linear(d_model, d_model)
17
18     def forward(self, x, past_keys=None, past_values=None):
19         # past_keys, past_values: prefix cache parameters, shape (batch_size, num_heads, prefix_len,
20         head_dim)
21         b, t, c = x.size()
22
23         q = self.q_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
24         k = self.k_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
25         v = self.v_proj(x).view(b, t, self.num_heads, self.head_dim).transpose(1, 2)
26
27         if past_keys is not None and past_values is not None:
28             # Prepend virtual prefix keys/values to the computed sequences
29             k = torch.cat([past_keys, k], dim=-2)
30             v = torch.cat([past_values, v], dim=-2)
31
32         scores = torch.matmul(q, k.transpose(-2, -1)) / (self.head_dim ** 0.5)
33         attn_weights = torch.softmax(scores, dim=-1)
34         out = torch.matmul(attn_weights, v)
35         out = out.transpose(1, 2).contiguous().view(b, t, c)
36         return self.out_proj(out)

```

Prompt Tuning

Mathematical Foundation

Prompt tuning prepends p learnable embedding vectors $P \in \mathbb{R}^{p \times d_{emb}}$ to the input word representations:

$$\tilde{X} = [P; X_e] \in \mathbb{R}^{(p+T) \times d_{emb}} \quad (84)$$

where $X_e \in \mathbb{R}^{T \times d_{emb}}$ is the original sequence word embedding matrix. The entire transformer backbone parameters θ_{base} remain frozen, and gradients are backpropagated only to the prompt parameters P :

$$\nabla_P \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \tilde{X}_{1..p}} \quad (85)$$

During backpropagation, the gradients are passed through all layers of the frozen transformer and update only the prompt embeddings.

Architecture Flow Diagram

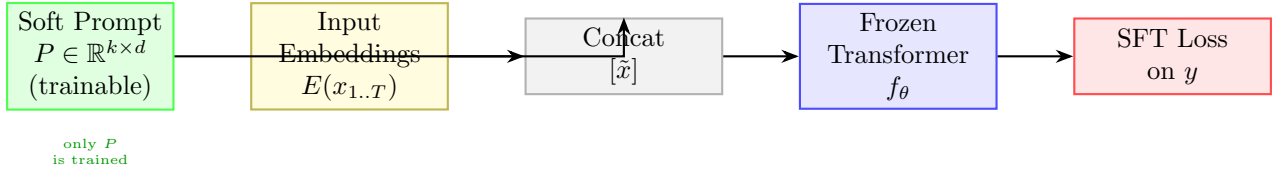


Figure 18: Prompt Tuning: soft tokens prepended at the embedding layer; the full transformer is frozen.

State Transition Table

Property	Before	After
Trainable params	Full model	$k \times d_{emb}$ only
Prompt format	Discrete hand-crafted	Continuous soft vectors
Transformer weights	All trainable	All frozen
Task switching	Reload fine-tuned model	Swap P only
Sensitivity to init	N/A	Strong (init from class labels helps)

Table 18: State properties before and after prompt tuning.

Python Implementation

```

1 import torch
2 import torch.nn as nn
3
4 class PromptTuningEmbedding(nn.Module):
5     def __init__(self, base_embeddings, prompt_len=10):
6         super().__init__()
7         self.base_embeddings = base_embeddings
8         for p in self.base_embeddings.parameters():
9             p.requires_grad = False
10
11         self.prompt_len = prompt_len
12         self.prompt_embeddings = nn.Parameter(torch.randn(prompt_len, base_embeddings.embedding_dim))
13
14     def forward(self, input_ids):
15         # input_ids shape: (batch_size, seq_len)
16         batch_size = input_ids.size(0)
17         base_embs = self.base_embeddings(input_ids) # (batch_size, seq_len, d_emb)
18
19         # Expand prompt embeddings to batch size
20         prompts = self.prompt_embeddings.unsqueeze(0).expand(batch_size, -1, -1) # (batch_size, prompt_len, d_emb)
21
22         # Concatenate prompt prefix with token embeddings
23         return torch.cat([prompts, base_embs], dim=1)
  
```

P-Tuning v2

Mathematical Foundation

P-Tuning v2 extends prompt tuning by prepending learnable virtual tokens $P^{(l)} \in \mathbb{R}^{p \times 2d_{\text{attn}}}$ directly to the attention key and value states at every layer $l \in [1, L]$ of the network. Unlike prefix tuning, P-Tuning v2 removes the MLP reparameterization network, directly optimizing the prefix parameters $P^{(l)}$:

$$\tilde{K}^{(l)} = [P_K^{(l)}; K^{(l)}] \tag{86}$$

$$\tilde{V}^{(l)} = [P_V^{(l)}; V^{(l)}] \tag{87}$$

This deep prefix injection resolves the capacity limitation of shallow prompt tuning, allowing small models (< 10B parameters) to achieve performance comparable to full fine-tuning.

Architecture Flow Diagram

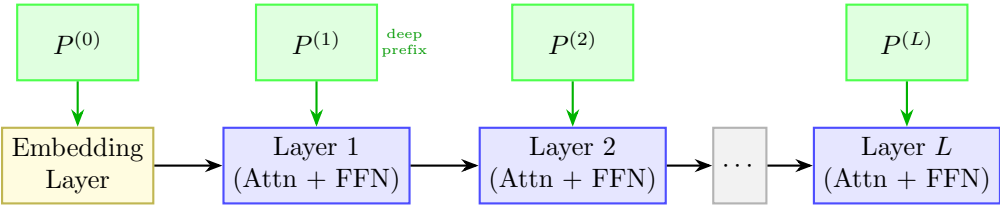


Figure 19: P-Tuning v2: independent trainable prefix tokens injected at every transformer layer.

State Transition Table

Property	Before (Prompt Tuning)	After (P-Tuning v2)
Prefix layers	Embedding only	All L layers
Trainable params	$k \times d$	$L \times l \times d$
NLU performance	Degrades at small scale	Competitive at all scales
Conditioning depth	Shallow (first layer)	Deep (every layer)
Task-specific storage	Single P matrix	L prefix matrices

Table 19: State properties comparing Prompt Tuning and P-Tuning v2.

Python Implementation

```

1 import torch
2 import torch.nn as nn
3
4 class PTuningV2Model(nn.Module):
5     def __init__(self, num_layers, num_heads, head_dim, prefix_len=10):
6         super().__init__()
7         self.num_layers = num_layers
8         self.num_heads = num_heads
9         self.head_dim = head_dim
10        self.prefix_len = prefix_len
11
12        # Directly optimized layer-wise prefixes
13        self.prefixes = nn.ParameterList([
14            nn.Parameter(torch.randn(2, num_heads, prefix_len, head_dim))
15            for _ in range(num_layers)
16        ])
17
18    def forward(self, batch_size, layer_idx):
19        layer_prefixes = self.prefixes[layer_idx]
20        return layer_prefixes.unsqueeze(1).expand(-1, batch_size, -1, -1, -1)

```

Contents

Part IV

Preference Alignment Methods

Reinforcement Learning from Human Feedback (RLHF)

Mathematical Foundation

RLHF has two phases. First, a Reward Model (RM) r_ϕ is trained on human-ranked pairs $(y_w \succ y_l \mid x)$ with the Bradley-Terry objective:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x, y_w, y_l)} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))] \quad (88)$$

Then, the policy π_θ is optimised via PPO to maximise expected reward while staying close to the reference policy π_{ref} :

$$\mathcal{J}_{\text{RLHF}} = \mathbb{E}_{x, y \sim \pi_\theta} [r_\phi(x, y)] - \beta \mathbb{KL}[\pi_\theta(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x)] \quad (89)$$

Architecture Flow Diagram

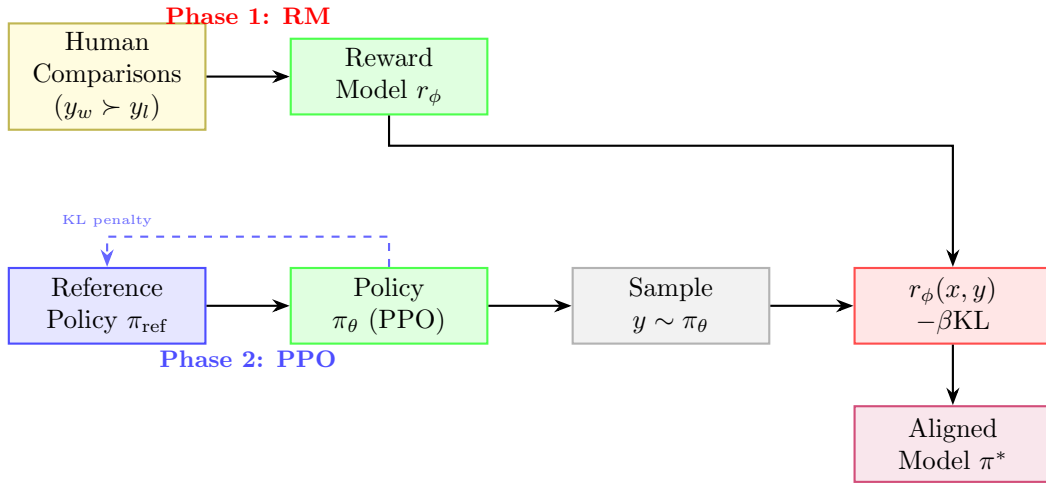


Figure 20: RLHF: reward model trained on human comparisons then used to guide PPO policy optimisation.

State Transition Table

Property	Before	After
Policy type	SFT checkpoint	RLHF-aligned policy
Reward signal	None	Human preference score
KL from reference	0	Bounded by β
Human preference win-rate	$\sim 50\%$	$> 70\%$
Training complexity	Single loop	Two-phase (RM + PPO)

Table 20: State properties before and after RLHF.

Direct Preference Optimization (DPO)

Mathematical Foundation

DPO reformulates RLHF into a single supervised objective, bypassing the explicit reward model. Starting from the closed-form optimal policy under KL constraint, the loss becomes:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right] \quad (90)$$

The implicit reward is $r(x, y) = \beta \log(\pi_{\theta}(y|x)/\pi_{\text{ref}}(y|x))$. No sampling loop or value network is needed.

Architecture Flow Diagram

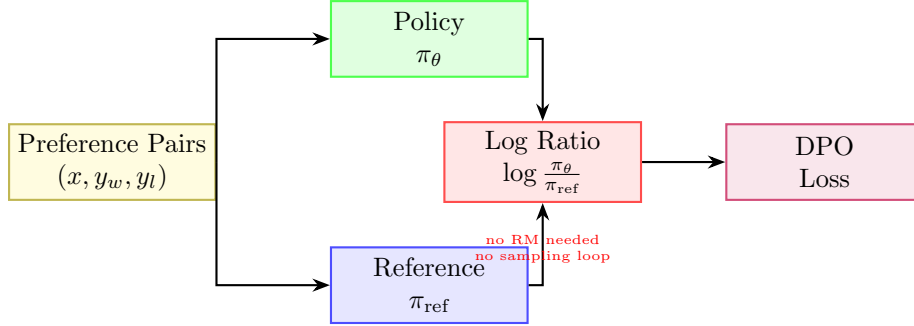


Figure 21: DPO: both policy and frozen reference score chosen/rejected responses; loss uses their log-ratio difference.

State Transition Table

Property	Before (RLHF)	After (DPO)
Reward model	Explicit r_{ϕ}	Implicit in log-ratio
Sampling loop	Yes (PPO rollouts)	No
Reference policy	Frozen copy	Frozen copy
Training stability	Sensitive to reward hacking	More stable
Compute cost	High (2-phase)	Lower (single SFT-like pass)

Table 21: State properties comparing RLHF and DPO.

Identity Preference Optimization (IPO)

Mathematical Foundation

IPO fixes a theoretical limitation of DPO: when the optimal policy places all mass on y_w , the DPO log-ratio diverges. IPO introduces a bounded objective by replacing the log-sigmoid with a squared-error hinge on the preference gap h :

$$h(x, y_w, y_l) = \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \quad (91)$$

$$\mathcal{L}_{\text{IPO}} = \mathbb{E} \left[\left(h(x, y_w, y_l) - \frac{1}{2\beta} \right)^2 \right] \quad (92)$$

The target $\frac{1}{2\beta}$ acts as a regulariser preventing the preference gap from growing unboundedly.

Architecture Flow Diagram

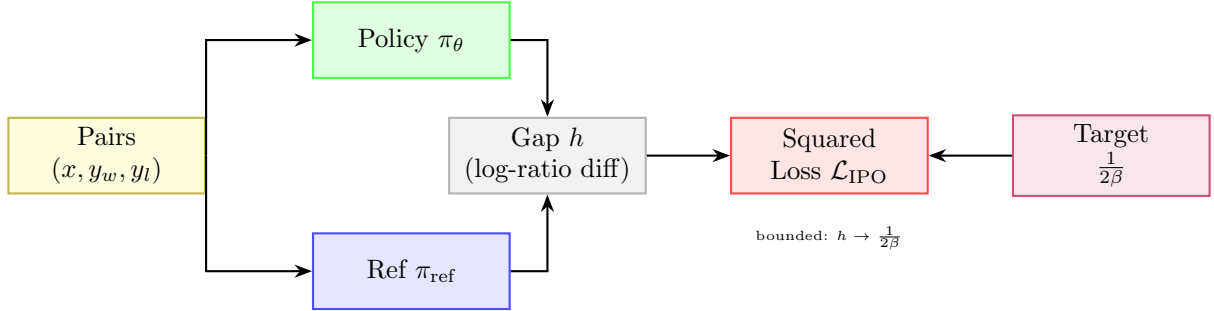


Figure 22: IPO: like DPO but the log-ratio gap is penalised toward a finite target $1/(2\beta)$.

State Transition Table

Property	Before (DPO)	After (IPO)
Loss type	Log-sigmoid	Squared error
Divergence at optimum	Unbounded	Bounded by $\frac{1}{2\beta}$
Target preference gap	Maximise	Converge to $\frac{1}{2\beta}$
Regularisation	KL implicit	Explicit squared bound
Training saturation	Can saturate	More stable

Table 22: State properties comparing DPO and IPO.

Odds Ratio Preference Optimization (ORPO)

Mathematical Foundation

ORPO combines SFT loss and an odds-ratio contrast in a single objective no reference model needed. The odds of generating y under policy π_θ is $\text{odds}(y|x) = \pi_\theta(y|x)/(1 - \pi_\theta(y|x))$. The combined loss is:

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} - \lambda \mathbb{E} \left[\log \sigma \left(\log \frac{\text{odds}(y_w|x)}{\text{odds}(y_l|x)} \right) \right] \quad (93)$$

The SFT term increases $\pi_\theta(y_w|x)$ while the odds-ratio term simultaneously penalises y_l without requiring a frozen reference copy.

Architecture Flow Diagram

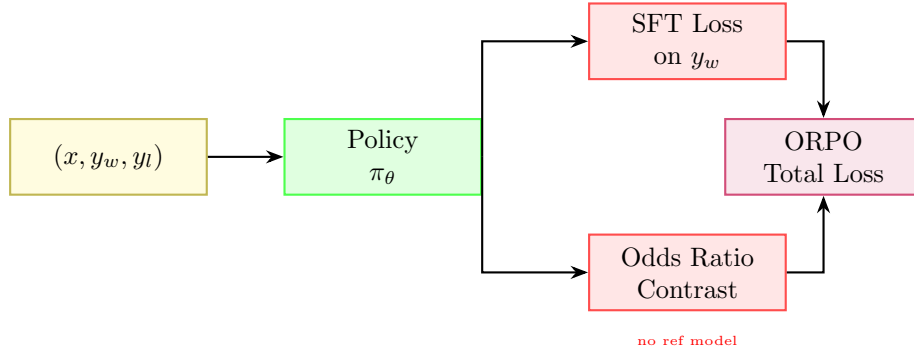


Figure 23: ORPO: SFT loss on the chosen response combined with an odds-ratio preference contrast.

State Transition Table

Property	Before	After
Reference model	Required (DPO/RLHF)	Not required
Training phases	2 (SFT then align)	1 (joint)
Loss terms	Single	SFT + odds-ratio
Memory overhead	2× model	1× model only
Data format	Prompt + completion	Triplet (x, y_w, y_l)

Table 23: State properties before and after ORPO.

Kahneman-Tversky Optimization (KTO)

Mathematical Foundation

KTO aligns LLMs by maximising a utility function inspired by Kahneman-Tversky prospect theory. It operates on *individual* samples labelled good ($z = 1$) or bad ($z = 0$), not pairwise comparisons:

$$v(y, z) = \begin{cases} w_+(r(x, y) - r_{\text{KL}}) & z = 1 \\ w_-(r(x, y) - r_{\text{KL}}) & z = 0 \end{cases} \quad (94)$$

$$\mathcal{L}_{\text{KTO}} = \mathbb{E}[1 - v(y, z)] \quad (95)$$

where $r_{\text{KL}} = \beta \text{KL}[\pi_\theta || \pi_{\text{ref}}]$ is a KL-penalty baseline. The asymmetric weights $w_+ \neq w_-$ model loss-aversion.

Architecture Flow Diagram

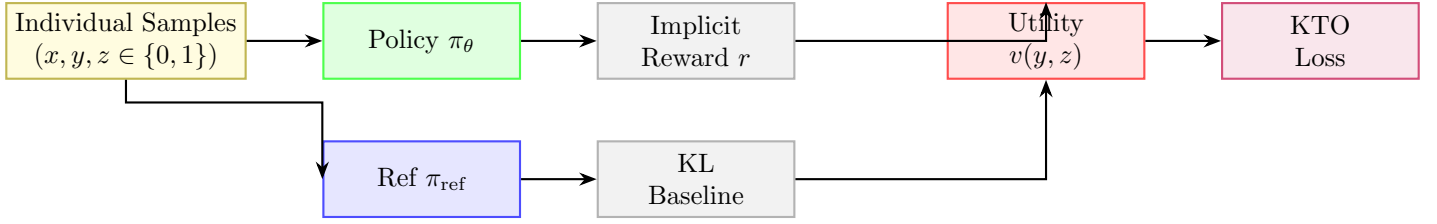


Figure 24: KTO: each sample is individually labelled good/bad; utility function uses prospect theory asymmetry.

State Transition Table

Property	Before	After
Data format	Pairwise (y_w, y_l)	Individual (y, z)
Data collection effort	High (pairwise)	Lower (binary label)
Loss-aversion modelling	None	Asymmetric w_+, w_-
Reference model	Optional	Used for KL baseline
Applicable at scale	Full pairs required	Works with any ratio

Table 24: State properties before and after KTO.

Simple Preference Optimization (SimPO)

Mathematical Foundation

SimPO eliminates the reference model by using average log-probability as an implicit reward and adding a target reward margin γ :

$$r_{\text{SimPO}}(x, y) = \frac{\beta}{|y|} \log \pi_{\theta}(y | x) \quad (96)$$

$$\mathcal{L}_{\text{SimPO}} = -\mathbb{E}[\log \sigma(r_{\text{SimPO}}(x, y_w) - r_{\text{SimPO}}(x, y_l) - \gamma)] \quad (97)$$

Length normalisation prevents the model from favouring short responses, and $\gamma > 0$ enforces a minimum gap between chosen and rejected rewards.

Architecture Flow Diagram

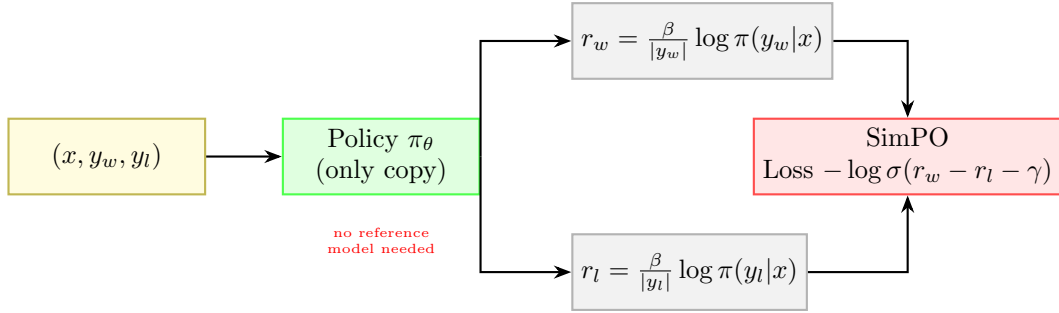


Figure 25: SimPO: length-normalised log-probs replace the reference log-ratio; margin γ enforces quality gap.

State Transition Table

Property	Before (DPO)	After (SimPO)
Reference model	Required	Not needed
Reward definition	Log-ratio π/π_{ref}	Length-norm avg log-prob
Length bias	Possible	Mitigated by $1/ y $
Margin hyperparameter	None	$\gamma > 0$
Memory saving	None	No second model forward pass

Table 25: State properties comparing DPO and SimPO.

Constrained Preference Optimization (CPO)

Mathematical Foundation

CPO formulates alignment as a constrained optimisation: maximise reward while staying within a trust region ϵ of the reference policy measured in reverse KL divergence:

$$\max_{\pi_{\theta}} \mathbb{E}[r(x, y)] \quad \text{s.t.} \quad \mathbb{KL}[\pi_{\text{ref}} || \pi_{\theta}] \leq \epsilon \quad (98)$$

The Lagrangian relaxation yields a combined training objective:

$$\mathcal{L}_{\text{CPO}} = -\mathbb{E}[r(x, y)] + \mu (\mathbb{KL}[\pi_{\text{ref}} || \pi_{\theta}] - \epsilon) \quad (99)$$

where $\mu \geq 0$ is a dual variable updated online to enforce the constraint.

Architecture Flow Diagram

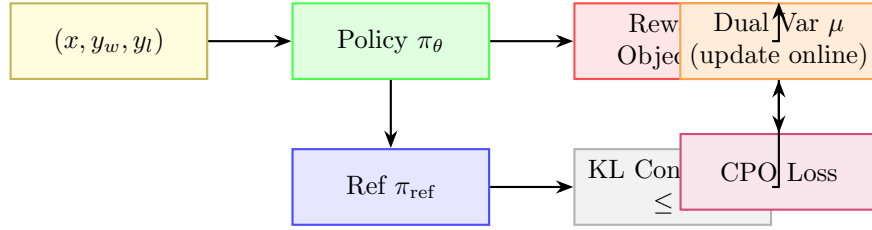


Figure 26: CPO: reward maximisation with an explicit KL constraint; dual variable μ adapts online.

State Transition Table

Property	Before	After
Constraint form	Soft KL penalty	Hard KL constraint $\leq \epsilon$
Dual variable μ	Fixed	Dynamically updated
Policy deviation	Unbounded	Bounded within trust region
Reward maximisation	Soft	Subject to constraint
Training dynamics	Standard	Saddle-point optimisation

Table 26: State properties before and after CPO.

Anchored Preference Optimization (APO)

Mathematical Foundation

APO introduces an anchor term that explicitly regularises the policy toward the reference model while simultaneously pushing chosen responses up and rejected responses down:

$$\mathcal{L}_{\text{APO}} = \mathbb{E} \left[\underbrace{-\log \sigma(\beta(r_w - r_l))}_{\text{preference}} + \lambda \underbrace{\left(\log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right)^2}_{\text{anchor}} \right] \quad (100)$$

The anchor term is minimised when both the chosen and rejected log-ratios equal a fixed reference, preventing the model from drifting.

Architecture Flow Diagram

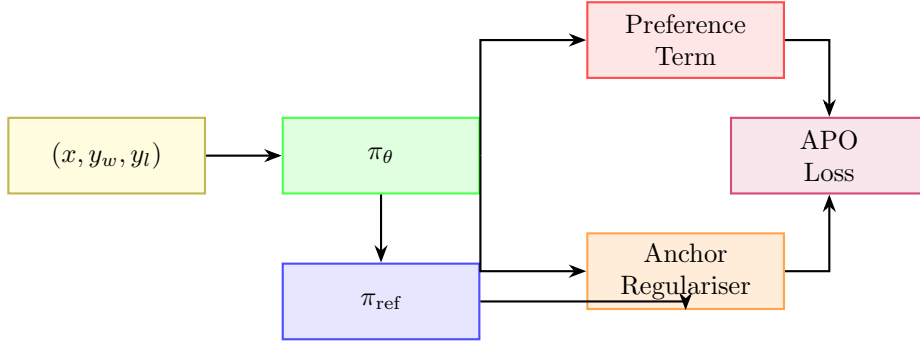


Figure 27: APO: combines standard preference loss with an anchor regulariser against the reference model.

State Transition Table

Property	Before (DPO)	After (APO)
Anchor regularisation	Implicit KL	Explicit squared anchor
Reference role	Log-ratio denominator	Explicit target
Policy drift	Can drift	Penalised
Loss terms	1	2 (preference + anchor)
Stability	Moderate	Higher (dual grounding)

Table 27: State properties comparing DPO and APO.

Rank Responses from Human Feedback (RRHF)

Mathematical Foundation

RRHF generates K candidate responses $\{y^{(1)}, \dots, y^{(K)}\}$ from the model, ranks them by human or AI annotators, and trains the model to assign higher log-probability to higher-ranked responses via a pairwise ranking loss:

$$\mathcal{L}_{\text{RRHF}} = \sum_{i < j} \max(0, \text{score}(y^{(j)}) - \text{score}(y^{(i)}) + \delta) + \lambda \mathcal{L}_{\text{SFT}}(y^{(1)}) \quad (101)$$

where $\text{score}(y^{(k)}) = \log P_{\theta}(y^{(k)}|x)/|y^{(k)}|$ is the normalised log-prob. The SFT term anchors the top-ranked response.

Architecture Flow Diagram

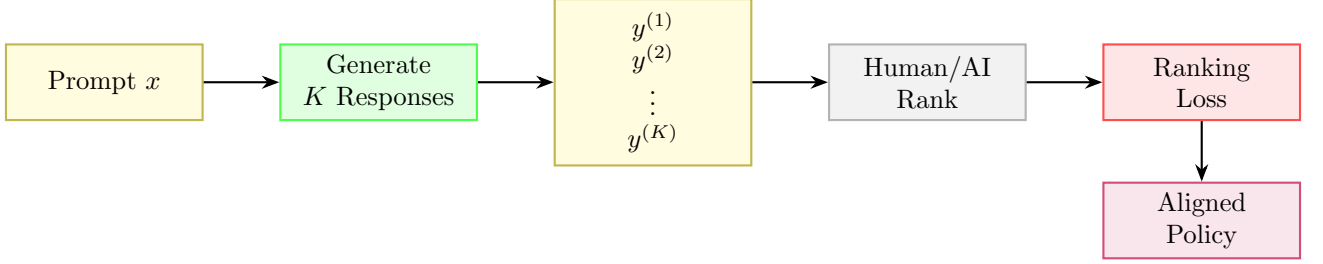


Figure 28: RRHF: K responses are generated and ranked; policy trained to match rank order via pairwise loss.

State Transition Table

Property	Before	After
Responses per prompt	1	K (ranked)
Supervision signal	Binary (win/lose)	Full rank ordering
Loss type	Log-sigmoid	Pairwise margin
Top-response training	Separate SFT	Joint SFT + ranking
Annotation granularity	Single comparison	Full K -way ranking

Table 28: State properties before and after RRHF.

Reinforcement Learning from AI Feedback (RLAIF)

Mathematical Foundation

RLAIF replaces expensive human annotators with a capable AI model (“AI annotator” $\mathcal{A}$) to generate preference labels at scale. The AI annotator scores pairs via a prompt p :

$$\hat{r}(x, y_i, y_j) = P_{\mathcal{A}}(y_i \succ y_j \mid p, x, y_i, y_j) \quad (102)$$

These AI-generated preferences are then used identically to human preferences to train the RM and run PPO (or DPO):

$$\mathcal{L}_{\text{RLAIF}} = -\mathbb{E}[\log \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))], \quad (y_w \succ y_l) \text{ from } \mathcal{A} \quad (103)$$

Architecture Flow Diagram

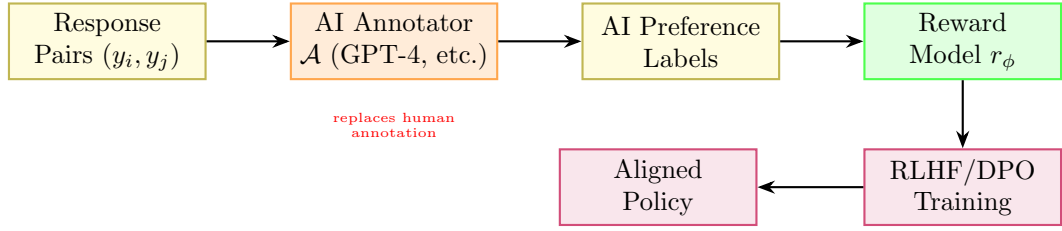


Figure 29: RLAIF: an AI model generates preference labels at scale, replacing costly human annotators.

State Transition Table

Property	Before (RLHF)	After (RLAIF)
Annotation source	Humans	AI annotator $\mathcal{A}$
Annotation cost	High (\$\$\$)	Low (API calls)
Annotation scale	Limited (thousands)	Massive (millions)
Annotation consistency	Variable	Consistent per model
Annotation bias	Human bias	AI model bias

Table 29: State properties comparing RLHF and RLAIF.

Constitutional AI

Mathematical Foundation

Constitutional AI uses a set of principles $\mathcal{C} = \{c_1, \dots, c_M\}$ (the “constitution”) to iteratively critique and revise model outputs before training. In the RL phase, an AI Feedback model scores responses against the constitution:

$$r_{\text{CAI}}(x, y) = \frac{1}{M} \sum_{m=1}^M P_{\mathcal{A}}(\text{harmless} \mid c_m, x, y) \quad (104)$$

Two training phases: (1) supervised revision via critique-revise loop, (2) RLHF with AI-generated preference labels from the constitution.

Architecture Flow Diagram

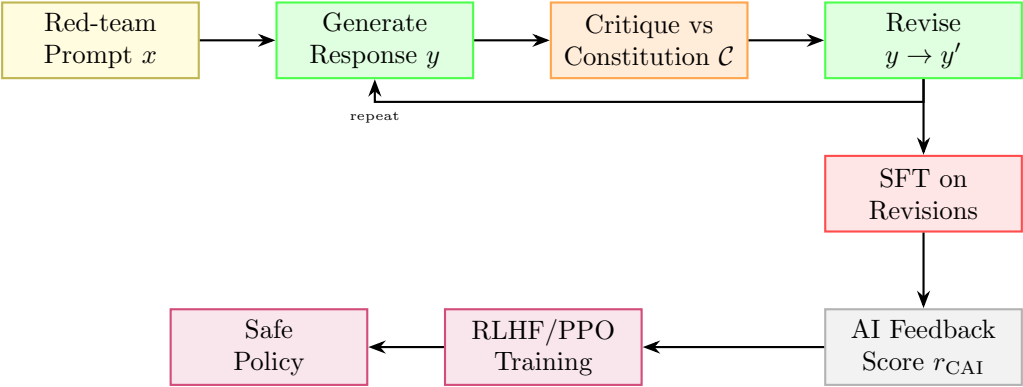


Figure 30: Constitutional AI: critique-revise loop creates SFT data; AI feedback from constitution guides RLHF.

State Transition Table

Property	Before	After
Safety signal	None / Human labels	AI + constitutionl principles
Training phases	SFT only	Critique-revise SFT + RLAIIF
Red-team coverage	Manual	Systematic via constitution
Harmlessness	Moderate	Substantially improved
Helpfulness trade-off	N/A	Explicitly balanced

Table 30: State properties before and after Constitutional AI training.

Part V

Reinforcement Learning Training Methods

Proximal Policy Optimization (PPO)

Mathematical Foundation

PPO is the core RL algorithm used in RLHF. It clips the probability ratio to prevent large policy updates:

$$\rho_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}, \quad \hat{A}_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (105)$$

$$\mathcal{L}_{\text{PPO}} = -\mathbb{E}_t[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)] \quad (106)$$

A value network V_ϕ is trained simultaneously. For LLMs, tokens are actions and the RM score at end-of-sequence provides the sparse reward.

Architecture Flow Diagram

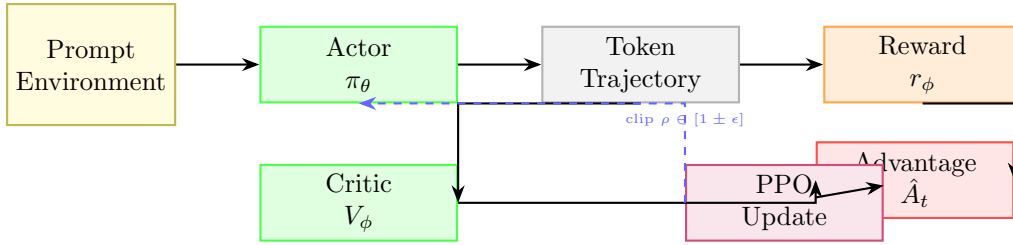


Figure 31: PPO: actor generates trajectory, critic estimates value, advantage drives clipped surrogate update.

State Transition Table

Property	Before	After
Policy update size	Unconstrained	Clipped by ϵ
Value network	None	Trained V_ϕ
Advantage estimation	N/A	GAE $\hat{A}_t$
Training stability	Unstable (vanilla PG)	Stable (clip constraint)
Reward signal	Sparse (end of seq)	Discounted returns

Table 31: State properties before and after PPO setup.

Group Relative Policy Optimization (GRPO)

Mathematical Foundation

GRPO removes the critic (value network) by using group-level reward statistics. For a prompt x , it samples G outputs $\{y^{(1)}, \dots, y^{(G)}\}$ and computes relative advantages within the group:

$$\hat{A}^{(i)} = \frac{r^{(i)} - \text{mean}(r^{(1:G)})}{\text{std}(r^{(1:G)})} \quad (107)$$

$$\mathcal{L}_{\text{GRPO}} = -\frac{1}{G} \sum_{i=1}^G \min(\rho^{(i)} \hat{A}^{(i)}, \text{clip}(\rho^{(i)}, 1 \pm \epsilon) \hat{A}^{(i)}) + \beta \text{KL}[\pi_{\theta} \parallel \pi_{\text{ref}}] \quad (108)$$

Architecture Flow Diagram

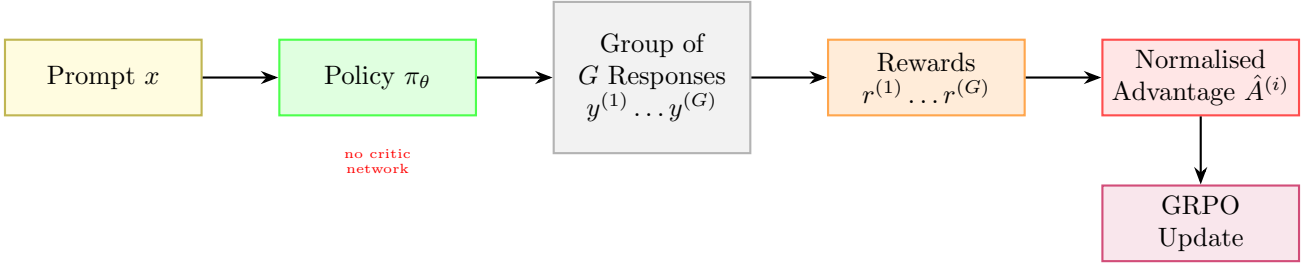


Figure 32: GRPO: group of responses scored together; within-group normalised reward replaces value network.

State Transition Table

Property	Before (PPO)	After (GRPO)
Critic / value network	Required	Not needed
Advantage estimation	GAE (per token)	Group normalisation
Memory overhead	2× model (actor + critic)	≈ 1× model
Samples per prompt	1	G (group sampling)
Variance reduction	Value baseline	Mean group reward

Table 32: State properties comparing PPO and GRPO.

Rejection Fine-Tuning (RFT)

Mathematical Foundation

RFT generates K candidate solutions per problem using the current policy, filters by a correctness verifier $\mathcal{V}$, and fine-tunes on the correct subset essentially a self-improvement loop without RL:

$$\mathcal{D}_{\text{correct}} = \{(x, y^{(k)}) : \mathcal{V}(x, y^{(k)}) = 1, y^{(k)} \sim \pi_{\theta}, k = 1..K\} \quad (109)$$

$$\mathcal{L}_{\text{RFT}} = - \sum_{(x,y) \in \mathcal{D}_{\text{correct}}} \sum_t \log P_{\theta}(y_t | x, y_{<t}) \quad (110)$$

The process repeats: train on correct outputs, generate new candidates, filter again.

Architecture Flow Diagram

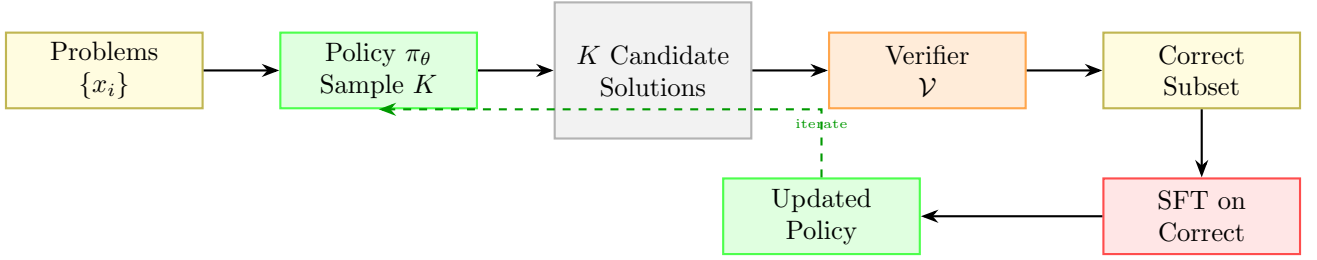


Figure 33: RFT: policy samples K solutions, verifier filters correct ones, SFT improves the policy iteratively.

State Transition Table

Property	Before	After
Data source	Static human labels	Self-generated correct solutions
RL component	Required (PPO)	None (SFT only)
Verifier	None	Required (exact-match, unit tests)
Pass@K improvement	Baseline	Improves with iterations
Training complexity	High (RL)	Standard SFT loop

Table 33: State properties before and after RFT.

REINFORCE

Mathematical Foundation

REINFORCE is the foundational policy gradient algorithm. For a trajectory $\tau = (s_0, a_0, s_1, a_1, \dots)$, the gradient of expected return is:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [G(\tau) \nabla_{\theta} \log \pi_{\theta}(\tau)], \quad G(\tau) = \sum_{t=0}^T \gamma^t r_t \quad (111)$$

For LLMs, the trajectory is the token sequence, each token is an action, and r_T (end reward) is discounted back. A baseline b (e.g., mean reward) reduces variance:

$$\nabla_{\theta} J \approx \frac{1}{N} \sum_n (G_n - b) \nabla_{\theta} \log \pi_{\theta}(y_n | x_n) \quad (112)$$

Architecture Flow Diagram

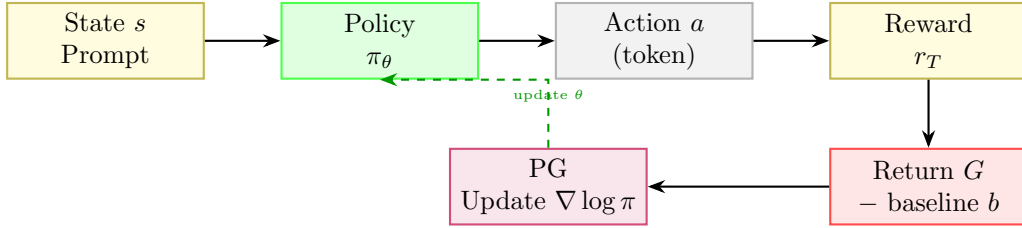


Figure 34: REINFORCE: policy samples action sequence, receives return G , updates via policy gradient.

State Transition Table

Property	Before	After
Gradient type	Supervised CE	Policy gradient $G \nabla \log \pi$
Variance	N/A	High (needs baseline)
Critic network	Not needed	Not needed
Reward type	N/A	Any differentiable or sparse
Update frequency	Per batch	Per episode/trajectory

Table 34: State properties before and after REINFORCE setup.

Advantage Actor-Critic (A2C / A3C)

Mathematical Foundation

A2C uses a shared backbone with two heads: a policy head (actor) and a value head (critic). The advantage $A(s, a) = Q(s, a) - V(s)$ reduces gradient variance:

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}_t[A(s_t, a_t) \log \pi_\theta(a_t|s_t)] \quad (113)$$

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_t[(r_t + \gamma V(s_{t+1}) - V(s_t))^2] \quad (114)$$

A3C extends A2C with asynchronous parallel workers each running independent environments and gradients are asynchronously applied to a central parameter server.

Architecture Flow Diagram

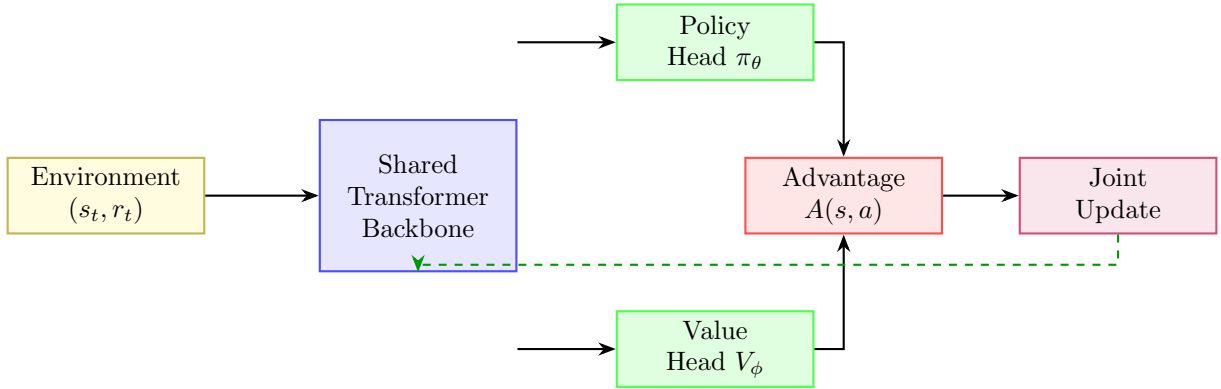


Figure 35: A2C: shared backbone feeds actor (policy) and critic (value) heads; advantage combines their outputs.

State Transition Table

Property	Before (REINFORCE)	After (A2C)
Variance reduction	Baseline only	Full advantage estimation
Value network	Separate	Shared backbone, separate head
Gradient stability	Low	Higher (advantage)
Parallelism	None	A3C: asynchronous workers
Sample efficiency	Low	Higher (on-policy)

Table 35: State properties comparing REINFORCE and A2C.

Self-Play Reinforcement Learning

Mathematical Foundation

In Self-Play RL, the model competes against previous versions of itself. At iteration n , the opponent is a snapshot $\pi_{\theta_{n-1}}$ (or mixture of past policies). The reward is determined by competitive outcome:

$$r(y_\theta, y_{\theta_{\text{old}}}) = \begin{cases} +1 & \text{if } y_\theta \text{ is judged better} \\ -1 & \text{otherwise} \end{cases} \quad (115)$$

$$\mathcal{J} = \mathbb{E}_{\pi_\theta, \pi_{\theta_{\text{old}}}} [r(y_\theta, y_{\theta_{\text{old}}})] \quad (116)$$

This creates an ever-escalating difficulty curriculum since the opponent improves alongside the model.

Architecture Flow Diagram

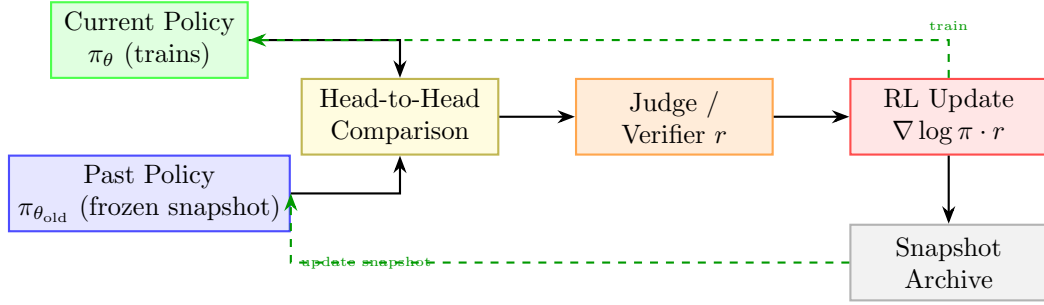


Figure 36: Self-Play RL: current policy competes against a frozen past snapshot; win/loss drives RL updates.

State Transition Table

Property	Before	After
Opponent	None / static	Evolving past self
Difficulty curriculum	Fixed	Auto-scaling
Reward source	External judge	Comparative win/loss
Policy snapshot archive	N/A	Maintained
Capability ceiling	Fixed by data	Self-determined

Table 36: State properties before and after Self-Play RL setup.

Monte Carlo Tree Search RL (MCTS-RL)

Mathematical Foundation

MCTS-RL builds a reasoning tree by alternating between *selection* (UCT formula), *expansion*, *rollout*, and *backpropagation*. The UCT score for node n :

$$\text{UCT}(n) = \frac{W(n)}{N(n)} + c \sqrt{\frac{\ln N(\text{parent}(n))}{N(n)}} \quad (117)$$

where $W(n)$ is total rollout reward and $N(n)$ is visit count. The policy is improved by training on trajectories weighted by MCTS visit counts:

$$\pi_{\theta} \leftarrow \pi_{\theta} + \eta \mathbb{E}[\text{MCTS visits}(a|s) \cdot \nabla_{\theta} \log \pi_{\theta}(a|s)] \quad (118)$$

Architecture Flow Diagram

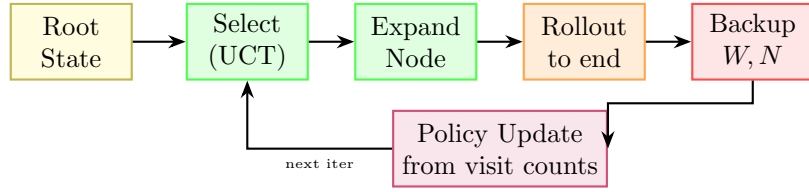


Figure 37: MCTS-RL: select (UCT), expand, rollout, backpropagate; visit counts guide policy improvement.

State Transition Table

Property	Before	After
Search strategy	Greedy / beam	Tree search (UCT)
State value estimate	None	$W(n)/N(n)$ from rollouts
Planning horizon	Local	Multi-step look-ahead
Compute per query	Inference only	Inference + tree expansion
Policy quality	Single-pass	Iteratively refined

Table 37: State properties before and after MCTS-RL integration.

Search-Augmented RL (Search-RL)

Mathematical Foundation

Search-RL augments the reasoning process with an explicit search over candidate next steps. At each step t , the policy generates B candidates $\{s_t^{(1)}, \dots, s_t^{(B)}\}$, a verifier scores them, and the best is selected:

$$s_t^* = \arg \max_b V(x, s_{1:t-1}, s_t^{(b)}) \quad (119)$$

Training uses the full selected trajectory as RL experience:

$$\mathcal{L}_{\text{SearchRL}} = - \sum_t \mathbb{I}[\mathcal{V}(x, y) = 1] \log \pi_\theta(s_t^* \mid x, s_{1:t-1}) \quad (120)$$

Architecture Flow Diagram

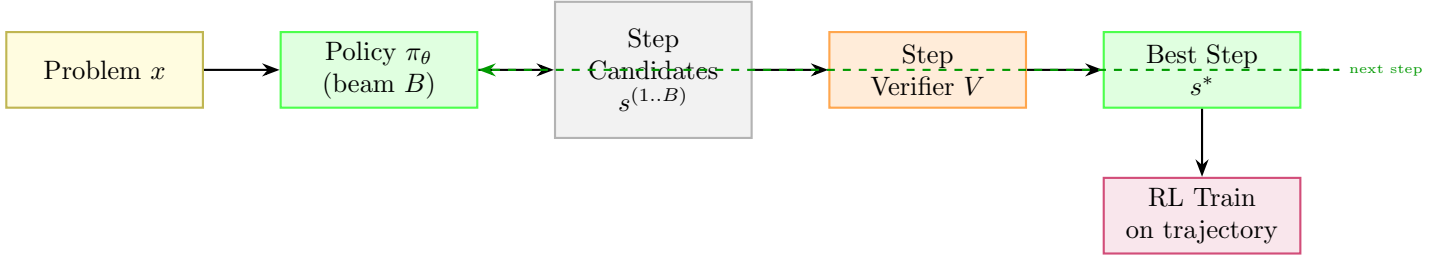


Figure 38: Search-RL: at each step, beam search generates candidates; verifier selects best; policy trained on trajectory.

State Transition Table

Property	Before	After
Step selection	Greedy / top-1	Verifier-guided beam search
Search breadth	1	B candidates per step
Verifier	None	Step-level verifier V
Training signal	Outcome reward	Step-selected trajectory
Error recovery	None	Search can backtrack

Table 38: State properties before and after Search-RL.

Tree-of-Thought Training

Mathematical Foundation

Tree-of-Thought (ToT) Training teaches the model to generate, evaluate, and prune multiple reasoning branches. The model is trained on annotated trees where each branch b_i has a quality score $q_i \in [0, 1]$. The training objective rewards correct branches and penalises dead-ends:

$$\mathcal{L}_{\text{ToT}} = - \sum_i q_i \sum_t \log P_{\theta}(b_{i,t} \mid x, b_{i,<t}) + (1 - q_i) \sum_t \log(1 - P_{\theta}(b_{i,t} \mid x, b_{i,<t})) \quad (121)$$

A separate scoring model $s_{\phi}(x, b_i)$ is trained to evaluate intermediate thought nodes and guide beam search at inference.

Architecture Flow Diagram

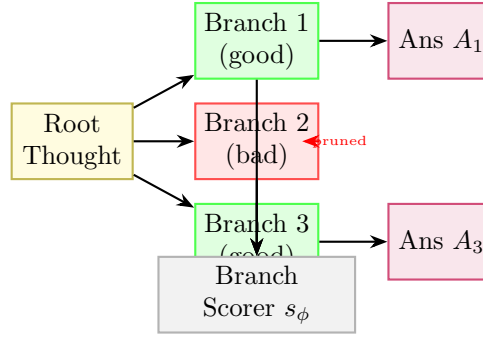


Figure 39: ToT Training: good branches lead to answers; bad branches are pruned; scorer trained to evaluate all.

State Transition Table

Property	Before	After
Reasoning structure	Linear chain	Branching tree
Dead-end handling	None	Explicit pruning
Branch evaluator	None	Trained s_{ϕ}
Search at inference	Linear	Guided tree beam
Complex planning	Weak	Strong

Table 39: State properties before and after Tree-of-Thought Training.

Self-Consistency Training

Mathematical Foundation

Self-Consistency Training generates K independent reasoning paths and uses majority voting to determine the correct answer. The model is then trained to increase probability of the majority-vote answer y^* :

$$y^* = \arg \max_y \sum_{k=1}^K \mathbf{1}[\text{answer}(y^{(k)}) = y], \quad y^{(k)} \sim \pi_\theta(\cdot|x) \tag{122}$$

$$\mathcal{L}_{\text{SC}} = - \sum_{\{k: \text{answer}(y^{(k)})=y^*\}} \sum_t \log P_\theta(y_t^{(k)} | x, y_{<t}^{(k)}) \tag{123}$$

Paths that agree with the majority vote are reinforced; minority paths are not trained on (or penalised).

Architecture Flow Diagram

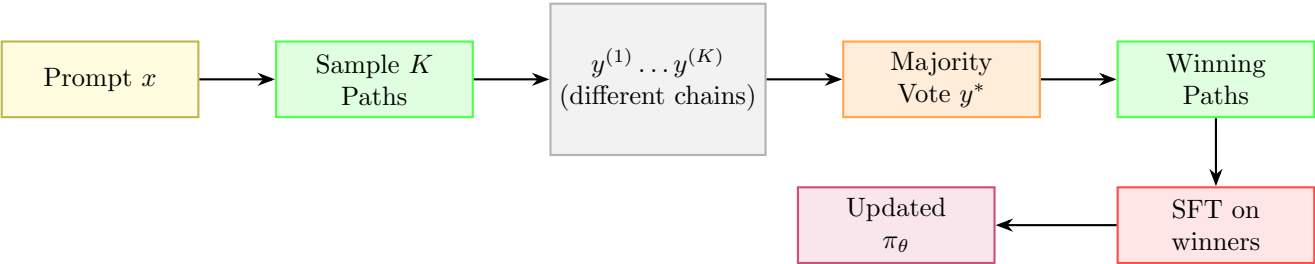


Figure 40: Self-Consistency Training: majority-vote over K paths identifies winning chains; model trained on those.

State Transition Table

Property	Before	After
Inference mode	Single greedy	K diverse samples
Answer selection	Argmax	Majority vote
Training target	All completions	Majority-consistent paths only
Accuracy ceiling	Limited by single pass	Higher (ensemble effect)
Compute cost	$1\times$	$K\times$ at sampling time

Table 40: State properties before and after Self-Consistency Training.

Part VI

Reward Modeling

Reward Model (RM)

Mathematical Foundation

A Reward Model takes a prompt x and a completion y and outputs a scalar preference score. It is trained on human-ranked pairs $(y_w \succ y_l)$ via the Bradley-Terry log-likelihood:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x, y_w, y_l)} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))] \quad (124)$$

The RM is typically initialised from a fine-tuned LLM with the final token's hidden state projected to a scalar via a linear head $W_r \in \mathbb{R}^{d \times 1}$:

$$r_\phi(x, y) = W_r^\top h_{\text{last}}(x, y) \quad (125)$$

Architecture Flow Diagram

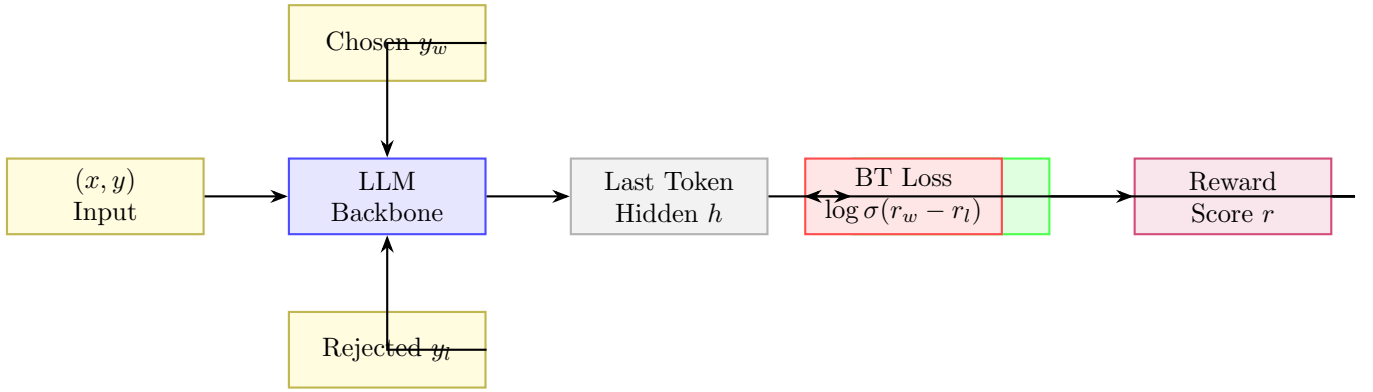


Figure 41: Reward Model: LLM backbone maps (x, y) pair to scalar via linear head; trained on chosen/rejected pairs.

State Transition Table

Property	Before	After
Output type	Token distribution	Scalar reward
Final layer	LM head	Scalar head W_r
Training data	Text completions	(x, y_w, y_l) preference pairs
Loss function	Cross-entropy	Bradley-Terry log-likelihood
Usage	Language generation	RLHF reward signal

Table 41: State properties before and after reward model training.

Process Reward Model (PRM)

Mathematical Foundation

A PRM assigns a reward to each intermediate reasoning step s_t rather than to the final answer alone. Given a solution prefix $(s_1, \dots, s_t)$, the PRM outputs a binary step quality $q_t \in \{0, 1\}$:

$$r_{\text{PRM}}(x, s_{1:T}) = \prod_{t=1}^T r_t, \quad r_t = P_\phi(q_t = 1 \mid x, s_{1:t}) \quad (126)$$

The PRM is trained on step-level human labels with binary cross-entropy:

$$\mathcal{L}_{\text{PRM}} = - \sum_t [q_t \log r_t + (1 - q_t) \log(1 - r_t)] \quad (127)$$

Architecture Flow Diagram

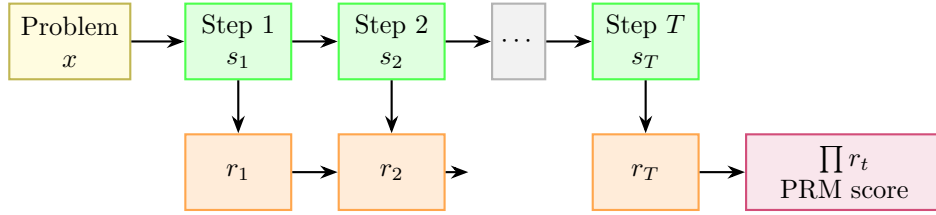


Figure 42: PRM: each reasoning step independently scored; product of step scores gives overall process reward.

State Transition Table

Property	Before (RM)	After (PRM)
Reward granularity	Final answer only	Per-step
Error localisation	None	Identifies failing step
Label annotation	Answer-level	Step-level (expensive)
Reward signal density	Sparse (1 per traj.)	Dense (T per traj.)
Training on correct solutions	Required	Not required

Table 42: State properties comparing RM and PRM.

Outcome Reward Model (ORM)

Mathematical Foundation

An ORM judges only the *correctness of the final answer* y_{ans} , ignoring intermediate steps. For verifiable tasks (math, code), correctness can be checked automatically:

$$r_{\text{ORM}}(x, y) = \mathbf{1}[\text{verify}(y_{\text{ans}}, y_{\text{gold}}^*)] \quad (128)$$

For open-ended tasks, the ORM is a learned binary classifier trained on (correct, incorrect) outcome labels:

$$\mathcal{L}_{\text{ORM}} = -[c \log P_{\phi}(y) + (1 - c) \log(1 - P_{\phi}(y))], \quad c \in \{0, 1\} \quad (129)$$

Architecture Flow Diagram

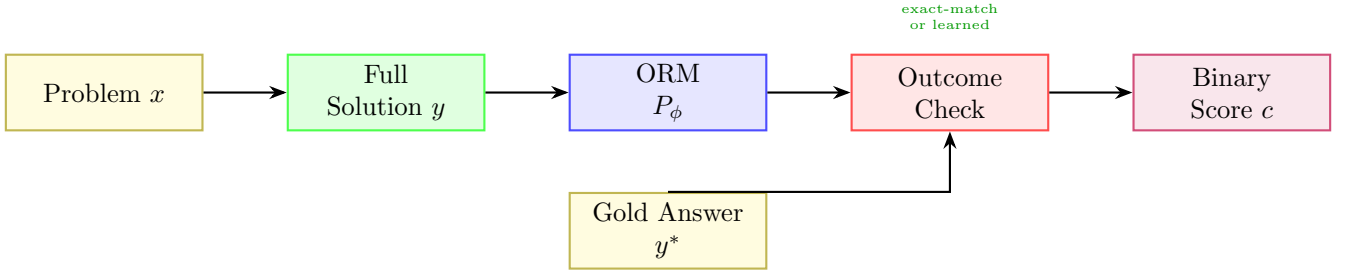


Figure 43: ORM: scores the final answer against gold; can be rule-based (exact-match) or learned classifier.

State Transition Table

Property	Before	After
Reward scope	N/A	Final answer only
Intermediate credit	None	None
Annotation cost	N/A	Low (binary label)
Verifiable tasks	Manual check	Automatic (rule-based)
Open-ended tasks	None	Learned classifier

Table 43: State properties before and after ORM setup.

Verifier Model

Mathematical Foundation

A Verifier Model is a binary classifier that determines whether a candidate solution is correct. Unlike an ORM that outputs a probability, verifiers typically use a hard decision rule learned from (correct, incorrect) solution pairs:

$$\hat{v}(x, y) = \mathbf{1}[P_\phi(\text{correct} \mid x, y) \geq \tau] \quad (130)$$

Verifiers are used in test-time search (Best-of-N) and training-time filtering (RFT). The decision threshold τ trades off precision and recall on the verification task:

$$\mathcal{L}_{\text{ver}} = -[v \log P_\phi + (1 - v) \log(1 - P_\phi)], \quad v \in \{0, 1\} \quad (131)$$

Architecture Flow Diagram

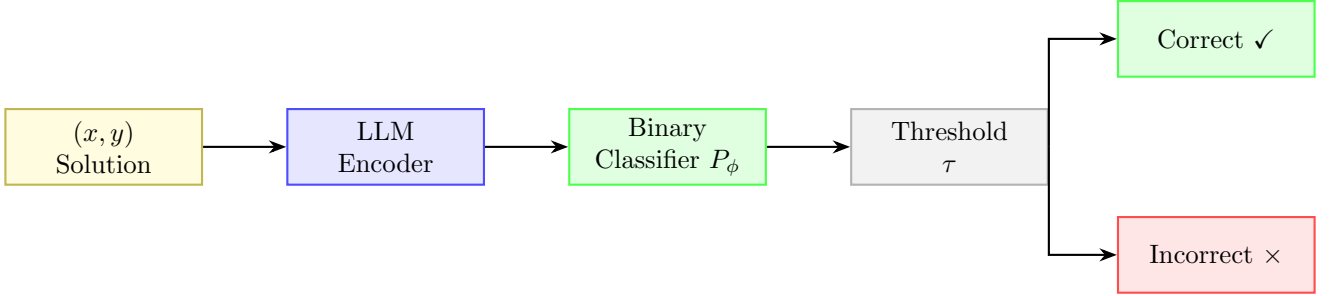


Figure 44: Verifier Model: encodes solution, classifies as correct/incorrect via learned threshold.

State Transition Table

Property	Before	After
Output	Continuous score	Binary decision
Threshold	N/A	Tunable τ
Primary use	Ranking	Filtering (accept/reject)
Training data	Ranked pairs	Correct/incorrect labels
Precision/recall trade-off	N/A	Controlled via τ

Table 44: State properties before and after Verifier Model training.

Critic Model

Mathematical Foundation

A Critic Model generates natural-language *critiques* of candidate outputs and optionally a numerical score. It is trained on (output, critique, revision) triplets. The critique generation loss:

$$\mathcal{L}_{\text{crit}} = - \sum_t \log P_{\phi}(c_t \mid x, y, c_{<t}) \tag{132}$$

where c is the gold critique. Critics enable self-refinement loops: the model generates y , the critic evaluates it producing c , and the model revises $y \rightarrow y'$ conditioned on (x, y, c) :

$$y' \sim P_{\theta}(\cdot \mid x, y, c) \tag{133}$$

Architecture Flow Diagram

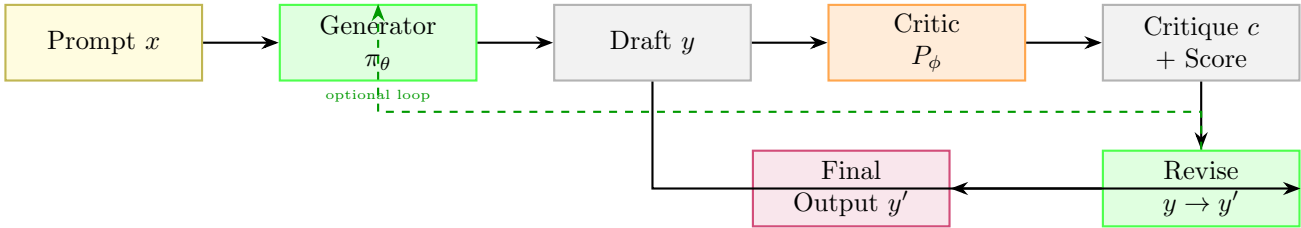


Figure 45: Critic Model: generates natural-language critique of a draft; generator revises using the critique.

State Transition Table

Property	Before	After
Feedback type	Scalar reward	Natural-language critique
Interpretability	Low (scalar)	High (textual feedback)
Revision loop	Not applicable	Generator refines on critique
Training data	Preference pairs	(output, critique, revision) triplets
Use case	RM scoring	Self-refinement / RLHF

Table 45: State properties before and after Critic Model training.

Mathematical Foundations, Architecture Diagrams, and State Analysis
of Modern LLM Training Methods — Part 3:
Data Generation, Curation, Distillation, Compression,
Agentic, Multimodal, Knowledge Injection, and Deployment Chief Strategist J June 12, 2026

Contents

Part VII

Synthetic Data Generation

Self-Instruct

Mathematical Foundation

Self-Instruct bootstraps an instruction dataset from a small seed pool $\mathcal{S}$ by using the LLM itself as a generator. At each iteration, k seed instructions are sampled and the model generates a new instruction $\hat{x}$, its input, and output:

$$(\hat{x}, \hat{i}, \hat{y}) \sim P_{\theta}(\cdot \mid \text{few-shot}(\mathcal{S})) \quad (134)$$

Generated examples pass a quality filter (ROUGE similarity $< \delta$ to existing pool, length check):

$$\mathcal{S} \leftarrow \mathcal{S} \cup \{(\hat{x}, \hat{i}, \hat{y})\} \text{ iff } \max_{s \in \mathcal{S}} \text{ROUGE}(\hat{x}, s) < \delta \quad (135)$$

The growing pool is used to fine-tune θ , improving the quality of subsequent generations.

Architecture Flow Diagram

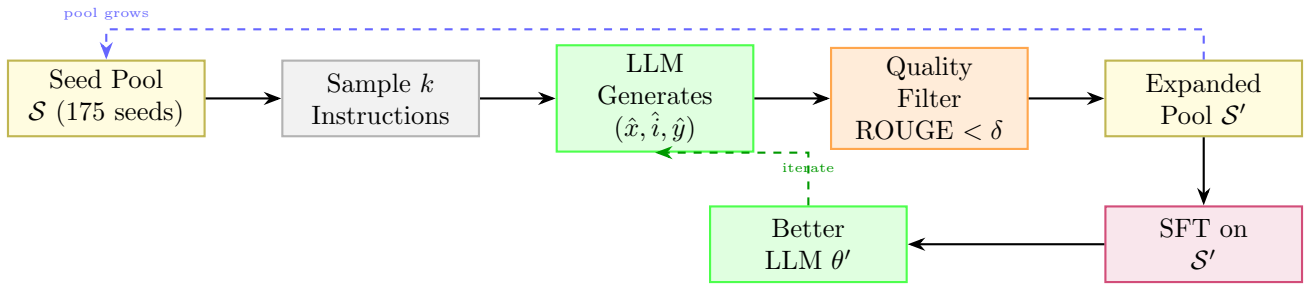


Figure 46: Self-Instruct: LLM generates new instructions from seed pool; quality filter expands the pool for SFT.

State Transition Table

Property	Before	After
Pool size	175 seed instructions	52,000+ generated
Annotation source	Human-written seeds	LLM-generated
Diversity filter	N/A	ROUGE deduplication
Training data cost	Low (seeds only)	Essentially free to scale
Instruction following	Weak	Strong

Table 46: State properties before and after Self-Instruct data generation.

Evol-Instruct

Mathematical Foundation

Evol-Instruct evolves existing instructions into harder variants via two mutation strategies. *In-depth* evolution adds constraints, increases steps, or requires deeper reasoning. *In-breadth* evolution creates new topic-adjacent instructions:

$$\hat{x} = M_{\text{depth}}(x) \text{ or } \hat{x} = M_{\text{breadth}}(x), \quad \hat{x} \sim P_{\text{teacher}}(\cdot \mid \text{evolve-prompt}, x) \quad (136)$$

Evolved instructions are validated (non-empty, no refusal, dissimilar from parent) and collected into $\mathcal{D}_{\text{evol}}$. Difficulty increases monotonically with evolution rounds E :

$$\mathbb{E}[\text{difficulty}(\hat{x}_E)] > \mathbb{E}[\text{difficulty}(\hat{x}_{E-1})] \quad (137)$$

Architecture Flow Diagram

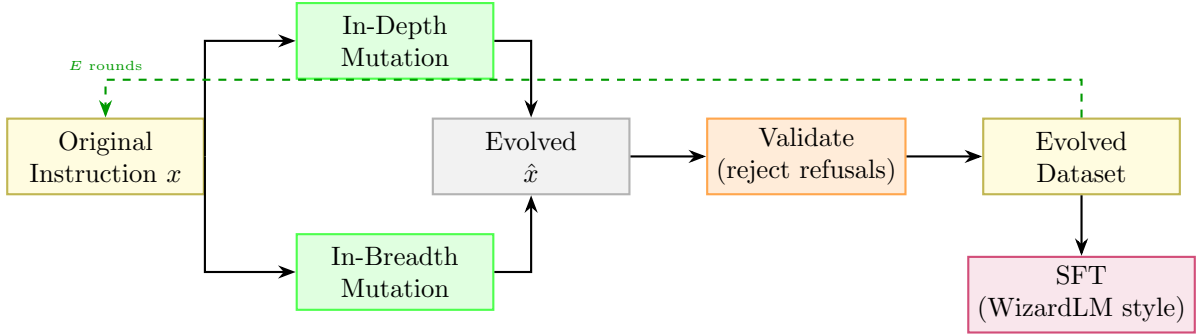


Figure 47: Evol-Instruct: in-depth and in-breadth mutations generate harder variants; validated pool used for SFT.

State Transition Table

Property	Before	After
Instruction difficulty	Low (seed)	Progressively higher (per round E)
Instruction diversity	Limited	Broad via breadth mutations
Data source	Human-written	LLM-evolved
Evolution rounds	0	E (typically 4–8)
Model performance	Baseline	WizardLM-style improvements

Table 47: State properties before and after Evol-Instruct.

Evol-Code

Mathematical Foundation

Evol-Code applies the same evolutionary principle to programming tasks, using code-specific mutation operators: adding input/output constraints, combining two functions, increasing time/space complexity, or switching programming language:

$$\hat{p} = M_{\text{code}}(p), \quad M \in \{\text{constrain, combine, optimize, translate}\} \quad (138)$$

Each evolved problem $\hat{p}$ is solved by a teacher model, and the (problem, solution) pair is validated by unit-test execution:

$$(\hat{p}, \hat{s}) \in \mathcal{D}_{\text{code}} \iff \text{unit_tests}(\hat{s}, \hat{p}) = \text{PASS} \quad (139)$$

Architecture Flow Diagram

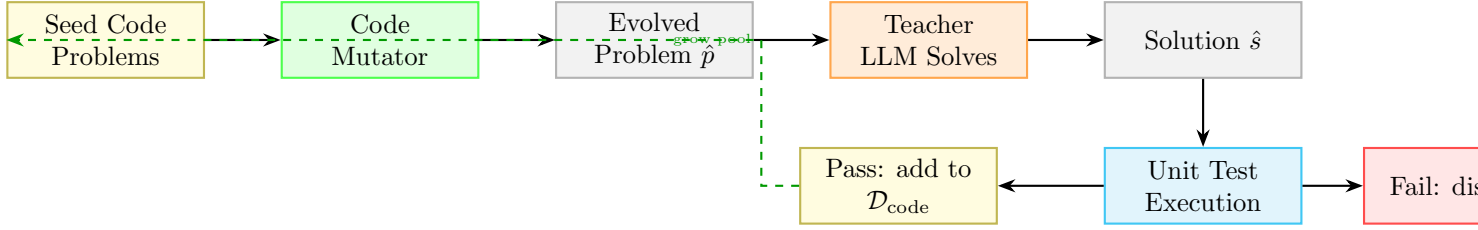


Figure 48: Evol-Code: code problems are mutated, solved by teacher, validated by unit tests before dataset inclusion.

State Transition Table

Property	Before	After
Problem complexity	Simple seed tasks	Multi-constraint, optimised
Validation method	Human review	Automated unit tests
Data quality gate	None	Unit-test pass rate
Programming languages	Fixed	Multi-language via translate mutation
Code reasoning ability	Baseline	Significantly improved

Table 48: State properties before and after Evol-Code data generation.

Synthetic Data Generation

Mathematical Foundation

A powerful teacher model P_T generates synthetic (input, output) pairs from topic/schema prompts, which are then used to train a smaller student P_S . The synthetic data distribution approximates the real task distribution P_{real} :

$$\mathcal{D}_{\text{synth}} = \{(x_i, y_i) : x_i \sim P_{\text{topic}}, y_i \sim P_T(\cdot | x_i)\} \quad (140)$$

Quality is improved by filtering on a scoring model or perplexity threshold τ_p :

$$(x, y) \in \mathcal{D}_{\text{final}} \iff \text{score}(x, y) \geq \tau_s \text{ and } \text{PPL}(y) \leq \tau_p \quad (141)$$

Architecture Flow Diagram

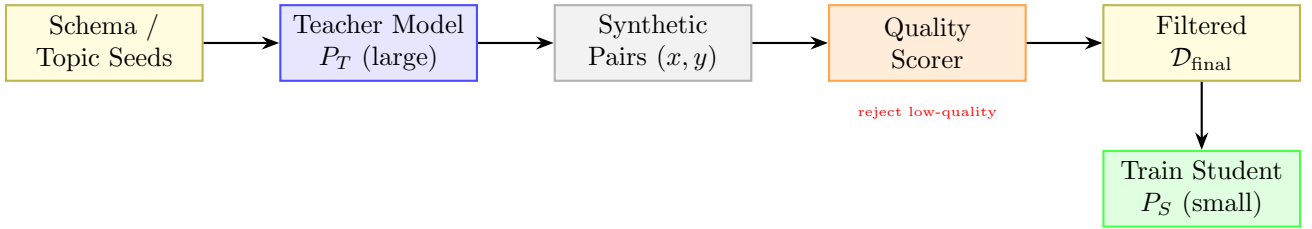


Figure 49: Synthetic Data Generation: teacher generates pairs from schema seeds; quality filter builds training set for student.

State Transition Table

Property	Before	After
Data source	Human annotation	LLM-generated at scale
Annotation cost	High	Very low (API cost)
Data volume	Thousands	Millions scalable
Quality control	Human review	Automated scoring
Domain coverage	Limited	Any topic via schema

Table 49: State properties before and after synthetic data generation.

Rejection Sampling

Mathematical Foundation

Rejection Sampling generates N candidate completions from the current policy and keeps only those that pass a quality threshold, building a filtered fine-tuning corpus:

$$\mathcal{D}_{\text{RS}} = \{(x, y^{(k)}) : y^{(k)} \sim \pi_{\theta}(x), r(x, y^{(k)}) \geq \tau, k = 1..N\} \quad (142)$$

Expected acceptance rate at threshold τ : $\alpha = P_{y \sim \pi_{\theta}}[r(x, y) \geq \tau]$. After SFT on $\mathcal{D}_{\text{RS}}$, the model's probability mass shifts toward high-reward completions:

$$\mathbb{E}_{y \sim \pi_{\theta'}}[r(x, y)] > \mathbb{E}_{y \sim \pi_{\theta}}[r(x, y)] \quad (143)$$

Architecture Flow Diagram

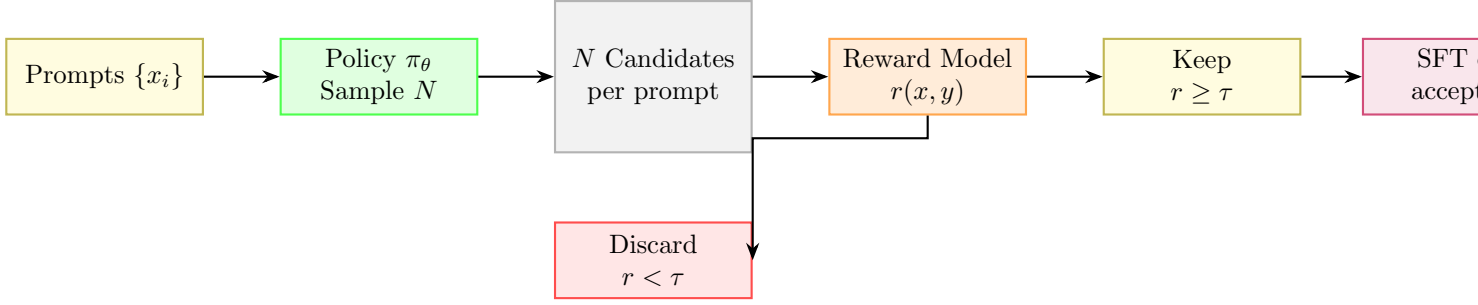


Figure 50: Rejection Sampling: N completions generated; only those above reward threshold kept for SFT.

State Transition Table

Property	Before	After
Training data quality	Mixed	High (above threshold τ)
Samples per prompt	1	N (with αN accepted)
Reward model	Optional	Required as filter
Iteration benefit	None	Compound with model updates
RL complexity	None	None (SFT only)

Table 50: State properties before and after Rejection Sampling.

Best-of-N Sampling

Mathematical Foundation

Best-of-N (BoN) sampling selects the highest-reward completion from N independent samples at inference time:

$$y^* = \arg \max_{k \in 1..N} r(x, y^{(k)}), \quad y^{(k)} \sim \pi_\theta(\cdot | x) \quad (144)$$

The expected reward improvement over single-sample baseline scales as:

$$\mathbb{E}[\max_k r(x, y^{(k)})] \approx \mu_r + \sigma_r \cdot \Phi^{-1}\left(\frac{N}{N+1}\right) \quad (145)$$

where μ_r, σ_r are the reward mean and std under π_θ , and Φ^{-1} is the inverse normal CDF. BoN can also be used to build SFT datasets by collecting (x, y^*) pairs.

Architecture Flow Diagram

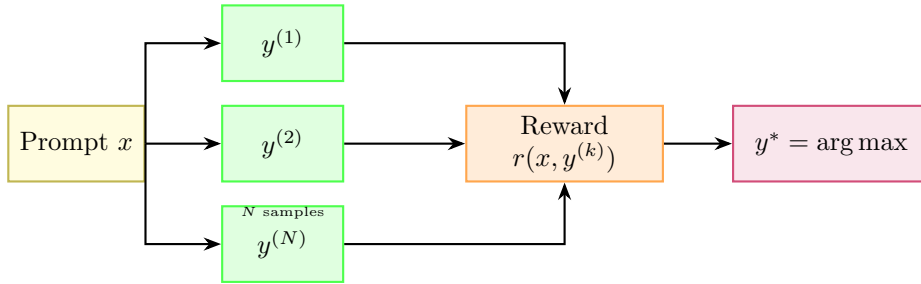


Figure 51: Best-of-N: N samples scored by reward model; highest reward selected as final output.

State Transition Table

Property	Before (single)	After (BoN)
Samples per query	1	N
Compute at inference	$1 \times$	$N \times$
Expected reward	μ_r	$\mu_r + O(\sigma_r \log N)$
Model weights	Unchanged	Unchanged
Best usage	Cheap inference	Quality-critical generation

Table 51: State properties comparing single-sample and Best-of-N sampling.

Part VIII

Data Curation

Data Filtering

Mathematical Foundation

Data filtering removes low-quality examples from a raw corpus using a set of quality signals $\{q_1, \dots, q_M\}$. A sample (x, y) is accepted iff it passes all signal thresholds:

$$(x, y) \in \mathcal{D}_{\text{clean}} \iff \bigwedge_{m=1}^M q_m(x, y) \geq \tau_m \tag{146}$$

Common signals include perplexity under a small LM (detects gibberish), toxic content classifiers, length ratios, and language identification. A learned quality classifier c_ϕ can consolidate multiple signals:

$$c_\phi(x, y) = \sigma(w^\top [q_1(x, y), \dots, q_M(x, y)]) \tag{147}$$

Architecture Flow Diagram

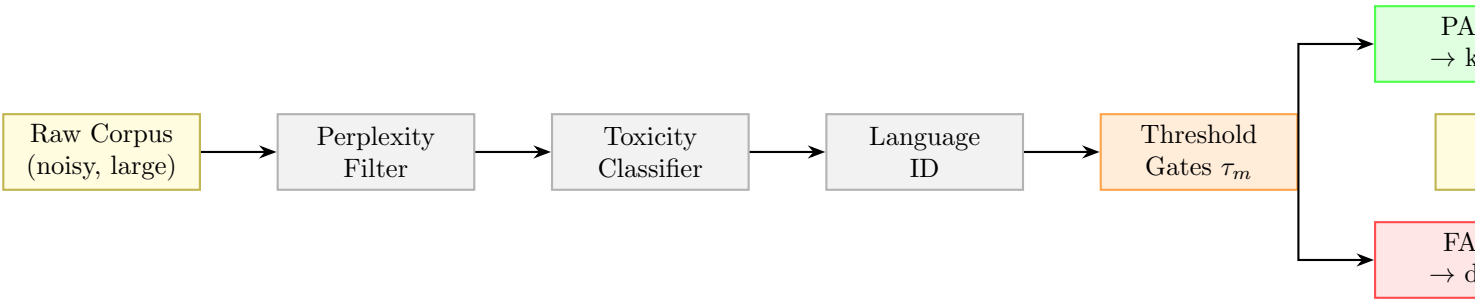


Figure 52: Data filtering pipeline: multiple quality signals applied in sequence; only passing samples retained.

State Transition Table

Property	Before	After
Corpus size	Very large (noisy)	Smaller (high quality)
Noise level	High	Low
Domain coverage	Broad (noisy)	Focused (clean)
Filter signals	None	M quality signals
Training efficiency	Low	Higher (less wasted compute)

Table 52: State properties before and after data filtering.

Deduplication

Mathematical Foundation

Deduplication removes near-duplicate documents to prevent memorisation and improve sample diversity. MinHash LSH estimates Jaccard similarity between documents:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \approx \frac{1}{K} \sum_{k=1}^K \mathbf{1}[h_k(A) = h_k(B)] \quad (148)$$

where $\{h_k\}$ are K independent hash functions and $h_k(A) = \min_{w \in \text{ngrams}(A)} h_k(w)$. Documents with $J(A, B) > \tau_J$ are considered duplicates and one is removed:

$$\mathcal{D}_{\text{dedup}} = \{d_i : \forall j < i, J(d_i, d_j) \leq \tau_J\} \quad (149)$$

Architecture Flow Diagram

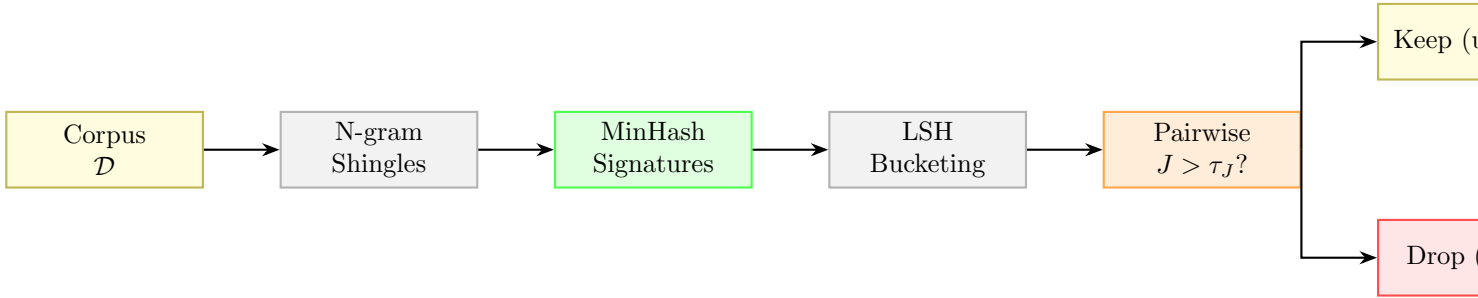


Figure 53: Deduplication: MinHash signatures group near-duplicates via LSH; high-similarity pairs discarded.

State Transition Table

Property	Before	After
Near-duplicate rate	15–30% (web data)	Near zero
Memorisation risk	High	Reduced
Effective token diversity	Low	High
Corpus size	$ \mathcal{D} $	$ \mathcal{D}_{\text{dedup}} < \mathcal{D} $
Compute wasted on duplicates	High	Eliminated

Table 53: State properties before and after deduplication.

Active Learning

Mathematical Foundation

Active Learning selects the most *informative* unlabelled examples for annotation, maximising model improvement per annotation dollar. Uncertainty sampling selects x^* with highest predictive entropy:

$$x^* = \arg \max_{x \in \mathcal{U}} \mathbb{H}[P_\theta(y | x)] = \arg \max_x - \sum_y P_\theta(y|x) \log P_\theta(y|x) \quad (150)$$

Query-by-committee uses disagreement between an ensemble $\{\theta_1, \dots, \theta_C\}$:

$$x^* = \arg \max_{x \in \mathcal{U}} \frac{1}{C} \sum_c \mathbb{KL}[P_{\theta_c}(\cdot|x) \| \bar{P}(\cdot|x)] \quad (151)$$

Architecture Flow Diagram

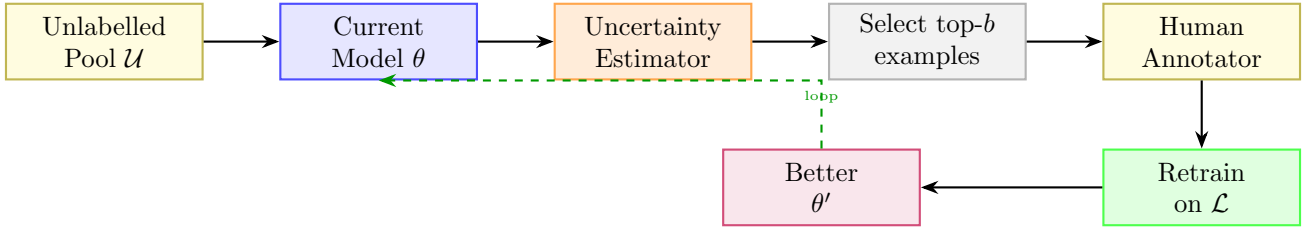


Figure 54: Active Learning: uncertainty estimator selects most informative examples; human annotates; model retrains.

State Transition Table

Property	Before	After
Annotation strategy	Random sampling	Uncertainty-driven
Labels per accuracy gain	Many	Few (targeted)
Unlabelled pool	Fully unused	Iteratively queried
Annotation budget	Fixed	Allocated by informativeness
Model improvement per label	Low	High

Table 54: State properties before and after Active Learning.

Curriculum Learning

Mathematical Foundation

Curriculum Learning trains on examples ordered by difficulty $d(x, y)$, starting with easy examples and progressively introducing harder ones. A competence function $c(t) \in [0, 1]$ controls what fraction of the difficulty range is accessible at step t :

$$c(t) = \min\left(1, c_0 + (1 - c_0) \frac{t}{T_{\text{ramp}}}\right) \quad (152)$$

The training distribution at step t only samples from examples with $d(x, y) \leq c(t) \cdot d_{\text{max}}$:

$$\mathcal{D}_t = \{(x, y) : d(x, y) \leq c(t) \cdot d_{\text{max}}\}, \quad (x, y) \sim \text{Uniform}(\mathcal{D}_t) \quad (153)$$

Architecture Flow Diagram

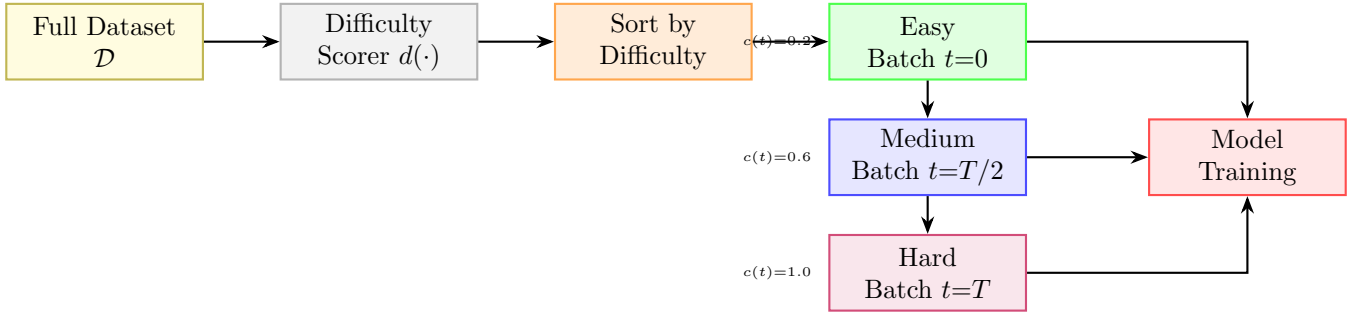


Figure 55: Curriculum Learning: examples sorted by difficulty; competence function progressively unlocks harder batches.

State Transition Table

Property	Before	After
Training order	Random	Difficulty-ordered
Early training batches	Mixed difficulty	Easy only
Convergence speed	Slower	Faster (easier start)
Final performance	Baseline	Improved
Difficulty metric	N/A	Perplexity, length, or task-specific

Table 55: State properties before and after Curriculum Learning.

Part IX

Knowledge Distillation

Knowledge Distillation

Mathematical Foundation

Knowledge Distillation transfers the “dark knowledge” of a large teacher P_T to a small student P_S by minimising a combination of cross-entropy with hard labels and KL divergence from teacher soft logits at temperature τ :

$$\mathcal{L}_{\text{KD}} = \alpha \mathcal{L}_{\text{CE}}(P_S, y_{\text{hard}}) + (1 - \alpha) \tau^2 \mathbb{KL}[P_T^{(\tau)} \| P_S^{(\tau)}] \quad (154)$$

where $P_k^{(\tau)} = \text{softmax}(z_k/\tau)$. Temperature $\tau > 1$ softens distributions to expose similarity structure between classes not visible in hard labels.

Architecture Flow Diagram

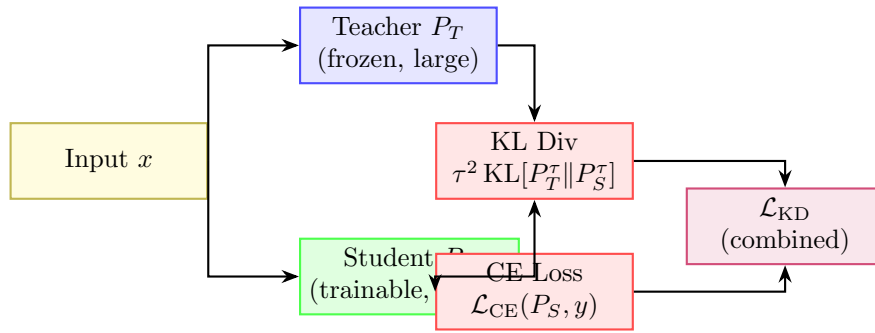


Figure 56: Knowledge Distillation: student learns from both hard labels (CE) and teacher soft logits (KL) simultaneously.

State Transition Table

Property	Before	After
Student accuracy	Low (random)	Near-teacher accuracy
Training signal	Hard labels only	Hard labels + soft teacher logits
Temperature τ	N/A	> 1 (typically 2–5)
Dark knowledge transfer	None	Via inter-class similarities
Student size	Large (teacher)	Small (~ 10 – $100\times$ fewer params)

Table 56: State properties before and after Knowledge Distillation.

Response Distillation

Mathematical Foundation

Response Distillation trains the student by directly imitating the teacher’s *full output sequence* rather than soft logits. The student minimises cross-entropy against teacher-generated responses:

$$\mathcal{L}_{\text{RD}} = - \sum_{(x, y_T) \in \mathcal{D}_T} \sum_t \log P_S(y_{T,t} \mid x, y_{T,<t}) \quad (155)$$

where $y_T \sim P_T(\cdot \mid x)$ are teacher completions. This is simpler than logit matching (no access to teacher logits required) and enables distillation from black-box APIs.

Architecture Flow Diagram

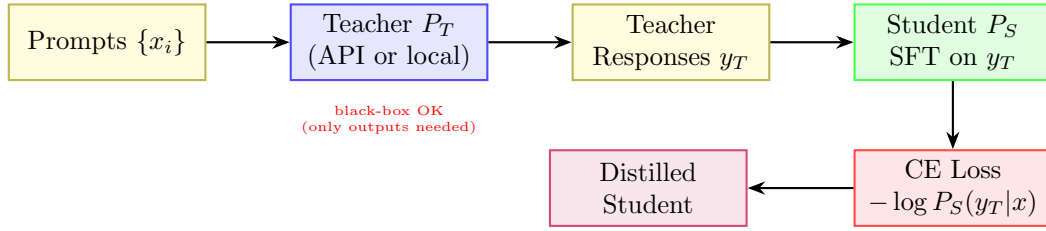


Figure 57: Response Distillation: teacher generates responses; student trained via SFT to imitate those outputs.

State Transition Table

Property	Before (KD)	After (Response KD)
Teacher access required	Logits (white-box)	Outputs only (black-box OK)
Loss type	KL on logits	CE on token sequence
Sequence-level knowledge	Partial	Full (complete generation)
Implementation complexity	High	Same as SFT
Applicable to proprietary APIs	No	Yes

Table 57: State properties comparing logit-level KD and Response Distillation.

Chain-of-Thought Distillation

Mathematical Foundation

CoT Distillation transfers the teacher’s *reasoning traces* to the student, so the student learns not just the answer but how to think step by step. The student is trained on (x, c_T, y_T) triplets where c_T is the teacher-generated chain:

$$\mathcal{L}_{\text{CoTD}} = - \sum_{(x, c_T, y_T)} \left[\sum_m \log P_S(c_{T,m} \mid x, c_{T,<m}) + \sum_t \log P_S(y_{T,t} \mid x, c_T, y_{T,<t}) \right] \quad (156)$$

This allows a small student to match the accuracy of a large teacher that reasons step by step.

Architecture Flow Diagram

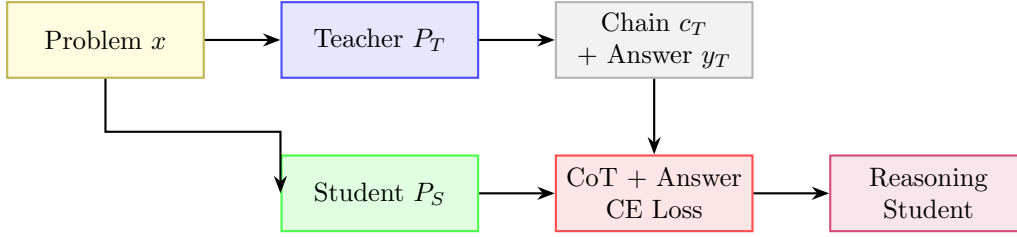


Figure 58: CoT Distillation: teacher’s reasoning chains and answers jointly train student to produce similar traces.

State Transition Table

Property	Before	After
Student reasoning	Direct answer	Step-by-step chain
Training target	Answer only	Chain + answer
Reasoning data source	None	Teacher-generated
Multi-step problem accuracy	Low	Significantly higher
Output verbosity	Terse	Explanatory

Table 58: State properties before and after CoT Distillation.

Policy Distillation

Mathematical Foundation

Policy Distillation minimises the KL divergence between a teacher policy π_T and student policy π_S over the full output distribution at every decoding step, not just the final token:

$$\mathcal{L}_{PD} = \mathbb{E}_{x \sim \mathcal{D}} \sum_t \mathbb{KL}[\pi_T(\cdot | x, y_{<t}) || \pi_S(\cdot | x, y_{<t})] \quad (157)$$

Unlike response distillation (which uses sampled strings), policy distillation uses teacher token-level distributions and requires teacher logit access. This aligns the student's generation *process* with the teacher's, not just its outputs.

Architecture Flow Diagram

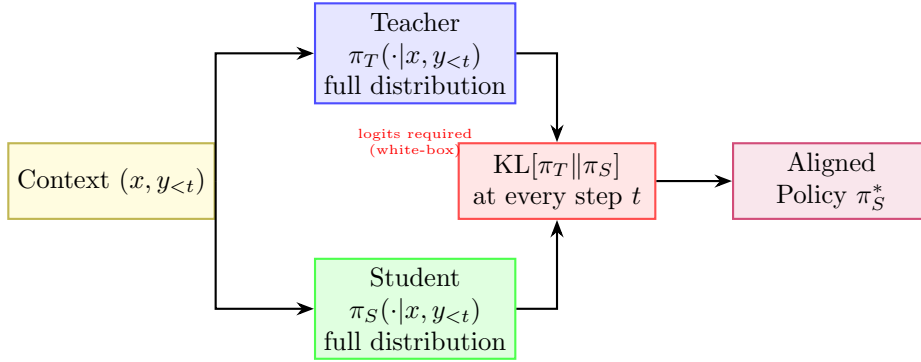


Figure 59: Policy Distillation: KL divergence between teacher and student distributions minimised at every token step.

State Transition Table

Property	Before (Response KD)	After (Policy KD)
Teacher signal	Sampled string	Full token distribution
KL divergence	N/A	Minimised per step
Teacher access	Black-box	White-box (logits)
Process alignment	Output only	Step-by-step
Overconfidence risk	Higher	Lower (distribution match)

Table 59: State properties comparing Response and Policy Distillation.

Reward Distillation

Mathematical Foundation

Reward Distillation compresses a large teacher reward model r_T into a smaller student reward model r_S by regression on teacher-generated scores:

$$\mathcal{L}_{\text{RewardD}} = \mathbb{E}_{(x,y)} [(r_T(x,y) - r_S(x,y))^2] \tag{158}$$

The student can also be trained with a ranking-consistency loss to preserve relative orderings even if absolute score values differ:

$$\mathcal{L}_{\text{rank}} = -\log \sigma(r_S(x, y_w) - r_S(x, y_l)) \text{ whenever } r_T(x, y_w) > r_T(x, y_l) \tag{159}$$

Architecture Flow Diagram

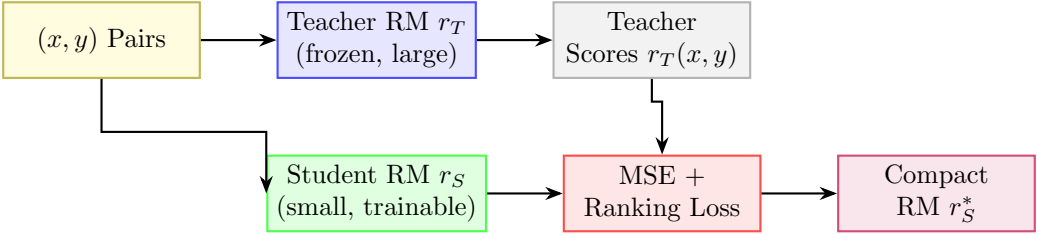


Figure 60: Reward Distillation: small student reward model trained via regression on large teacher reward scores.

State Transition Table

Property	Before	After
Reward model size	Large teacher	Small student
Inference latency	Slow	Fast
Score accuracy	High (teacher)	Near-teacher (distilled)
Training objective	Human preferences	Teacher score regression
Deployment cost	High	Low

Table 60: State properties before and after Reward Distillation.

Retrieval Distillation

Mathematical Foundation

Retrieval Distillation trains a student retriever q_S to match the relevance judgments of a larger teacher retriever q_T (or cross-encoder) without requiring access to teacher gradients. The student minimises the KL divergence between teacher and student retrieval distributions over a document set $\mathcal{C}$:

$$\mathcal{L}_{\text{RD}} = \mathbb{KL}[P_T(\cdot | q) || P_S(\cdot | q)], \quad P(d | q) \propto \exp(q_\phi(q)^\top e(d)) \tag{160}$$

where q_ϕ is the query encoder and $e(d)$ is the document embedding.

Architecture Flow Diagram

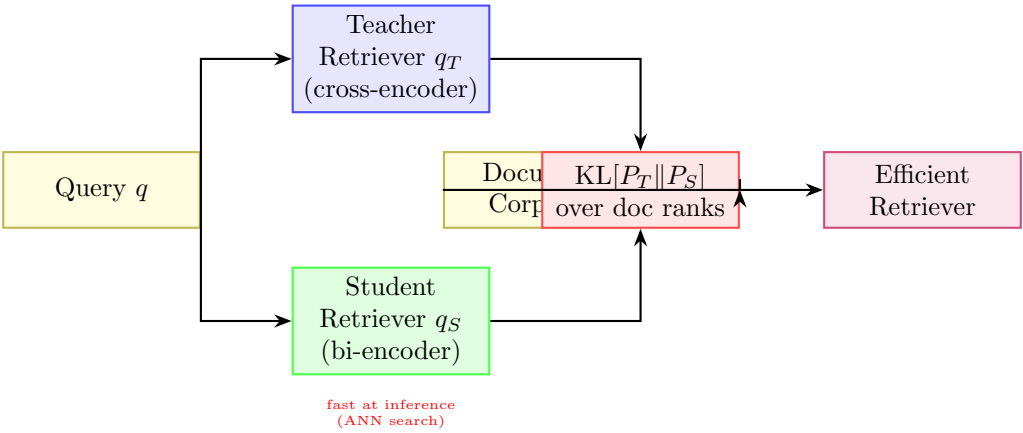


Figure 61: Retrieval Distillation: student bi-encoder trained to match teacher cross-encoder relevance distributions.

State Transition Table

Property	Before (Teacher)	After (Student)
Retriever type	Cross-encoder (slow)	Bi-encoder (fast ANN)
Query latency	$O(\mathcal{C})$	$O(\log \mathcal{C})$
Ranking quality	Very high	Near-teacher
Index requirement	None	Pre-computed embeddings
Training signal	Human judgments	Teacher soft scores

Table 61: State properties comparing teacher cross-encoder and student bi-encoder after retrieval distillation.

Part X

Model Compression

Quantization-Aware Training (QAT)

Mathematical Foundation

QAT simulates quantization during training using a straight-through estimator (STE) for gradients. For weight w , the quantized value at b bits with scale s :

$$\hat{w} = \text{clamp}(\text{round}(w/s), -2^{b-1}, 2^{b-1} - 1) \cdot s \quad (161)$$

During the backward pass the STE treats quantization as identity for gradient flow:

$$\frac{\partial \mathcal{L}}{\partial w} \approx \frac{\partial \mathcal{L}}{\partial \hat{w}} \cdot \mathbf{1}[|w| \leq s \cdot 2^{b-1}] \quad (162)$$

This makes the model aware of quantization error during training, yielding better accuracy than post-training quantization.

Architecture Flow Diagram

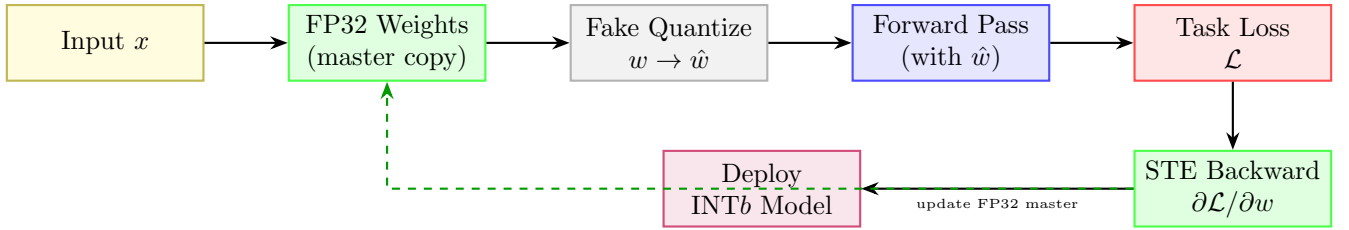


Figure 62: QAT: fake quantization in forward pass; STE allows gradients to flow to FP32 master weights.

State Transition Table

Property	Before (PTQ)	After (QAT)
Quantization awareness	None during training	Simulated in forward pass
Accuracy loss	1–5% degradation	<1% typical
Calibration data	Small set required	Full training data
Weight format	FP32	FP32 master, INTb deploy
Gradient flow	Normal	Via STE

Table 62: State properties comparing Post-Training Quantization and QAT.

Pruning

Mathematical Foundation

Pruning removes weights whose contribution to the model output is below a threshold, producing a sparse network. Magnitude-based unstructured pruning removes the fraction p of weights with the smallest absolute values:

$$m_i = \mathbf{1}[|w_i| \geq \tau_p], \quad W_{\text{sparse}} = W \odot m \tag{163}$$

where τ_p is chosen to achieve sparsity ratio $s = 1 - \|m\|_1/\|m\|_0$. Structured pruning removes entire heads or neurons (rows/columns) and is hardware-friendly:

$$\text{importance}(h) = \sum_{i \in h} |w_i| \cdot |\nabla_{w_i} \mathcal{L}| \tag{164}$$

Architecture Flow Diagram

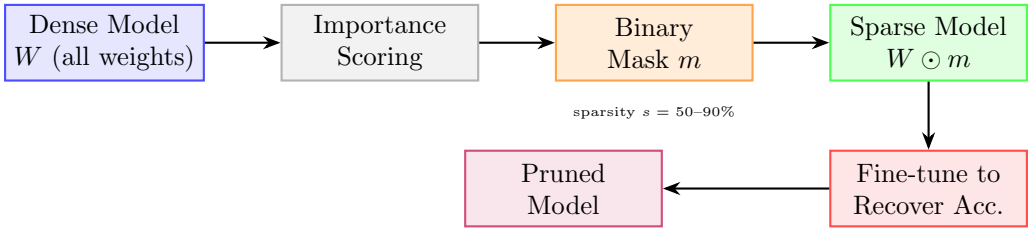


Figure 63: Pruning: importance scores determine binary mask; sparse model fine-tuned to recover accuracy.

State Transition Table

Property	Before	After
Model sparsity	0%	50–90%
Parameter count	$ \theta $	$(1 - s) \theta $
Inference FLOPs	Baseline	Reduced (structured)
Accuracy	Baseline	Slightly reduced
Memory footprint	Full	Reduced by s

Table 63: State properties before and after pruning.

Post-Training Quantization (PTQ)

Mathematical Foundation

PTQ converts a trained FP32 model to reduced precision (INT8, INT4) without additional training. Per-channel scale factors s_c are calibrated on a small unlabelled dataset $\mathcal{D}_{\text{cal}}$:

$$s_c = \frac{\max_{x \in \mathcal{D}_{\text{cal}}} |a_c(x)|}{2^{b-1} - 1}, \quad \hat{a}_c = \text{round}(a_c/s_c) \cdot s_c \quad (165)$$

GPTQ computes optimal quantization for each weight row by minimising the second-order reconstruction error using the Hessian:

$$\min_{\hat{W}} \|\hat{W}X - WX\|_F^2 \approx \sum_i (w_i - \hat{w}_i)^2 [H^{-1}]_{ii} \quad (166)$$

Architecture Flow Diagram

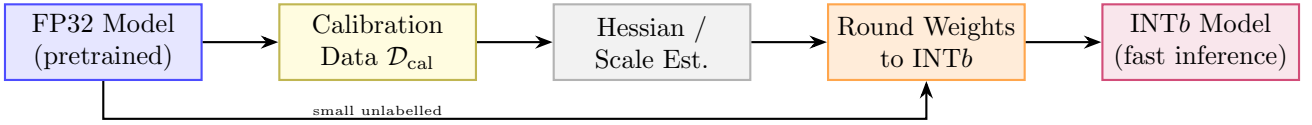


Figure 64: PTQ: calibration statistics guide per-channel scales; weights rounded to INT b without retraining.

State Transition Table

Property	Before	After
Weight precision	FP32 / BF16	INT8 / INT4
Memory usage	$4 \times n$ bytes	$1\text{--}0.5 \times n$ bytes
Retraining required	N/A	None (calibration only)
Accuracy loss	0%	0.5–3% (INT8), higher INT4
Calibration data	Not needed	Small sample (128–512 examples)

Table 64: State properties before and after Post-Training Quantization.

Part XI

Agentic Training

Tool-Use Training

Mathematical Foundation

Tool-Use Training teaches the model to generate structured tool invocations interleaved with reasoning (ReAct-style). The training data consists of trajectories $\tau = (t_1, a_1, o_1, \dots, t_N, a_N, o_N, y)$ where t_i is a thought, a_i is a tool call, and o_i is the observation. The model is trained to predict thought and action tokens conditioned on prior context and observations:

$$\mathcal{L}_{\text{tool}} = - \sum_{i=1}^N \sum_j \log P_{\theta}(t_{i,j} \mid \text{context}_{<i}) + \log P_{\theta}(a_{i,j} \mid \text{context}_{<i}, t_i) \quad (167)$$

Architecture Flow Diagram

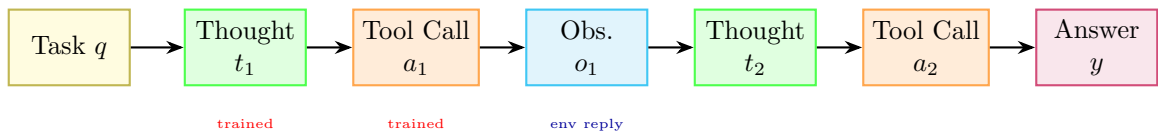


Figure 65: Tool-Use Training: thought-action-observation trajectory; model trained on thought and action tokens.

State Transition Table

Property	Before	After
Output format	Free text	Thought + tool call interleaved
Tool invocation	None	Structured API calls
Observation integration	N/A	Conditioned on tool results
Multi-step planning	Weak	Strong
Training data type	Static QA	Trajectory (thought, action, obs)

Table 65: State properties before and after Tool-Use Training.

Function Calling Training

Mathematical Foundation

Function Calling Training specifically trains the model to produce well-formed JSON function-call objects given a natural-language request and a tool schema. The model maps input x and schema $S = \{f_1, \dots, f_K\}$ to a structured call $c = \{\text{name: } f_k, \text{args: } \{a_1:v_1, \dots\}\}$:

$$\mathcal{L}_{\text{FC}} = -\log P_{\theta}(c \mid x, S) \tag{168}$$

where c is the ground-truth function call. Schema-aware token masking ensures the model only generates valid JSON that matches the function signature, reducing hallucination of non-existent arguments.

Architecture Flow Diagram

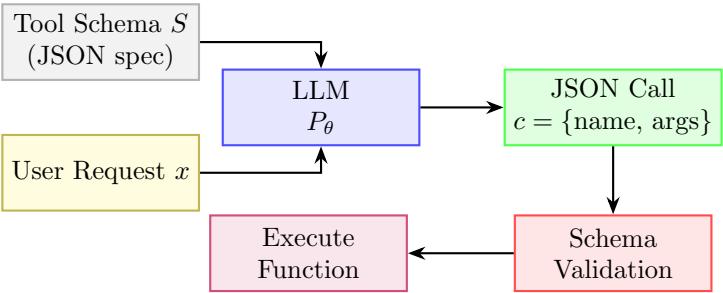


Figure 66: Function Calling Training: model receives request + schema, outputs structured JSON call validated against spec.

State Transition Table

Property	Before	After
Output format	Natural language	Structured JSON
Schema adherence	None	Enforced
Argument hallucination	High	Low
API integration	Manual parsing	Direct execution
Training data	Text completions	(request, schema, call) triplets

Table 66: State properties before and after Function Calling Training.

Multi-Agent Training

Mathematical Foundation

Multi-Agent Training trains a collection of specialised agents $\{\pi_1, \dots, \pi_K\}$ and an orchestrator π_0 to collaboratively solve complex tasks. The orchestrator decomposes a task T into sub-tasks $\{t_1, \dots, t_K\}$ and assigns them to specialists:

$$\pi_0 : T \rightarrow \{(t_k, k)\}_{k=1}^K, \quad \pi_k : t_k \rightarrow y_k, \quad y = \pi_0(\{y_k\}) \tag{169}$$

Training uses task-completion reward at the full-task level, distributed to agents via credit assignment:

$$r_k = r_{\text{task}} \cdot \mathbf{1}[\pi_k \text{ contributed to solution}] \tag{170}$$

Architecture Flow Diagram

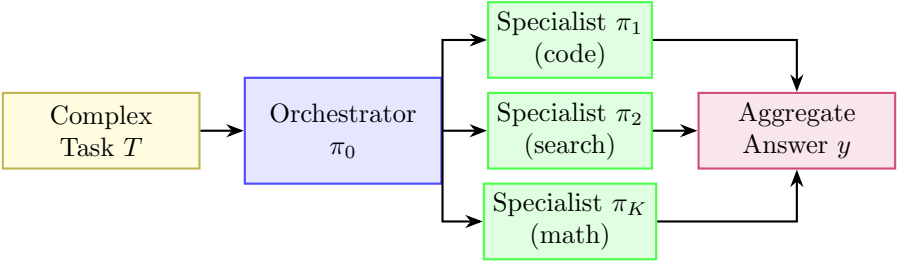


Figure 67: Multi-Agent Training: orchestrator decomposes task and assigns sub-tasks to specialised agents.

State Transition Table

Property	Before	After
Task handling	Single monolithic model	Orchestrator + specialists
Specialisation	None	Per-domain agents
Credit assignment	N/A	Per-agent contribution
Task complexity ceiling	Single-context limit	Decomposable to multiple contexts
Coordination	N/A	Learned delegation protocol

Table 67: State properties before and after Multi-Agent Training.

Part XII

Multimodal Training

Vision-Language Training

Mathematical Foundation

Vision-Language Training jointly trains a visual encoder f_v and language model f_l to process image-text pairs. A projection layer P aligns visual features to the LLM’s token embedding space:

$$v_{\text{tokens}} = P(f_v(I)), \quad \tilde{x} = [v_{\text{tokens}}; E(x_{\text{text}})] \tag{171}$$

Training uses a masked language modelling or auto-regressive loss on the text tokens conditioned on visual tokens:

$$\mathcal{L}_{\text{VL}} = - \sum_t \log P_{f_l}(x_t \mid v_{\text{tokens}}, x_{<t}) \tag{172}$$

Architecture Flow Diagram

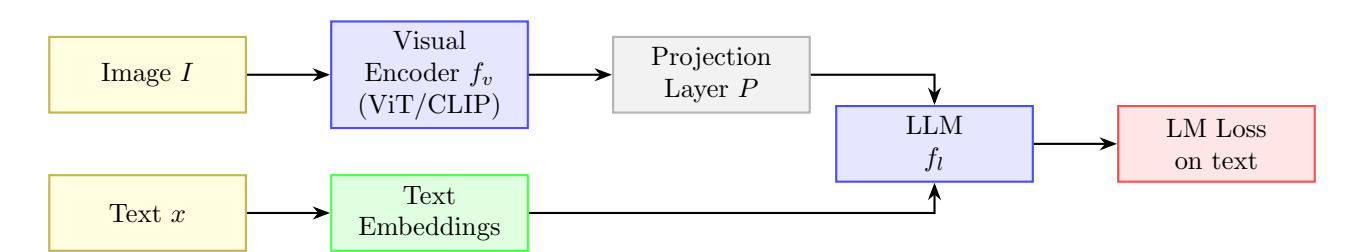


Figure 68: Vision-Language Training: visual encoder outputs projected to LLM token space; joint auto-regressive loss.

State Transition Table

Property	Before	After
Input modality	Text only	Text + images
Visual understanding	None	Strong (via encoder)
Projection layer	Not present	Trained to align modalities
Alignment quality	N/A	Measured by VQA, captioning
Training data	Text corpora	Image-caption / VQA pairs

Table 68: State properties before and after Vision-Language Training.

Grounding Training

Mathematical Foundation

Grounding Training teaches the model to link text spans to image regions (bounding boxes). Given a referring expression e and image I , the model predicts box coordinates (x_1, y_1, x_2, y_2) normalised to $[0, 1]^4$:

$$\hat{b} = f_{\theta}(I, e), \quad \mathcal{L}_{\text{GND}} = \text{L1}(\hat{b}, b^*) + \lambda(1 - \text{IoU}(\hat{b}, b^*)) \quad (173)$$

where b^* is the ground-truth box and IoU is intersection-over-union. The model generates boxes as text tokens (serialised coordinates), enabling unified seq2seq training.

Architecture Flow Diagram

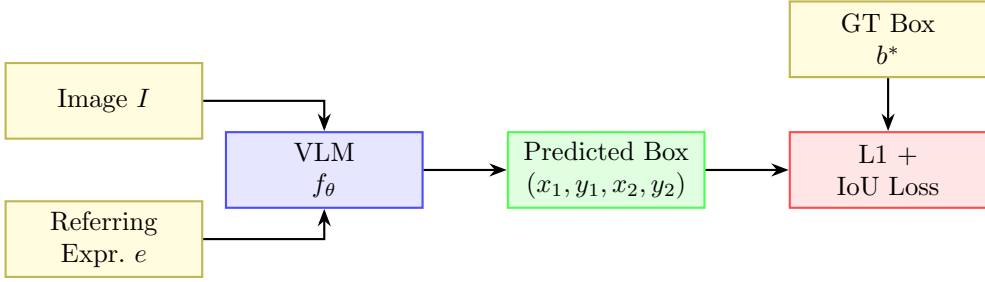


Figure 69: Grounding Training: VLM predicts bounding box coordinates from image and referring expression.

State Transition Table

Property	Before	After
Output type	Text description	Bounding box coordinates
Spatial understanding	Implicit	Explicit (pixel-level)
Loss type	Cross-entropy	L1 + IoU
Grounding ability	None	Text-to-region linking
Training data	Captions	(image, expression, bbox) triplets

Table 69: State properties before and after Grounding Training.

Part XIII

Knowledge Injection

Retrieval-Augmented Training

Mathematical Foundation

Retrieval-Augmented Training (RAT) trains the model to use retrieved documents $\mathcal{R}(x) = \{d_1, \dots, d_K\}$ as context. The model generates y conditioned on the query and retrieved set:

$$P_{\theta}(y \mid x, \mathcal{R}(x)) = \prod_t P_{\theta}(y_t \mid x, d_1, \dots, d_K, y_{<t}) \quad (174)$$

Training includes both retrieval-present and retrieval-absent examples to prevent over-reliance on context. A closed-book loss component ensures the model retains parametric knowledge:

$$\mathcal{L}_{\text{RAT}} = \lambda \mathcal{L}_{\text{open-book}} + (1 - \lambda) \mathcal{L}_{\text{closed-book}} \quad (175)$$

Architecture Flow Diagram

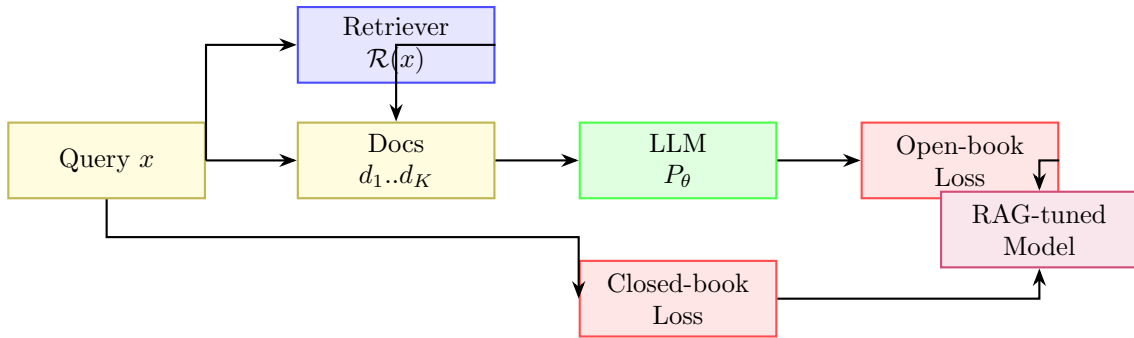


Figure 70: RAT: model trained on both retrieval-augmented and closed-book losses to balance open/parametric knowledge.

State Transition Table

Property	Before	After
Context awareness	Parametric only	Retrieved + parametric
Knowledge staleness	Fixed (training cutoff)	Dynamic (fresh retrieval)
Hallucination rate	Higher	Lower (grounded in docs)
Retriever dependency	None	Required at inference
Parametric retention	Full	Preserved (closed-book loss)

Table 70: State properties before and after Retrieval-Augmented Training.

Memory Tuning

Mathematical Foundation

Memory Tuning trains the model to store and retrieve episodic memories $\mathcal{M} = \{(k_i, v_i)\}$ in an external key-value store. The model learns to write memories via a write head w_θ and retrieve via dot-product attention:

$$\text{retrieve}(q) = \sum_i \text{softmax}(q^\top k_i / \sqrt{d}) \cdot v_i, \quad q = f_\theta^{\text{query}}(x) \quad (176)$$

Training uses a distractor task: memory-relevant queries should prefer correct memory entries, trained with a contrastive retrieval loss:

$$\mathcal{L}_{\text{mem}} = -\log \frac{\exp(q^\top k^+ / \tau)}{\sum_j \exp(q^\top k_j / \tau)} \quad (177)$$

Architecture Flow Diagram

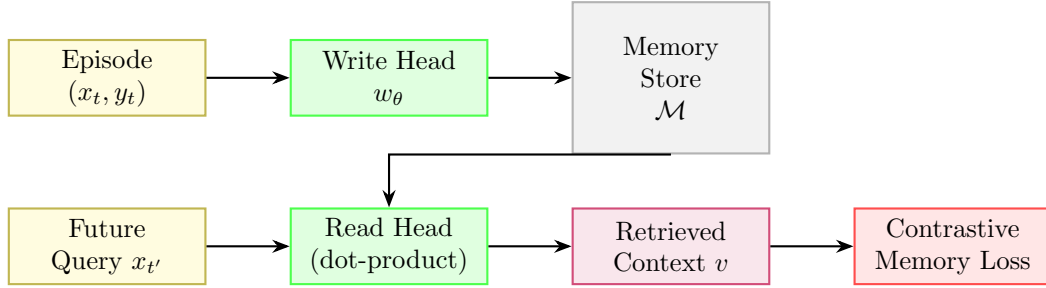


Figure 71: Memory Tuning: write head stores episodes; read head retrieves via attention; contrastive loss trains both.

State Transition Table

Property	Before	After
Memory type	Parametric only	Parametric + episodic KV store
Context window	Fixed T tokens	Unbounded (external memory)
Personalisation	None	Per-user episodic recall
Read/write mechanism	None	Learned attention-based
Long-term recall	Poor	Strong

Table 71: State properties before and after Memory Tuning.

Domain Knowledge Injection

Mathematical Foundation

Domain Knowledge Injection embeds structured domain knowledge (ontologies, knowledge graphs, fact databases) into the model. Entities e and relations r from a KG are embedded alongside token embeddings and injected via cross-attention:

$$h'_t = h_t + \text{CrossAttn}(h_t, \{e_i, r_{ij}\}_{(i,j) \in \mathcal{G}}) \quad (178)$$

Training uses a knowledge grounding loss that verifies generated text against retrieved facts:

$$\mathcal{L}_{\text{KI}} = \mathcal{L}_{\text{LM}} + \mu \mathcal{L}_{\text{fact-check}} \quad (179)$$

Architecture Flow Diagram

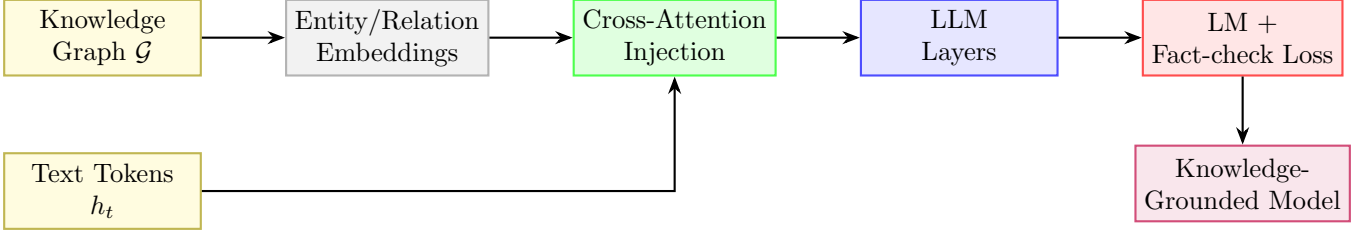


Figure 72: Domain Knowledge Injection: KG entity/relation embeddings injected into LLM via cross-attention.

State Transition Table

Property	Before	After
Knowledge source	Implicit (pretraining)	Explicit KG entities
Fact accuracy	Moderate	Significantly higher
Knowledge update	Retrain required	KG update sufficient
Cross-attention overhead	None	One extra module per layer
Domain coverage	General	Domain-specific + general

Table 72: State properties before and after Domain Knowledge Injection.

Part XIV

Deployment and Training Efficiency

Mixed Precision Training

Mathematical Foundation

Mixed Precision Training maintains FP32 master weights but computes forward and backward passes in FP16/BF16. A loss scaling factor S prevents underflow of small gradients:

$$\tilde{\mathcal{L}} = S \cdot \mathcal{L}, \quad g_{\text{FP32}} = \frac{1}{S} g_{\text{FP16}} \tag{180}$$

BF16 has the same exponent range as FP32 (8 bits) but fewer mantissa bits (7 vs 23), making it more numerically stable without loss scaling:

$$\text{memory}(\text{FP16}) = 2N \text{ bytes vs } \text{memory}(\text{FP32}) = 4N \text{ bytes} \tag{181}$$

Architecture Flow Diagram

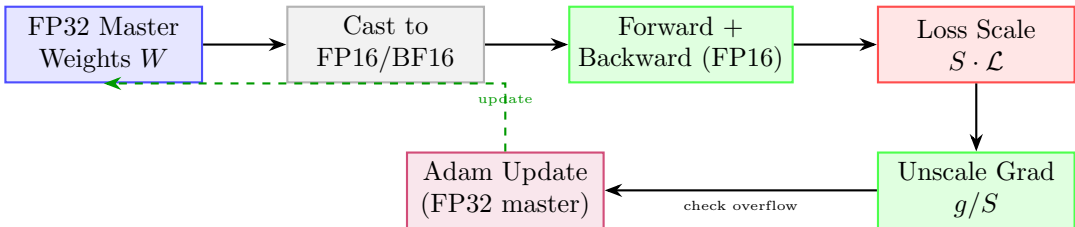


Figure 73: Mixed Precision Training: FP16 compute with FP32 master weights; loss scaling prevents gradient underflow.

State Transition Table

Property	Before (FP32)	After (Mixed FP16)
Compute precision	FP32	FP16/BF16
Memory footprint	4 <i>N</i> bytes	2 <i>N</i> + 4 <i>N</i> (half + master)
Training speed	Baseline	2–3× faster
Numerical stability	High	Managed via loss scaling
GPU tensor cores	Unused	Utilised (FP16/BF16 native)

Table 73: State properties before and after Mixed Precision Training.

Gradient Checkpointing

Mathematical Foundation

Gradient Checkpointing trades compute for memory by not storing all intermediate activations during the forward pass. Instead, activations at checkpoint nodes $\mathcal{C} \subset \{1, \dots, L\}$ are stored; all others are recomputed during the backward pass:

$$\text{Memory}_{\text{GC}} = O(\sqrt{L} \cdot d), \quad \text{vs } O(L \cdot d) \text{ without GC} \tag{182}$$

For L layers, optimal checkpointing every $\sqrt{L}$ layers minimises memory while adding one extra forward pass overhead:

$$\text{Compute}_{\text{GC}} \approx 2 \cdot \text{Compute}_{\text{standard}} \tag{183}$$

Architecture Flow Diagram

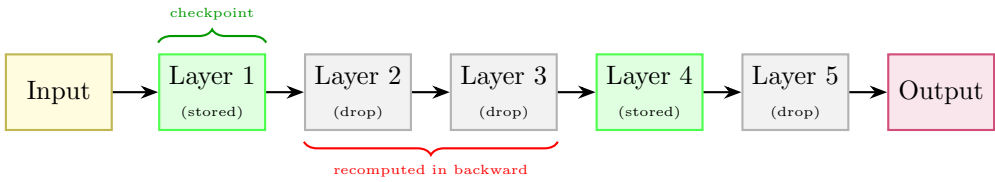


Figure 74: Gradient Checkpointing: only checkpoint activations stored; intermediate activations recomputed in backward.

State Transition Table

Property	Before	After
Activation memory	$O(L \cdot d)$	$O(\sqrt{L} \cdot d)$
Recomputation	None	Extra forward for non-checkpoints
Max batch size	Limited by activations	Larger (mem freed)
Compute overhead	Baseline	~30% more FLOPs
Large model feasibility	Memory OOM	Enabled

Table 74: State properties before and after Gradient Checkpointing.

Flash Attention

Mathematical Foundation

Flash Attention computes attention in IO-aware tiles that fit in SRAM, avoiding materialising the full $N \times N$ attention matrix in HBM. The standard attention complexity is $O(N^2d)$ in FLOPs and $O(N^2)$ in memory; Flash Attention maintains $O(N^2d)$ FLOPs but reduces memory to $O(N)$:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V \tag{184}$$

Computed in blocks of size $B_r \times B_c$ where $B_r = \lceil M/4d \rceil$, M = SRAM size, using online softmax to accumulate numerically stable output without full matrix materialisation.

Architecture Flow Diagram

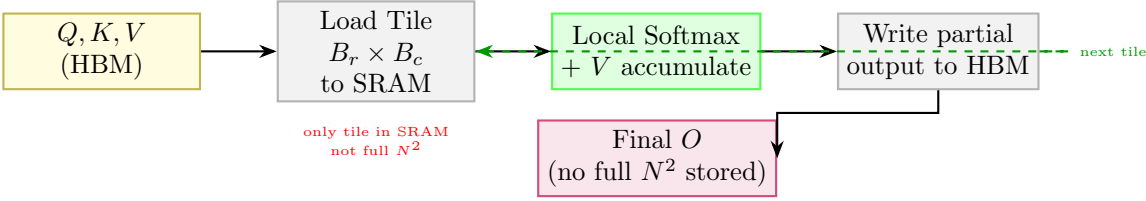


Figure 75: Flash Attention: tiled block-wise computation avoids materialising the full attention matrix in HBM.

State Transition Table

Property	Before (Standard)	After (Flash Attention)
Attention matrix memory	$O(N^2)$ HBM	$O(N)$ (tiles in SRAM)
HBM read/write passes	Multiple	Minimal (one pass)
Compute FLOPs	$O(N^2d)$	$O(N^2d)$ (same)
Speed (wall-clock)	Baseline	2–4× faster
Long-sequence feasibility	OOM at $N > 4096$	$N > 100,000$ feasible

Table 75: State properties comparing Standard and Flash Attention.

Fully Sharded Data Parallel (FSDP)

Mathematical Foundation

FSDP shards model parameters, gradients, and optimizer states across D devices, reducing per-device memory from $O(|\theta|)$ to $O(|\theta|/D)$. Before each layer's forward/backward, an All-Gather reconstructs the full parameter shard:

$$\theta_{\text{full}} = \text{AllGather}(\theta^{(1)}, \dots, \theta^{(D)}), \quad \theta_{\text{full}} \in \mathbb{R}^{|\theta|} \quad (185)$$

After the operation, θ_{full} is discarded. Gradient shards are reduced-scattered after the backward:

$$g^{(d)} = \text{ReduceScatter}(g_{\text{full}})[d], \quad \theta^{(d)} \leftarrow \theta^{(d)} - \eta g^{(d)} \quad (186)$$

Architecture Flow Diagram

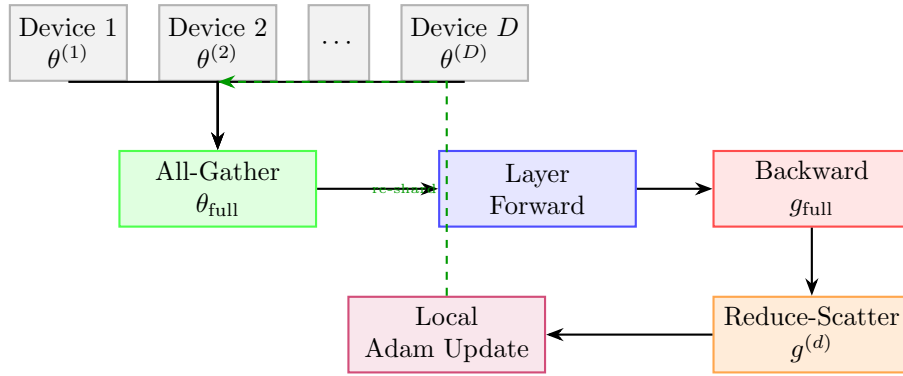


Figure 76: FSDP: parameters sharded across devices; All-Gather for compute, Reduce-Scatter for gradient aggregation.

State Transition Table

Property	Before (DDP)	After (FSDP)
Per-device param memory	$O(\theta)$	$O(\theta /D)$
Optimizer state memory	$O(\theta)$ per device	$O(\theta /D)$ per device
Communication pattern	AllReduce (gradients)	All-Gather + Reduce-Scatter
Max model size	GPU VRAM limited	$D \times$ larger
Training throughput	Baseline	Near-linear scaling

Table 76: State properties comparing DDP and FSDP.

DeepSpeed ZeRO

Mathematical Foundation

DeepSpeed ZeRO (Zero Redundancy Optimizer) progressively eliminates redundancy across three stages. For D devices and mixed-precision with Adam, the per-device memory at each stage:

$$\text{ZeRO-1: optimizer states sharded: } 4|\theta|/D + 6|\theta| \quad (187)$$

$$\text{ZeRO-2: + gradients sharded: } 4|\theta|/D + 4|\theta|/D + 4|\theta| \quad (188)$$

$$\text{ZeRO-3: + parameters sharded: } (4 + 4 + 4)|\theta|/D \quad (189)$$

ZeRO-3 achieves $12|\theta|/D$ bytes per device vs $16|\theta|$ bytes for full mixed-precision without sharding.

Architecture Flow Diagram

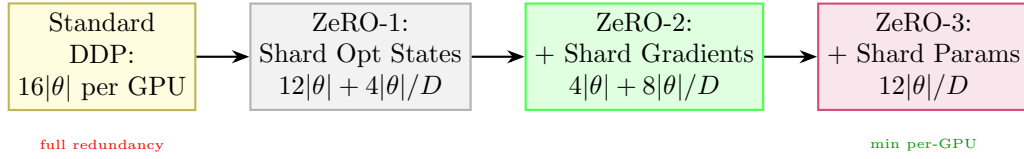


Figure 77: DeepSpeed ZeRO stages: each stage shards an additional component, progressively reducing per-GPU memory.

State Transition Table

Property	Standard DDP	ZeRO-3
Optimizer state memory	$8 \theta $ per device	$8 \theta /D$
Gradient memory	$4 \theta $ per device	$4 \theta /D$
Parameter memory	$4 \theta $ per device	$4 \theta /D$
Total per device	$16 \theta $	$16 \theta /D$
Max trainable size	GPU VRAM bound	$D \times$ larger

Table 77: Memory comparison between standard DDP and DeepSpeed ZeRO-3.